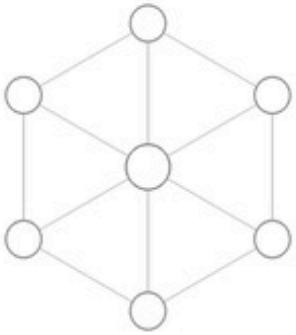
OPENSKILL IT STARTER

오픈스킬 IT 입문

MSA

Microservices Architecture



단계별로 시작하는 MSA

OPENSKILL BOOKS
오픈스킬북스

류재성, 최주호, 김주혁 지음

오픈스킬 IT 입문 - 단계별로 시작하는 MSA

초판 발행	2026년 10월 30일
지은이	류재성, 최주호, 김주혁
펴낸이	최주호
펴낸곳	오픈스킬북스
등록	제2026-000004호 (2026년 3월 20일)
주소	부산, 남구 오륙도로 85, 109동 1701호
전화	051-754-3648
이메일	getinthere@naver.com
ISBN	979-11-998558-1-6
정가	11,000원

© 2026 류재성, 최주호, 김주혁

이 책의 저작권은 지은이와 오픈스킬북스에 있습니다. 저작권법에 의해 보호를 받는 저작물이므로 무단 전재와 복제를 금합니다.

지은이 소개

류재성

전) (주)디오 ERP지원팀

현) (주)마음알지 시스템개발팀

현) 겟인데어 콘텐츠팀 프리랜서 / 메타코딩 유튜브 및 온라인 강의 운영자

'메타코딩' 출판 프로젝트 및 웹 강의 플랫폼 기획에 다수 참여

저서

AWS로 배우는 실전형 CI/CD 2판 (앤써북)

최주호

현) 겟인데어 대표

현) 유튜브 메타코딩 채널 운영

현) 지아이티아카데미 프리랜서 강사

전) 경남정보대학교 - 클라우드 학과 겸임교수

전) 패스트캠퍼스, 스파르타코딩, 인프런, 카카오캠퍼스 다수 강의 참여

전) 노마드랩 개발팀장

전) 군장병 공개 소프트웨어 역량 강화 - 국방부 장관상 수상

저서

AWS로 배우는 실전형 CI/CD 2판 (앤써북)

만들면서 배우는 플러터 앱 프로그래밍 (앤써북)

모두가 할 수 있는 플러터 UI 실전 (앤써북)

모두가 할 수 있는 플러터 UI 입문 (앤써북)

IoT 사물인터넷을 위한 라즈베리파이 5 정석 (앤써북)

JSPStudy의 JSP & Servlet 웹 프로그래밍 입문 + 활용 (앤써북)

그누위즈의 PHP&MySQL 웹 프로그래밍 입문 + 활용 (전면 개정판) (앤써북)

김주혁

전) (사)한국융합인재교육협회 교육팀장

현) (주)엠바스 마케팅&디자인 팀장

현) 겟인데어 콘텐츠팀 프리랜서 / 메타코딩 유튜브 및 온라인 강의 운영자

'메타코딩' 출판 프로젝트 및 웹 강의 플랫폼 기획에 다수 참여

머릿말

실무에서 비즈니스 요구사항을 시스템에 반영하다 보면, 시스템의 규모가 커짐에 따라 기존의 구조로는 유연한 대응이 어려워지는 시점이 옵니다. 저 역시 개발과 유지보수 과정에서 이러한 한계를 체감하며 자연스럽게 마이크로서비스 아키텍처(MSA)의 필요성을 느끼고 학습을 시작했습니다.

새로운 아키텍처를 도입하려면 낯선 기술과 방대한 개념들을 익혀야 합니다. 하지만 결국 핵심은 '시스템을 어떻게 효율적으로 나누고, 안정적으로 연결할 것인가'입니다. 앞으로의 개발 환경은 비즈니스 변화에 기민하게 대응할 수 있는 분산 시스템 설계 능력을 더욱 중요하게 요구할 것입니다.

이 책은 제가 MSA를 학습하고 정리한 과정의 결과물입니다. 거창한 이론이나 당장 쓰지 않을 복잡한 설정은 배제하고, 아키텍처를 이해하는 데 꼭 필요한 뼈대 지식만 담백하고 직관적으로 정리하는 데 집중했습니다.

새로운 아키텍처를 학습하며 실무 적용을 고민하는 분들에게, 이 책이 명확하고 실질적인 가이드가 되기를 바랍니다.

류재성

프롤로그

서버가 멈추다

밤 열한 시, 막 잠이 들려던 참에 휴대폰이 울렸습니다. 운영팀이었습니다.

운영팀 : "사이트가 전부 멈췄습니다. 확인 좀 해 주세요."

잠이 확 달아났습니다. 노트북을 열자 대시보드가 온통 에러 로그로 덮여 있었습니다. 저녁 일곱 시에 시작한 할인 이벤트로 트래픽이 몰리면서, 그 부하를 버티지 못한 서버가 다운된 것입니다.

서둘러 서버를 재시작했지만 바로 안정되지 않았고, 트래픽이 빠지기 시작한 새벽 한 시가 되어서야 겨우 안정을 되찾았습니다.

트래픽 때문에 사이트 전체가 멈춰 버리다니.

다음 날 아침, 팀장이 자리로 찾아왔습니다.

팀장 : "이번처럼 트래픽 때문에 사이트 전체가 다운되면 곤란해요. 재발 방지 대책 좀 찾아봐 줘요."

선뜻 답할 수 없었습니다. 지난번에도 트래픽이 몰렸을 때 서버를 늘려 봤지만, 비용과 노력만 들었을 뿐 같은 장애가 되풀이됐기 때문입니다. 결국 선배를 찾아갔습니다.

오픈이 : "선배님, 트래픽은 주문에만 몰렸는데 로그인이고 상품 조회고 할 것 없이 전부 멈춰 버렸어요. 이건 어떻게 해야 할까요?"

선배가 웃으며 대답했습니다.

선배 : "모든 기능이 한 서버에 다 뭉쳐 있어서 그래요. 백화점은 정전 한 번에 전 층이 같이 문을 닫지만, 길가의 독립 상점은 한 곳이 닫아도 옆 가게는 멀쩡하잖아요. **기능을 그 상점들처럼 독립된 서비스로 분리하면, 한 곳에 트래픽이 몰려 서버가 다운돼도 다른 기능은 멀쩡하거든요.** 이런 구조를 **MSA**라고 해요. 일단 나누는 것부터 시작해 봐요."

서비스를 나누다

오픈이 : "선배님, 그럼 어떤 것부터 시작하면 될까요?"

선배 : "우선 회원, 주문, 상품, 배달을 기능별로 서비스로 나눠요. 그리고 서비스끼리 전화를 거는 것처럼 직접 호출하게 만드는 거예요."

선배의 말대로 한 서버에 모여 있던 기능을 네 개의 서비스로 나눴습니다. 주문이 들어오면 주문 서비스가 상품 서비스에 재고를 줄여 달라고 요청하고, 이어서 배달 서비스에 배달 생성을 요청하도록 연결했습니다.

그런데 곧 문제에 부딪혔습니다. 중간에 에러가 나면 되돌릴 방법이 없었습니다. 예전처럼 데이터베이스가 하나일 때는 에러가 나면 전부 롤백하면 그만이었지만, 이제는 서비스마다 데이터베이스가 따로 나뉘어 있어 그럴 수가 없었습니다.

오픈이 : "선배님, 재고는 이미 줄였는데 배달이 실패하면 줄인 재고를 되돌릴 방법이 없어요. 어떻게 하죠?"

선배 : "자동으로 안 되니 직접 되돌려야죠. 배달에서 실패하면, 이전 단계인 상품 서비스에 '재고를 복구해 줘'라고 요청을 보내는 거예요. **한 단계씩 거꾸로 취소하는 겁니다.** 이걸 **보상 트랜잭션**이라고 불러요."

주문이 어디까지 진행됐는지 상태를 기록해 두고, 실패하면 이전 서비스로 취소 요청을 보내는 로직을 짰습니다. 단계마다 취소 코드를 일일이 붙이는 건 번거로웠습니다. 그래도 일부러 중간 단계를 실패시켜 보니, 앞서 끝난 단계가 한 단계씩 원래대로 되돌아갔습니다.

그렇게 기능별로 나눈 서비스들이 처음으로 하나의 흐름으로 함께 동작하기 시작했습니다.

구조를 다시 세우다

며칠 뒤, 팀장이 자리로 찾아왔습니다.

팀장 : "주문 기능을 테스트하려는데, 컨트롤러가 서비스에 직접 의존해서 가짜(Mock) 객체로 테스트할 수가 없어요. 이거 직접 의존하지 않게 구조 정리해 줘요."

곧바로 코드를 열어 봤습니다. 컨트롤러를 테스트하려면 서비스까지 함께 동작해야 했고, 가짜 객체를 넣으려 해도 컨트롤러 코드를 직접 고쳐야 했습니다. 고민에 빠진 채 다시 선배를 찾아가 코드를 보여 주며 물었습니다.

선배 : "지금은 결합도가 너무 높아서 그래요. 두 가지만 알면 돼요. 먼저 **구현이 아니라 인터페이스에 의존하게 만들어요.** 그래야 코드를 고치지 않고도 진짜 객체를 가짜 객체로 바꿔 테스트할 수 있어요. 이게 **클린 아키텍처**예요. 그리고 **흩어져 있는 핵심 비즈니스 로직은 도메인 안에 모아 둬요.** 그래야 바깥이 어떻게 바뀌어도 비즈니스 규칙은 흔들리지 않죠. 이걸 **도메인 주도 개발**이라고 해요."

클린 아키텍처와 도메인 주도 개발은 이름만 들어 봤지, 직접 적용해 보긴 처음이었습니다. 두 개념을 이해하고 나서, 익숙한 구조를 하나씩 뜯어고쳤습니다.

컨트롤러가 서비스에 직접 의존하는 대신, 그 사이에 USB 허브 같은 인터페이스를 두었습니다. 허브 뒤의 장치가 바뀌어도 컴퓨터는 아무 영향이 없듯, 컨트롤러는 '무엇을 할지'만 약속한 그 인터페이스에만 의존하고, '어떻게 할지'는 인터페이스를 구현한 서비스 쪽에 맡겼습니다.

검증 규칙도 정리했습니다. 재고 확인이나 주문 검증처럼 서비스 코드 곳곳에 흩어져 있던 규칙을 각 서비스의 도메인 객체 안으로 모았습니다. 규칙이 도메인 안에 모이니, 나중에 규칙이 바뀌어도 도메인만 고치면 됐습니다.

그러자 컨트롤러는 더 이상 서비스에 직접 의존하지 않게 됐습니다. 컨트롤러는 한 줄도 건드리지 않고, 인터페이스를 구현한 서비스만 가짜 객체로 바꿔 끼울 수 있었습니다.

비동기로 전환하다

동기 호출 방식으로 며칠을 운영하던 중, 또 다른 난관이 찾아왔습니다. 상품 서비스가 잠깐 죽은 사이에 들어온 주문이 전부 실패해 버렸습니다. 주문 서비스가 상품 서비스를 직접 호출하고 그 응답을 기다리는 구조라, 상대가 죽어 있으면 호출한 주문도 그대로 실패하는 것입니다.

오픈이 : "선배님, 상품 서비스 하나가 잠깐 죽었을 뿐인데 거기를 호출한 주문까지 다 같이 실패해 버리네요."

선배 : "직접 호출하지 말고, 메시지를 주고받는 비동기 방식으로 바꿔 봐요. **받는 쪽이 잠깐 죽어 있어도 메시지는 그대로 남아 있다가, 서버가 복구되면 그때 처리되거든요. Kafka**를 한번 써 봐요."

Kafka는 처음 다뤄 보는 도구였습니다. 메시지를 발행하고 구독하는 방식을 익힌 뒤, 서비스끼리 직접 호출하던 것을 하나씩 걷어내고 Kafka 메시지로 바꿨습니다.

오픈이 : "그런데 이렇게 메시지로 주고받으면, 중간에 실패했을 때 보상 트랜잭션은 누가 관리하나요?"

선배 : "전체 흐름을 관리하는 지휘자를 하나 두면 돼요. 각 서비스는 자기 일만 하고 결과만 보고하고, 그 지휘자가 어디까지 됐는지에 따라 다음 단계를 진행시키거나, 실패하면 이미 끝난 단계만 되돌리는 거죠."

선배 말대로 전체 흐름을 관리할 **오케스트레이터** 서비스를 만들었습니다. 각 서비스는 자기 일을 끝내면 결과만 메시지로 보고했고, 오케스트레이터는 그 보고를 받아 다음 단계를 진행시키거나, 실패하면 이미 끝난 단계를 되돌렸습니다.

이제 상품 서비스가 잠깐 멈춰도 주문은 메시지로 남아 있다가, 복구되면 하나씩 처리됐습니다. 흐름 중간이 실패해도 오케스트레이터가 끝난 단계만 되돌렸습니다.

실시간으로 알리다

며칠 뒤, 베타 테스터로 새 시스템을 써 본 동료가 떨떠름한 얼굴로 찾아왔습니다.

동료 : "어제 물건을 주문했는데, 화면이 계속 '처리 중'이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 보고서야 완료된 걸 알았어요."

확인해 보니 서버들끼리는 Kafka로 메시지를 주고받으며 처리를 끝냈지만, 정작 그 사실을 사용자에게는 알리지 않고 있었습니다. 사용자는 처음 받은 '처리 중' 상태에 그대로 멈춰 있었고, 결과를 보려면 직접 주문 내역을 다시 열어야 했습니다.

오픈이 : "선배님, 서버에서는 주문 처리가 완료됐는데 사용자는 처음 '처리 중' 응답만 받아서 끝난 걸 알 수가 없어요. 이건 어떻게 해결하죠?"

선배 : "처리가 끝난 순간 사용자에게 바로 알려 줘야 해요. 서버가 주문 완료 시점을 감지해서, 사용자 화면으로 먼저 말을 걸 수 있게 **WebSocket**을 붙여 봐요."

선배의 말을 듣고 코드를 다시 보니 어긋난 곳이 두 군데였습니다. 주문은 배달이 만들어지는 순간 곧바로 완료로 처리됐고, 정작 그 완료를 사용자에게는 알리지 않았습니다.

먼저 배달이 실제로 끝나는 순간에 주문을 완료 처리하도록 바꿨습니다. 남은 건 그걸 사용자에게 알리는 일이었습니다. WebSocket을 들여다보니 늘 쓰던 HTTP와는 정반대였습니다. HTTP가 한 번 주고받으면 끊기는 편지라면, **WebSocket은 한 번 연결하면 계속 이어지는 전화였습니다.** 연결이 살아 있으니 서버는 변화가 생기는 순간 바로 알릴 수 있었습니다. 그래서 주문이 완료되면 서버가 WebSocket으로 사용자 화면에 알림을 보내도록 연결했습니다.

수정된 코드로 직접 주문을 넣어 보았습니다. 새로고침을 누르지 않았는데도, '처리 중'이던 화면이 '주문 완료'로 바뀌었습니다.

동료 : "이제 새로고침 없이도 주문이 끝난 걸 바로 알 수 있겠네요."

완성된 시스템을 선배에게 보여 주었습니다.

오픈이 : "처음 서버가 멈췄던 밤엔 정말 막막했는데, 한 단계씩 부딪히며 오다 보니 결국 해내게 되네요."

선배는 흐뭇한 표정으로 답했습니다.

선배 : "처음부터 모든 정답을 알고 시작하는 사람은 없어요. 부딪히고, 고치고, 다시 만들다 보면 되는 거예요."

목차

머릿말 1

프롤로그 2

챕터 1. MSA란 무엇인가? 7

1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까? 10

1.1.1 처음에는 아무 문제가 없었다 10

1.1.2 성장하면서 균열이 생긴다 10

1.2 마이크로서비스 - 역할을 나눈다 11

1.2.1 백화점 vs 개별 상점 11

1.3 시스템과 핵심 과제 13

1.3.1 쇼핑몰 주문 시스템 13

1.3.2 서비스 간 요청의 두 가지 방식 13

1.3.3 분산 트랜잭션 — 이 책의 핵심 과제 15

1.4 이 책의 학습 흐름 16

챕터 2. 동기식 MSA 구현 18

2.1 공통 설정 - 모든 서비스가 공유하는 뼈대 21

2.2 회원 서비스 - JWT로 로그인하다 22

2.2.1 클래스 구성 22

2.3 상품 서비스 - 재고를 관리하다 23

2.3.1 클래스 구성 23

2.4 배달 서비스 - 배달을 생성하고 취소하다 23

2.4.1 클래스 구성 24

2.5 주문 서비스 - 보상 트랜잭션의 현장 24

2.5.1 클래스 구성 25

2.5.2 보상 트랜잭션 흐름 25

2.5.3 adapter - 외부 서비스 호출 설정 26

2.5.4 OrderService - 보상 트랜잭션의 핵심 27

2.6 Docker Compose - 네 개의 서비스를 한 번에 실행하다 28

2.6.1 서비스 실행 29

2.6.2 Hoppscotch와 인터셉터 설정 30

2.6.3 시나리오 1: 정상 주문 31

2.6.4 시나리오 2: 재고 부족 35

2.6.5 시나리오 3: 주소 누락 35

챕터 3. 도메인을 중심으로 38

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로 41

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브 42

3.3 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로 44

3.4 UseCase 인터페이스 + 도메인 캡슐화 도입 45

3.4.1 UseCase 인터페이스 정의 45

3.4.2 엔티티의 비즈니스 로직 — DDD의 핵심 45

3.4.3 OrderService - 인터페이스 구현 46

3.4.4 OrderController 수정 46

3.5 Gateway와 MySQL 인프라 47

3.5.1 Nginx - API Gateway 라우팅 47

3.5.2 MySQL - 데이터베이스 인프라 48

3.6 Kubernetes - YAML로 선언하는 배포 49
3.6.1 리소스 구조 설계 49
3.7 Minikube - 실행 및 결과 확인 51
3.7.1 Minikube 시작 51
3.7.2 이미지 빌드 51
3.7.3 배포 순서 52
3.7.4 배포 상태 확인 53
3.7.5 서비스 접근 54

챕터 4. 비동기 MSA 57
4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다 60
4.1.1 카운터 대기 vs 진동벨 61
4.2 Kafka - 메시지를 전달하는 우체국 61
4.2.1 프로듀서·토픽·컨슈머 61
4.2.2 컨슈머 그룹 63
4.3 Orchestration Saga - 지휘자가 흐름을 조율하다 65
4.3.1 이번 챕터에서 사용하는 토픽 맵 66
4.3.2 주문 요청 성공 흐름 66
4.3.3 주문 요청 실패 흐름 (보상 트랜잭션) 69
4.4 Kafka로 주고받기 - 발행과 구독 70
4.4.1 발행 - 토픽에 메시지를 넣는다 70
4.4.2 orchestrator - 흐름을 조율하는 코드 72
4.4.3 구독 - 토픽의 메시지를 받는다 72
4.5 Kubernetes - Kafka와 orchestrator 배포 73
4.6 실행 및 결과 확인 74
4.6.1 이미지 빌드 74
4.6.2 배포 순서 75
4.6.3 서비스 접근 76
4.6.4 비동기 흐름 테스트 77
4.6.5 보상 트랜잭션 확인 - 품절 상품 주문 78
4.7 더 알아보기 - Outbox 패턴 80

챕터 5. 실시간 알림 83
5.1 웹소켓 - 폴링의 한계를 넘다 86
5.1.1 폴링 vs 푸시 86
5.1.2 웹소켓 - 실시간 양방향 통신 87
5.2 배달 완료 - 생성과 완료를 분리한다 88
5.3 웹소켓 연결 흐름 89
5.3.1 브라우저와 주문 서비스의 연결 - HTTP에서 웹소켓으로 89
5.3.2 브라우저의 채널 구독 - 명부에 등록 90
5.3.3 주문 서비스의 알림 발송 - 같은 채널 찾아 전달 90
5.4 배달 서비스 - 배달 완료 API 91
5.4.1 createDelivery 수정 - 배달 생성·완료 분리 91
5.4.2 completeDelivery 추가 - 배달 완료 API 92
5.5 orchestrator - 배달 완료 이벤트 처리 추가 92
5.6 주문 서비스 - STOMP로 실시간 Push 구현 93
5.6.1 웹소켓 설정 93
5.6.2 주문 완료 시 알림 발송 94
5.7 프론트엔드 연결 94
5.7.1 업그레이드 헤더 전달 94
5.7.2 클라이언트 - STOMP 구독 95
5.8 전체 시스템 통합 테스트 96

5.8.1 Kubernetes 리소스 정의 96
5.8.2 이미지 빌드 97
5.8.3 배포 97
5.8.4 서비스 접근 98
5.8.5 통합 테스트 시나리오 98

에필로그 **107**

마치며 **108**

CHAPTER

1

MSA란 무엇인가?

이번 챕터가 끝나면

- **모놀리식**의 한계를 이해할 수 있습니다.
- **마이크로서비스**가 역할을 나눠 그 한계를 푸는 방식을 이해할 수 있습니다.
- 서비스를 나누며 떠오르는 핵심 과제, **분산 트랜잭션**을 이해할 수 있습니다.

챕터 1. MSA란 무엇인가?

밤 열한 시, 막 잠이 들려던 참에 휴대폰이 울렸습니다. 운영팀이었습니다.

운영팀 : "사이트가 전부 멈췄습니다. 확인 좀 해 주세요."

잠이 확 달아났습니다. 노트북을 열자 대시보드가 온통 에러 로그로 덮여 있었습니다. 저녁 일곱 시에 시작한 할인 이벤트로 트래픽이 몰리면서, 그 부하를 버티지 못한 서버가 다운된 것입니다.

서둘러 서버를 재시작했지만 바로 안정되지 않았고, 트래픽이 빠지기 시작한 새벽 한 시가 되어서야 겨우 안정을 되찾았습니다.

트래픽 때문에 사이트 전체가 멈춰 버리다니.

다음 날 아침, 팀장이 자리로 왔습니다.

팀장 : "이번처럼 트래픽 때문에 사이트 전체가 다운되면 곤란해요. 재발 방지 대책 좀 찾아봐요."

오픈이는 바로 답을 하지 못했습니다. 지난번에 트래픽이 몰렸을 때도 서버를 증설해 봤지만, 비용과 노력만 잔뜩 들었을 뿐 같은 장애가 되풀이됐습니다. 결국 선배를 찾아갔습니다.

오픈이 : "선배님, 이번에 트래픽이 몰렸다고 사이트가 통째로 죽어버렸는데요. 이거 어떻게 해야 할까요?"

선배 : "모든 기능이 한 서버에 다 뭉쳐 있어서 그래요. 한쪽에 부하가 걸리면 전체가 다 같이 멈추는 거죠. 이럴 때는 기능을 독립된 서비스로 분리해야 해요. 그러면 한 곳에 트래픽이 몰려 서버가 다운되더라도, 다른 기능은 멀쩡하거든요."

준비하기. 챕터 2부터 시작될 실습을 위해 미리 준비

이 챕터는 개념만 다루므로 직접 코드를 작성하지 않습니다. 챕터 2부터 실습이 시작되니, 그전에 도구 설치와 레포 위치를 미리 확인해 두세요.

1. 실습 환경

도구	사용 챕터	설치 주소
Docker Desktop	챕터 2~	https://www.docker.com/products/docker-desktop/

도구	사용 챕터	설치 주소
Minikube	챕터 3~	https://minikube.sigs.k8s.io/
Hoppscotch (브라우저 확장)	챕터 2~	https://hoppscotch.io/

2. 챕터별 소스 코드

실습 코드는 두 레포로 제공됩니다.

레포	용도	주소
start	직접 작성하는 예제(실습용). 클론해서 진행	`https://github.com/metacoding-12-msa/start`
final	완성된 전체 코드. 막히면 참고	`https://github.com/metacoding-12-msa/final`

두 레포 모두 챕터별 폴더로 구성됩니다.

챕터	폴더	주제
챕터 2	`ex01`	동기 REST + 보상 트랜잭션
챕터 3	`ex02`	DDD + 클린 아키텍처 + Kubernetes
챕터 4	`ex03`	Kafka + Orchestration Saga
챕터 5	`ex04`	WebSocket 실시간 알림

챕터 2부터 start 레포를 클론해 해당 폴더에서 실습하고, 막히면 final 레포의 같은 폴더에서 완성 코드를 확인하세요.

1.1 모놀리식 - 쇼핑몰을 하나의 서버로 만들면 어떻게 될까?

1.1.1 처음에는 아무 문제가 없었다

백화점을 떠올려 보세요. 수십 개의 매장이 한 건물 안에 모여 있습니다. 고객은 한 곳에서 모든 것을 해결할 수 있습니다. 백화점 입장에서는 전기·냉방·보안·고객 데이터를 한 곳에서 통합 관리할 수 있습니다. 이 구조는 단순하고 효율적입니다.



그림 1-1. 백화점 - 모든 매장이 한 건물에 모여 있는 구조

소프트웨어에서도 동일하게 적용됩니다. 처음에는 **하나의 서버에 모든 기능을 넣는 모놀리식(Monolithic) 구조**가 단순하고 관리가 편합니다.

1.1.2 성장하면서 균열이 생긴다

한 공간에 모든 매장이 모여 있는 백화점은 시간이 지나면서 문제가 보이기 시작합니다. 한 매장에 화재가 발생하면 전 층이 함께 대피해야 합니다. 또 정전이 발생하면 건물 전체가 함께 멈춥니다.

이 약점은 소프트웨어의 모놀리식에서도 그대로 나타납니다.



그림 1-2. 모놀리식 쇼핑몰 - 모든 기능이 하나의 서버에

모든 기능이 한 서버에서 함께 돌아가다 보니, **한 곳의 문제가 곧 전체의 문제가 됩니다.** 주문에 트래픽이 몰리면 회원·상품·배달까지 함께 느려지고, 특정 기능만 확장하고 싶어도 서버 전체를 확장해야 합니다. 또 작은 수정에도 전체를 재배포해야 합니다.

하나의 서버에 다 모여 있으니, 어젯밤 서버가 전부 같이 멈춘 거였구나.

1.2 마이크로서비스 - 역할을 나눈다

1.2.1 백화점 vs 개별 상점

개별 상점 방식은 백화점과 구조 자체가 다릅니다. 각 매장이 독립된 건물로 운영되어, 자신만의 전기·냉방·입구를 가집니다. 덕분에 한 매장에 화재가 발생하거나 문을 닫아도, 다른 매장은 그대로 영업할 수 있습니다.



그림 1-3. 개별 상점 - 각 매장이 독립된 건물로 운영되는 구조

개별 상점처럼 하나의 큰 서버 대신 **기능별로 서비스를 분리한 구조가 MSA(Microservice Architecture)** 입니다.



그림 1-4. 마이크로서비스 쇼핑몰 - 기능별로 서비스를 분리

서비스를 분리하면 각 서비스는 독립적으로 배포하고, 독립적으로 확장할 수 있습니다. 그래서 주문 서비스에 문제가 발생해도 다른 서비스는 영향을 받지 않습니다.

1.3 시스템과 핵심 과제

문제를 이해했으니, 이제 만들어볼 시스템을 설계해 보겠습니다.

1.3.1 쇼핑몰 주문 시스템

이 책의 쇼핑몰 주문 시스템은 4개의 마이크로서비스로 구성됩니다.

서비스	포트	역할
주문 서비스	8081	주문 생성·조회·취소 (핵심)
상품 서비스	8082	상품 목록, 재고 조회 및 증감
회원 서비스	8083	로그인, JWT 발급, 사용자 조회
배달 서비스	8084	배달 생성·조회·취소

주문 서비스를 중심으로 상품 서비스와 배달 서비스가 연결됩니다. 사용자가 주문하면 주문 서비스가 상품 서비스의 재고를 차감하고, 배달 서비스에 배달을 생성합니다.

1.3.2 서비스 간 요청의 두 가지 방식

각 서비스가 분리되면 서비스 사이의 통신이 별도로 필요합니다. 이 책에서는 두 가지 방식으로 구현합니다.

방식 1. 직접 호출로 동기적 처리

첫 번째는 **각 서비스가 다른 서비스를 직접 호출하는 방식**입니다. 주문 서비스가 상품 서비스에 "재고 줄여줘"라고 요청한 후 응답이 올 때까지 기다리고, 응답이 돌아오면 다시 배달 서비스를 호출합니다.

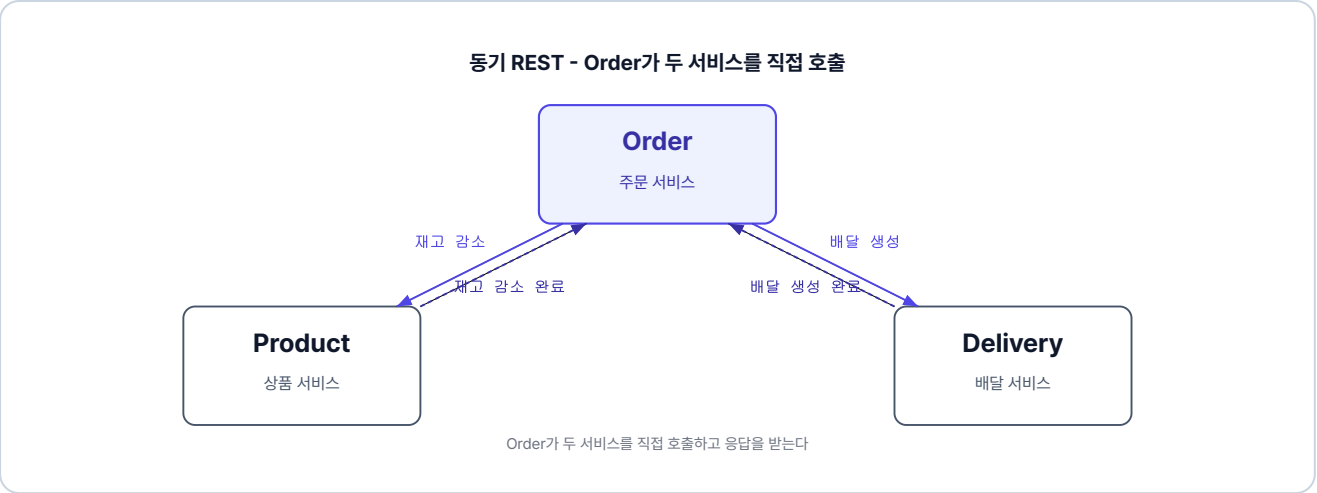


그림 1-5. 동기 REST - Order가 두 서비스를 직접 호출

이 방식은 구현이 단순합니다. 다만 호출한 서비스가 응답할 때까지 멈춰 있는 동기적 호출 방식이라, 한 단계가 지연되면 다음 단계도 지연됩니다.

방식 2. 메시지로 비동기 전환

다음은 서비스끼리 직접 호출하지 않고, **메시지로 요청을 주고받는 방식**입니다. 한 서비스가 메시지를 발행하면, 다른 서비스가 메시지를 받아서 처리합니다.



그림 1-6. 비동기 메시지 - 서비스를 분리하는 구조

발행한 서비스는 응답을 기다리지 않고 바로 다음 작업으로 넘어가므로, 한 서비스에서 응답이 지연되어도 다른 서비스는 지연되지 않습니다.

두 방식을 학습하며 각각의 장단점을 알아보겠습니다.

1.3.3 분산 트랜잭션 — 이 책의 핵심 과제

서비스를 분리하면 곧장 새로운 문제가 생깁니다. 모놀리식에서는 주문·재고·배달을 **하나의 트랜잭션으로 묶을 수 있기 때문에** 중간에 실패하면 전부 **자동 롤백**됩니다.

그런데 MSA에서는 각 서비스가 **독립된 데이터베이스**를 가집니다. 트랜잭션은 하나의 데이터베이스 안에서만 동작하므로, 서로 다른 DB에 걸친 작업은 하나의 트랜잭션으로 묶을 수 없습니다. 이렇게 여러 DB에 걸친 작업을 하나로 묶어야 하는 상황을 **분산 트랜잭션**이라고 합니다.

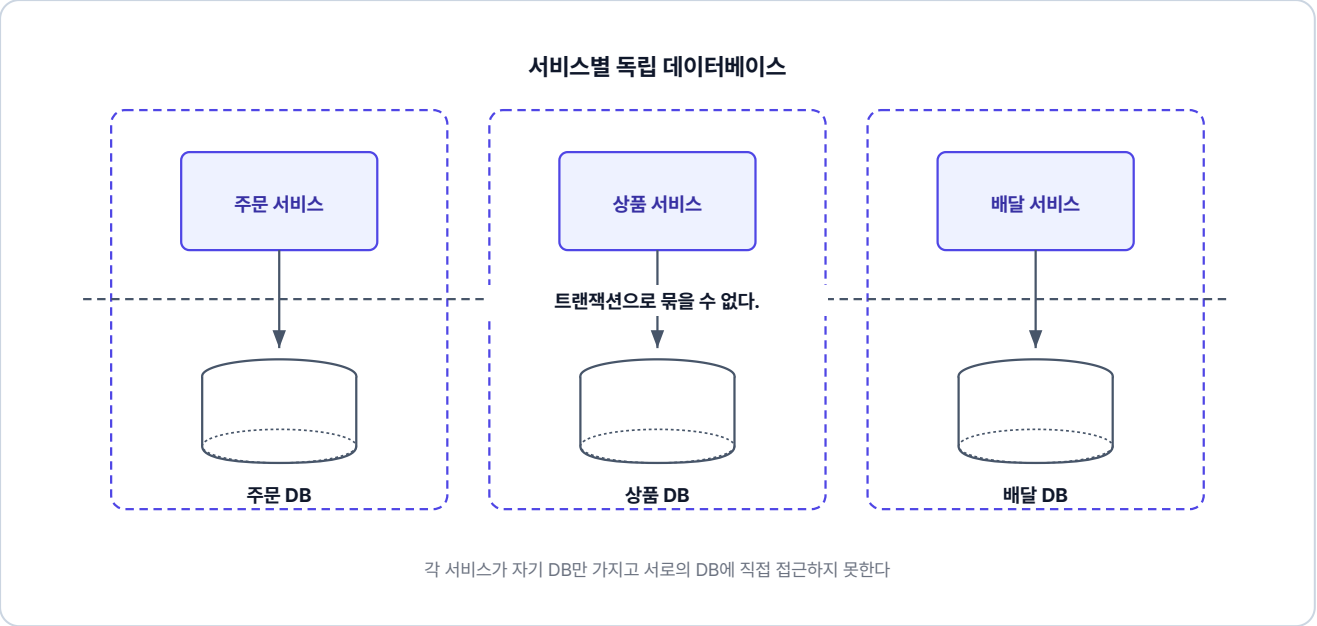


그림 1-7. 서비스별 독립 데이터베이스

> **분산 트랜잭션(Distributed Transaction)이란?** 여러 독립된 데이터베이스에 걸친 작업을 하나의 논리적 단위로 처리해야 하는 상황입니다. MSA에서는 서비스마다 DB가 분리되어 있으므로 단일 트랜잭션이 불가능하고, 별도의 전략이 필요합니다.

오픈이 : "선배님, 재고는 줄였는데 배달이 실패하면 줄인 재고를 원복해야 하잖아요. 이런 걸 되돌리는 방법은 없나요?"

선배 : "자동으로 안 되니 직접 되돌려야 해요. 재고를 줄였으면 원복하고, 배달을 만들었으면 취소하고. 한 단계씩 거꾸로 되돌리는 거예요. 이걸 **보상 트랜잭션**이라고 해요."

> **보상 트랜잭션(Compensating Transaction)이란?** 중간에 실패가 나면 이미 끝낸 작업을 역순으로 돌려 원래 상태로 되돌리는 방법입니다.

서비스를 분리하고, 분산 트랜잭션을 해결하는 것이 MSA의 과제입니다.

1.4 이 책의 학습 흐름

이 책은 하나의 시스템이 단계별로 진화하는 여정입니다. 각 챕터는 이전 챕터의 한계를 해결하는 방식으로 진행됩니다.



그림 1-8. 이 책의 학습 흐름

각 챕터에서 다루는 내용은 다음과 같습니다.

- 챕터 2에서는 각 서비스가 동기적으로 직접 호출하고, 중간에 실패하면 보상 트랜잭션으로 되돌립니다.
- 챕터 3에서는 도메인과 클린 아키텍처를 중심으로 서비스 구조를 다시 설계합니다.
- 챕터 4에서는 동기 방식을 메시지를 통한 비동기 방식으로 전환합니다.
- 챕터 5에서는 처리가 끝난 순간 사용자에게 실시간으로 알립니다.

이 책은 코드 작성보다 전체적인 개념과 흐름을 이해하고, 단계별로 실습하며 익히는 것을 목표로 하고 있습니다. 한 단계씩 학습하며 MSA의 큰 흐름을 따라가 보겠습니다.

이것만은 기억하자

- → **모놀리식**은 처음에는 단순하지만, 서비스가 커지면 배포·장애·확장 문제가 생깁니다.
- → **마이크로서비스**는 기능별로 서비스를 분리하여 각자 독립적으로 배포하고 확장할 수 있게 합니다.
- → 각 서비스의 DB가 분리되어 있어 단일 트랜잭션으로 묶을 수 없습니다. 이것이 MSA의 핵심 과제인 **분산 트랜잭션**입니다.
- → 분산 트랜잭션은 서비스끼리 **직접 호출·보상**하는 방식과, **메시지로 비동기 통신**하는 방식으로 학습합니다.

CHAPTER

2

동기식 MSA 구현

서비스를 연결하다

이번 챕터가 끝나면

- 여러 서비스를 **REST**로 직접 호출해 주문 흐름을 동기로 잇는 구조를 이해할 수 있습니다.
- 실패 시 앞 작업을 되돌리는 **보상 트랜잭션**을 이해할 수 있습니다.

챕터 2. 동기식 MSA 구현 - 서비스를 연결하다

오픈이 : "선배님, 그럼 어떤 것부터 시작하면 될까요?"

선배 : "우선 회원, 주문, 상품, 배달, 이렇게 기능별로 서비스를 나누는 거예요. 그리고 서비스끼리 전화를 거는 것처럼 직접 호출하는 거죠."

방향이 정해졌으니 이제 만들어 보겠습니다.

이번 챕터의 핵심은 주문 서비스입니다. 주문 서비스에서 주문 요청이 들어오면, 주문 서비스가 상품 서비스와 배달 서비스를 직접 호출하고 응답하는 흐름을 따라가 보겠습니다.

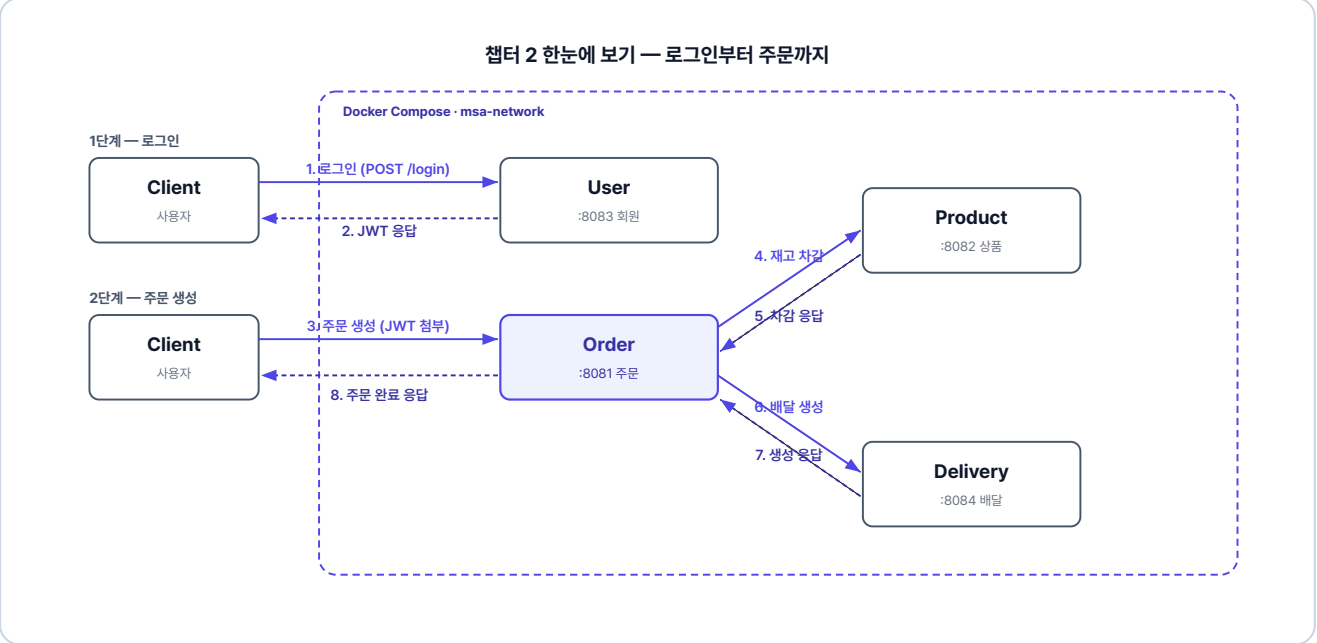


그림 2-1. 챕터 2 한눈에 보기 - 로그인부터 주문까지

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex01
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex01` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex01 디렉토리

```
ex01/
├── delivery/           # 포트 8084
├── docker-compose.yml  # 전체 서비스 실행
├── order/              # 포트 8081
├── product/            # 포트 8082
└── user/               # 포트 8083
```

각 서비스 내부는 동일한 구조입니다. 주문 서비스 기준으로 보여드리며, 회원/상품/배달 서비스도 같은 구조입니다.

주문 서비스 패키지 구조

```
src/main/java/com/metacoding/order/
├── adapter/                                # 주문 서비스에만 존재
│   ├── DeliveryClient.java                 # [참고] 배달 서비스 호출
│   ├── dto/                                # 어댑터용 DTO
│   └── ProductClient.java                  # [참고] 상품 서비스 호출
├── core/
│   ├── config/
│   │   ├── RestClientConfig.java           # [참고] JWT 헤더 전달 인터셉터
│   │   └── WebConfig.java                  # [참고] JWT 필터 등록
│   ├── filter/
│   │   └── JwtAuthenticationFilter.java    # [참고] JWT 인가 필터
│   ├── handler/
│   │   ├── ex/                             # 커스텀 예외 (Exception400~500)
│   │   └── GlobalExceptionHandler.java     # [참고] 전역 예외 처리
│   └── util/
│       ├── JwtProvider.java                # [참고] JWT 파싱/검증
│       ├── JwtUtil.java                    # [참고] JWT 생성
│       └── Resp.java                       # [참고] 표준 응답 래퍼
└── orders/
    ├── Order.java                          # [참고] JPA 엔티티
    ├── OrderController.java                # [참고] REST 컨트롤러
    ├── OrderRepository.java                # [참고] Spring Data JPA
    ├── OrderRequest.java / OrderResponse.java  # [참고]
    ├── OrderService.java                   # [작성] 비즈니스 로직
    └── OrderStatus.java                    # [참고] 주문 상태 enum
Dockerfile                                  # [참고] Docker 이미지 빌드
```

> **회원/상품/배달 서비스는 `adapter/` 패키지와 `RestClientConfig`가 없고, 나머지 구조는 동일합니다.**

3. 실습 환경

도구	용도	비고
Docker Desktop	4개 서비스를 컨테이너로 실행	https://www.docker.com/products/docker-desktop/
Hoppscotch	API 호출 결과 확인	https://hoppscotch.io/ (설치 불필요, 브라우저 확장만 추가)

4. 실습 순서

1. 공통 설정(JWT·표준 응답·예외 처리)이 담긴 `core/` 패키지 살펴보기
2. 회원·상품·배달 서비스의 핵심 코드 살펴보기
3. 주문 서비스에 RestClient + 보상 트랜잭션 작성하기
4. Docker Compose로 4개 서비스를 한 번에 띄우고 시나리오 3개 검증하기

2.1 공통 설정 - 모든 서비스가 공유하는 뼈대

각 서비스는 **Spring Boot**로 만들어진 독립 프로젝트입니다. 서버가 분리되어 **세션을 공유할 수 없으므로 JWT 토큰**으로 인증합니다.

4개 서비스가 공통으로 쓰는 **JWT 인증·표준 응답·예외 처리**는 `core/` 패키지에 모아 둡니다. 각 컴포넌트의 역할은 다음과 같습니다.

컴포넌트	역할
JwtAuthenticationFilter	매 요청마다 JWT를 검증하고 사용자 정보를 컨트롤러로 전달합니다.
JwtUtil / JwtProvider	JwtUtil은 토큰을 발급하고 검증하는 핵심 로직이고, JwtProvider는 요청 헤더에서 토큰을 꺼내 JwtUtil에 넘깁니다.
Resp	모든 API 응답을 동일한 형태로 통일하는 래퍼입니다.

컴포넌트	역할
GlobalExceptionHandler	전역에서 발생한 예외를 잡아 일관된 에러 응답으로 변환합니다.
WebConfig	JWT 필터를 인증이 필요한 경로에 등록합니다.

주문 서비스로 가기 전, 회원·상품·배달 서비스는 참고 코드라 클래스 단위로 간단히 살펴봅니다. 자세한 구현은 완성 레포(GitHub)를 참고하세요.

2.2 회원 서비스 - JWT로 로그인하다

회원 서비스는 **로그인과 사용자 조회**를 담당합니다. 사용자가 `POST /login`으로 아이디와 비밀번호를 보내면, 회원 서비스가 DB에서 조회하고 비밀번호를 검증합니다. 검증에 성공하면 **JWT 토큰**을 응답 데이터에 담아 돌려줍니다. 이 토큰이 이후 모든 서비스 요청의 **인증 수단**이 됩니다.

HTTP 메서드	경로	기능
POST	/login	로그인 (JWT 발급)
GET	/api/users/{userId}	사용자 조회

2.2.1 클래스 구성

회원 서비스의 로그인과 사용자 조회를 다음 다섯 클래스가 처리합니다.

클래스	역할
User	사용자 정보를 담는 엔티티입니다.
UserRepository	사용자 데이터를 DB에서 조회·저장합니다.
UserRequest / UserResponse	회원 API의 요청과 응답 형식을 정의합니다.
UserController	로그인과 사용자 조회 API를 제공합니다.
UserService	로그인 시 비밀번호를 검증하고 JWT를 발급하며, 사용자 조회 요청을 처리합니다.

2.3 상품 서비스 - 재고를 관리하다

상품 서비스는 **상품 목록 조회와 재고 증감**을 담당합니다. 주문 서비스가 주문을 생성할 때 **재고 감소 API**를 호출하고, 주문이 취소되거나 실패하면 **재고 증가 API**로 되돌립니다.

HTTP 메서드	경로	기능
GET	/api/products/{productId}	상품 조회
PUT	/api/products/{productId}/decrease	재고 감소
PUT	/api/products/{productId}/increase	재고 증가

2.3.1 클래스 구성

상품 조회와 재고 증감을 다음 다섯 클래스가 처리합니다.

클래스	역할
Product	상품 정보를 담는 엔티티이며, 재고 증감 로직을 자기 안에 둡니다.
ProductRepository	상품 데이터를 DB에서 조회·저장합니다.
ProductRequest / ProductResponse	상품 API의 요청과 응답 형식을 정의합니다.
ProductController	상품 조회와 재고 증감 API를 제공합니다.
ProductService	재고 감소 전 상품 존재·재고·가격을 검증한 뒤 재고를 줄이거나 늘립니다.

2.4 배달 서비스 - 배달을 생성하고 취소하다

배달 서비스는 **배달 생성과 취소**를 담당합니다. 회원이나 상품 서비스와 달리, 주문과 배달에는 현재 진행 상황을 나타내는 **상태(`status`) 값**이 있습니다. 대기 상태인 `PENDING`으로 시작해 처리가 완료되면 `COMPLETED`, 취소되면 `CANCELLED`로 바뀝니다.

HTTP 메서드	경로	기능
POST	/api/deliveries	배달 생성
GET	/api/deliveries/{deliveryId}	배달 조회
PUT	/api/deliveries/{orderId}	배달 취소

2.4.1 클래스 구성

배달 생성과 취소를 다음 여섯 클래스가 처리합니다.

클래스	역할
Delivery	배달 정보와 현재 상태(PENDING / COMPLETED / CANCELLED)를 담는 엔티티입니다.
DeliveryStatus	배달 상태(대기·완료·취소)를 정의하는 열거형입니다.
DeliveryRepository	배달 데이터를 DB에서 조회·저장합니다.
DeliveryRequest / DeliveryResponse	배달 API의 요청과 응답 형식을 정의합니다.
DeliveryController	배달 생성·조회·취소 API를 제공합니다.
DeliveryService	배달 생성 시 완료까지 처리하며, 조회와 취소도 처리합니다.

2.5 주문 서비스 - 보상 트랜잭션의 현장

주문 서비스는 **주문 생성·조회·취소**를 담당합니다. 주문 요청 처리를 위해 상품·배달 서비스를 직접 호출하는 흐름의 중심에 있습니다.

HTTP 메서드	경로	기능
POST	/api/orders	주문 생성
GET	/api/orders/{orderId}	주문 조회

HTTP 메서드	경로	기능
PUT	/api/orders/{orderId}	주문 취소

2.5.1 클래스 구성

주문 생성과 보상 트랜잭션을 다음 여섯 클래스가 처리합니다.

클래스	역할
Order	주문 정보와 현재 상태(PENDING / COMPLETED / CANCELLED)를 담는 엔티티입니다.
OrderStatus	주문 상태(대기·완료·취소)를 정의하는 열거형입니다.
OrderRepository	주문 데이터를 DB에서 조회·저장합니다.
OrderRequest / OrderResponse	주문 API의 요청과 응답 형식을 정의합니다.
OrderController	주문 생성·조회·취소 API를 제공합니다.
OrderService	**[작성]** 주문 생성 시 보상 트랜잭션을 수행하며, 조회와 취소도 처리합니다.

2.5.2 보상 트랜잭션 흐름

서비스의 구조를 파악했으니, 이제 주문 서비스의 진짜 역할을 살펴봅니다. 주문 요청이 들어오면 주문 서비스는 상품·배달 서비스를 호출합니다. 이때 작업이 실패하면, **주문 서비스가 직접 보상 트랜잭션을 실행**해 원래 상태로 되돌립니다.

어느 단계에서 실패하면 무엇을 되돌려야 하는지 먼저 그려 보겠습니다.

단계	동작	실패 시 보상
1	재고 감소 (상품 서비스)	없음 (아직 진행 안 함)
2	배달 생성 (배달 서비스)	재고 복구 (1단계 되돌리기)
3	주문 완료	배달 취소 + 재고 복구 (2, 1단계 되돌리기)

단계	동작	실패 시 보상
4	성공 응답 반환	—

예를 들어 2단계 배달 생성에서 실패하면, 이미 줄어든 재고 한 단계만 되돌리면 됩니다. 3단계에서 실패하면 배달 취소와 재고 복구를 모두 되돌립니다. 진행한 만큼만 역순으로 되돌리려면, **어디까지 갔는지를 코드에서 기록**해 둬야 합니다.

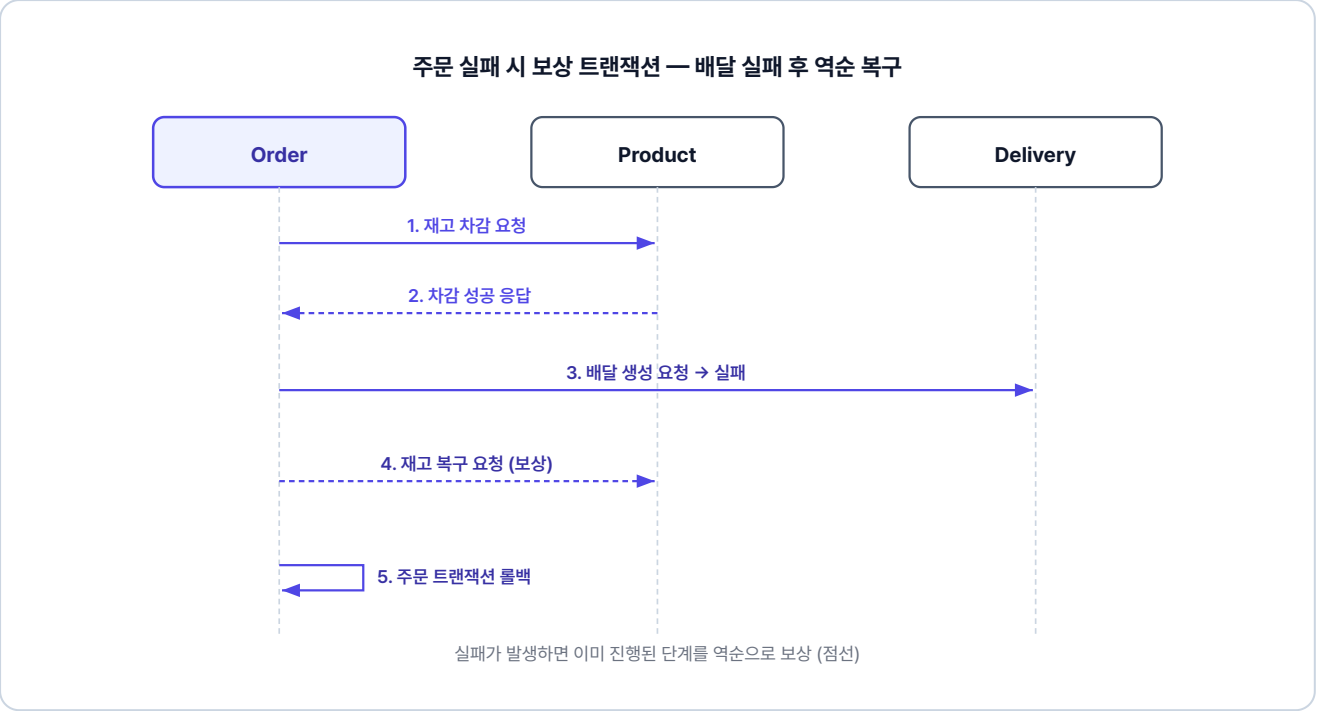


그림 2-2. 주문 실패 시 보상 트랜잭션 흐름

2.5.3 adapter - 외부 서비스 호출 설정

`adapter/` 폴더에는 외부 서비스를 호출하는 **클라이언트** 두 개가 있습니다.

RestClientConfig가 모든 외부 호출에 **JWT 토큰을 자동으로 실어** 줍니다. 그리고 **ProductClient**와 **DeliveryClient**가 각자 상품 서비스와 배달 서비스로 요청을 보냅니다.

클래스	역할
RestClientConfig	RestClient에 인터셉터를 등록하여 외부 호출 시 JWT를 자동 전달합니다.
ProductClient	상품 서비스의 decreaseQuantity(재고 감소)와 increaseQuantity(재고 복구)를 호출합니다.

클래스	역할
DeliveryClient	배달 서비스의 createDelivery(배달 생성)와 cancelDelivery(배달 취소)를 호출합니다.

2.5.4 OrderService - 보상 트랜잭션의 핵심

이제 이번 챕터의 핵심입니다. **보상 트랜잭션 패턴**을 직접 구현합니다. 코드를 읽을 때 `productDecreased`와 `deliveryCreated` **두 플래그를 추적**하면서 읽어보세요. 단계가 성공할 때마다 플래그를 `true`로 바꾸고, 실패 시 catch 블록에서는 `true`**로 표시된 단계만 역순으로 되돌립니다**.

주문 서비스의 `orders/OrderService.java`를 열고 아래 메서드를 작성합니다.

실습 1 **주문 서비스 - orders/OrderService.java. createOrder - 보상 트랜잭션 핵심**

```
@Transactional
public OrderResponse createOrder(int userId, int productId,
        int quantity, Long price, String address) {
    // 보상트랜잭션을 위한 변수 선언
    boolean productDecreased = false;
    boolean deliveryCreated = false;

    // 보상트랜잭션에서 id를 전달해야해서 상위로 빼둠
    Order createdOrder = null;

    try {
        // 1. 주문 생성
        createdOrder = orderRepository.save(
                Order.create(userId, productId, quantity, price));

        // 최소 주문 금액 검증
        if (quantity * price < 1000) {
            throw new Exception400("최소 주문 금액은 1,000원입니다.");
        }

        // 2. 상품 재고 차감
        productClient.decreaseQuantity(
                new ProductRequest(productId, quantity, price));
        productDecreased = true;

        // 3. 배달 생성
        deliveryClient.createDelivery(
                new DeliveryRequest(createdOrder.getId(), address));
        deliveryCreated = true;
```

```
            // 4. 주문 완료
            createdOrder.complete();
            return OrderResponse.from(createdOrder);

        } catch (Exception e) {
            // 배달 취소
            if (deliveryCreated) {
                deliveryClient.cancelDelivery(createdOrder.getId());
            }

            // 재고 복구
            if (productDecreased) {
                productClient.increaseQuantity(
                        new ProductRequest(productId, quantity, price));
            }
            throw new Exception500("주문 생성 중 오류가 발생했습니다: " + e.getMessage());
        }
    }
```

주문 데이터는 catch에서 따로 되돌리지 않아도 자동으로 롤백됩니다. 외부 서비스(상품·배달)만 직접 보상하면 됩니다.

재고를 줄였으면 다시 늘리고, 배달을 만들었으면 취소하고... 그 말이 이거였구나.

> **MSA에서 int/Long 대신 UUID를 쓰는 이유**
>
> 이 책은 주문 ID를 순차 증가 방식(int/Long)으로 만듭니다. 단일 데이터베이스를 쓰는 작은 서비스에서는 이 방식이 단순하고 편합니다. 하지만 서비스 규모가 커지면 임의의 난수인 UUID를 사용해야 합니다. 가장 큰 이유는 **보안**과 **분산 환경에서의 충돌 방지** 때문입니다.
>
> 순차 번호는 1037 다음이 1038임을 누구나 알 수 있습니다. 외부에서 **주소의 번호만 바꿔 악의적으로 남의 주문을 조회하거나, 하루 전체 주문량을 쉽게 유추할 수 있습니다**. 반면 UUID는 랜덤 값이기 때문에 다음 번호나 주문량을 추측할 수 없습니다. 또한, 시스템 규모가 커져 데이터베이스를 여러 대로 나누게 되면, 각 DB가 **동일한 ID를 생성해 번호가 겹치는 치명적인 문제**가 생길 수 있습니다. UUID를 사용하면 데이터를 여러 곳으로 분산시켜도 번호 충돌이 발생하지 않습니다.

2.6 Docker Compose - 네 개의 서비스를 한 번에 실행하다

시나리오를 따라가기 전에, 각 서비스에 어떤 데이터가 미리 등록되어 있는지 살펴봅니다. 이번 챕터는 **H2 in-memory DB**를 사용합니다. 데이터 정의는 각 서비스의 `db/data.sql`에 있습니다.

서비스	더미 데이터
회원	계정 3개: ssar, cos, love (비밀번호는 모두 1234)
상품	MacBook Pro(재고 10), iPhone 15(**재고 0 품절**), AirPods(재고 10)
배달	주문 3건에 대응하는 배달 데이터, 모두 COMPLETED 상태
주문	사용자별 주문 3건(완료·취소·대기)

2.6.1 서비스 실행

각 서비스는 **동일한 구조**의 Dockerfile로 **컨테이너 안에서 빌드하고 실행**됩니다. 주문 서비스의 Dockerfile을 예로 살펴봅니다.

참고 **order/Dockerfile**

```
FROM eclipse-temurin:21-jdk              # JDK 21 베이스 이미지
WORKDIR /app                             # 작업 디렉터리 설정
COPY . .                                 # 프로젝트 소스 복사
RUN chmod +x gradlew                     # gradlew 실행 권한 부여
RUN ./gradlew bootJar -x test            # 테스트 없이 실행 가능한 JAR 빌드
RUN cp build/libs/*.jar app.jar          # 빌드된 JAR를 app.jar로 복사
ENTRYPOINT ["java", "-jar", "app.jar"]   # 컨테이너 시작 시 JAR 실행
```

`ex01` 디렉토리의 `docker-compose.yml`은 네 서비스를 **하나의 네트워크로 묶어 한 번에 실행**합니다.

서비스	build.context	호스트:컨테이너 포트	네트워크
order-service	./order	8081:8081	msa-network
user-service	./user	8083:8083	msa-network
product-service	./product	8082:8082	msa-network
delivery-service	./delivery	8084:8084	msa-network

`msa-network`로 묶여 있기 때문에, 컨테이너끼리는 **서비스 이름**(예: `http://product-service:8082`)으로 통신할 수 있습니다.

프로젝트가 위치한 폴더로 이동 후, 터미널에서 Docker Compose로 4개 서비스를 한 번에 빌드하고 실행합니다.

터미널 **Docker Compose 실행**

```
docker compose up
```

ex01 — docker compose up

```
user-service   | Started UserApplication in 4.231 seconds (port: 8083)
product-service   | Started ProductApplication in 4.512 seconds (port: 8082)
order-service   | Started OrderApplication in 5.103 seconds (port: 8081)
delivery-service   | Started DeliveryApplication in 4.687 seconds (port: 8084)
----------------------------------------------------------------------------
4개 서비스 기동 완료
```

그림 2-3. Docker Compose 실행 결과

2.6.2 Hoppscotch와 인터셉터 설정

서비스를 실행했으니, 이제 API를 호출해 잘 동작하는지 확인해 보겠습니다. 이 책에서는 브라우저에서 API를 호출하는 도구인 **Hoppscotch**(https://hoppscotch.io/)를 사용합니다.

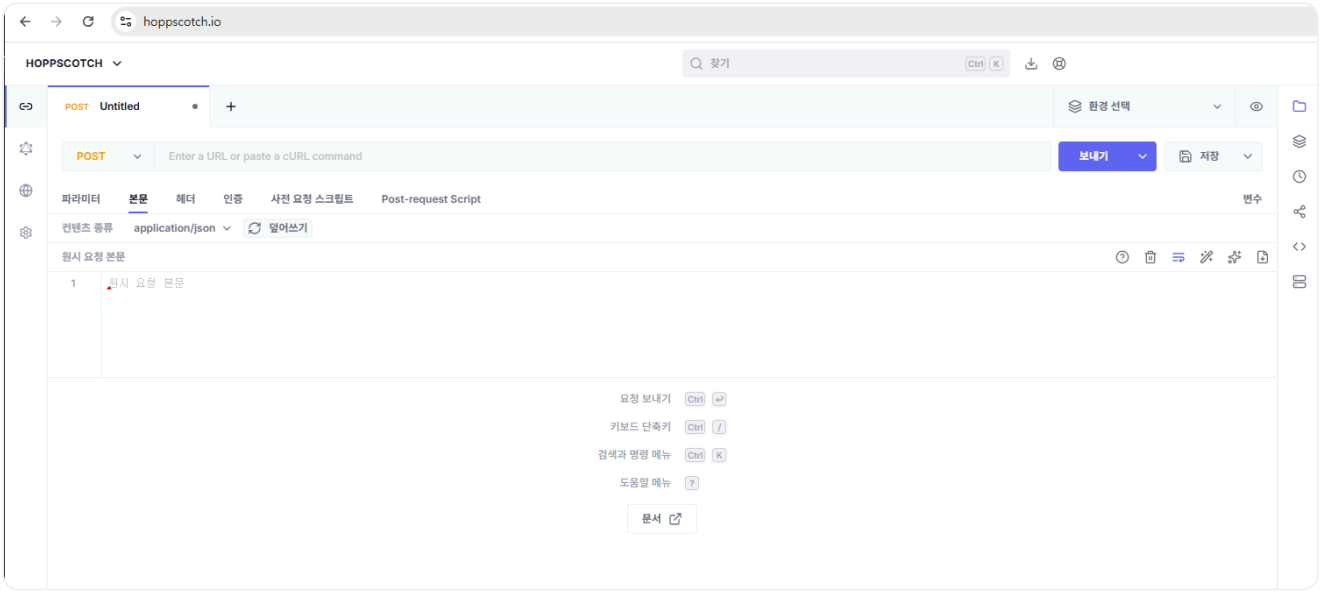


그림 2-4. Hoppscotch 화면

웹 브라우저는 보안 때문에 내 컴퓨터의 localhost로 바로 요청을 보내지 못합니다. 그래서 요청을 대신 전달해 주는 **Hoppscotch Browser Extension**을 Chrome 웹 스토어에서 설치하고, 설정 > Interceptor에서 익스텐션을 선택합니다.

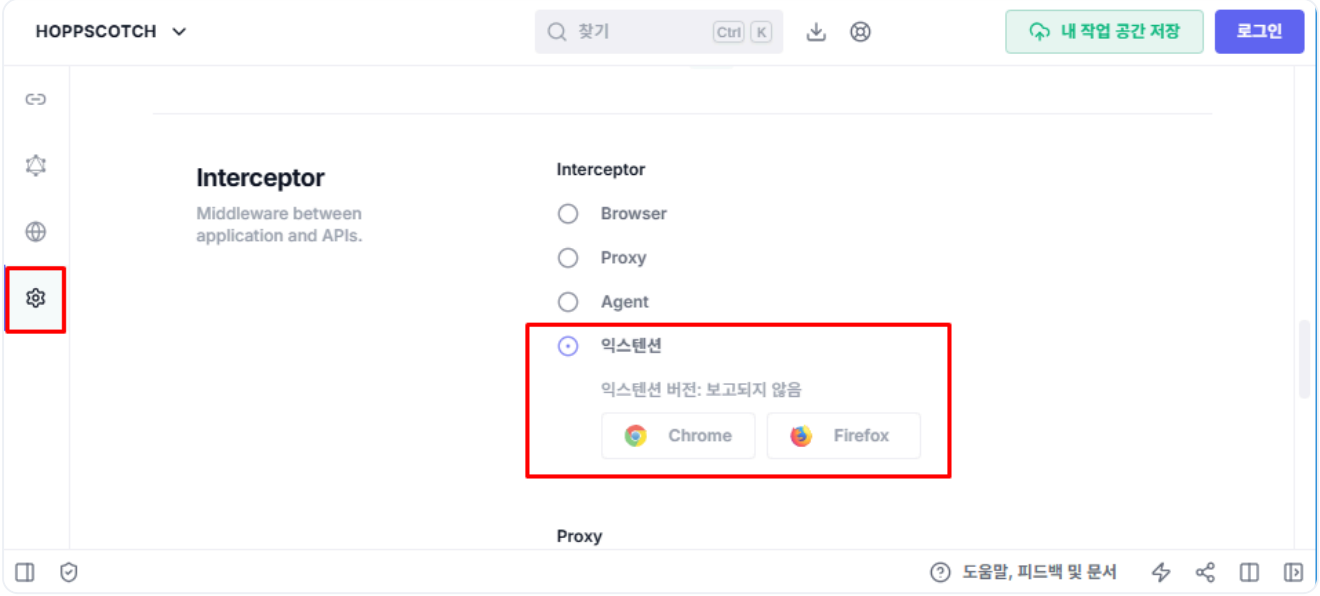


그림 2-5. Browser Extension 인터셉터 설정

2.6.3 시나리오 1: 정상 주문

먼저 로그인하여 JWT 토큰을 받습니다. 이때 콘텐츠 종류(Content-Type)를 `application/json`으로 설정해야 합니다. 이 헤더는 서버에게 "내가 보내는 데이터는 JSON 형식이다"라고 알리는 역할을 합니다.

Hoppscotch 로그인

```
POST http://localhost:8083/login

{
  "username": "ssar",
  "password": "1234"
}
```

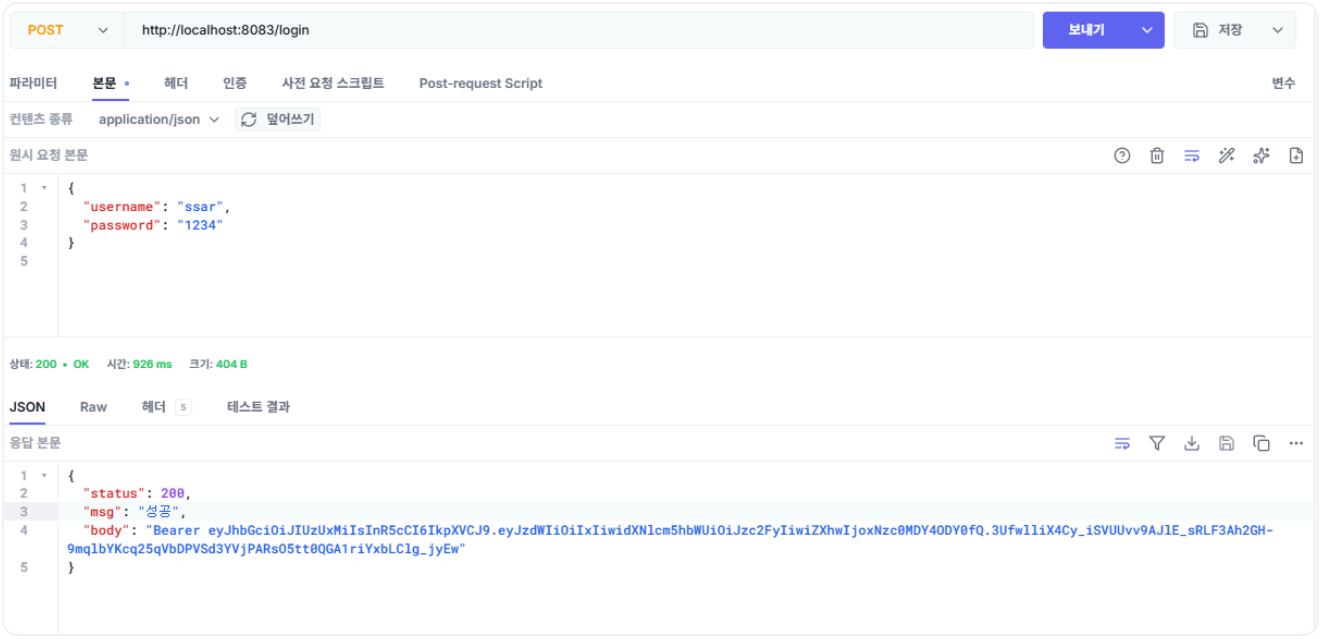


그림 2-6. 로그인 API 호출 결과

응답 데이터에 포함된 JWT 토큰을 확인할 수 있습니다.

받은 토큰을 Hoppscotch의 **인증 > 인증 유형(Bearer)** 항목의 토큰 필드에 넣습니다.

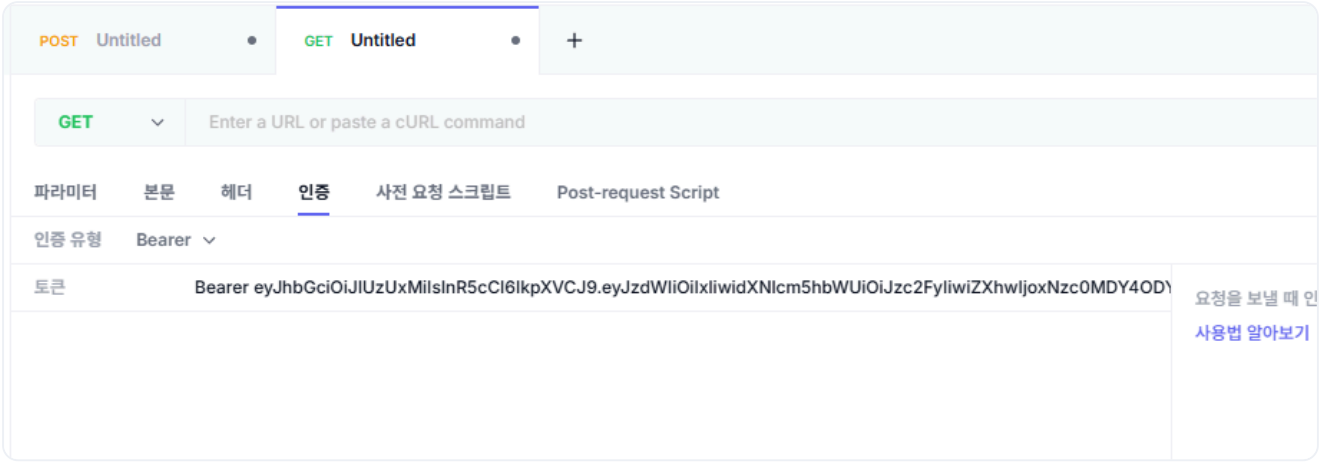


그림 2-7. Bearer 토큰 설정

다음으로 상품 ID가 1인 MacBook Pro 1개를 주문합니다. 요청 데이터는 상품 정보(`productId`, `quantity`, `price`)와 배달 주소(`address`)를 한 번에 담습니다.

Hoppscotch 주문 생성

```
POST http://localhost:8081/api/orders

{
```

```
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

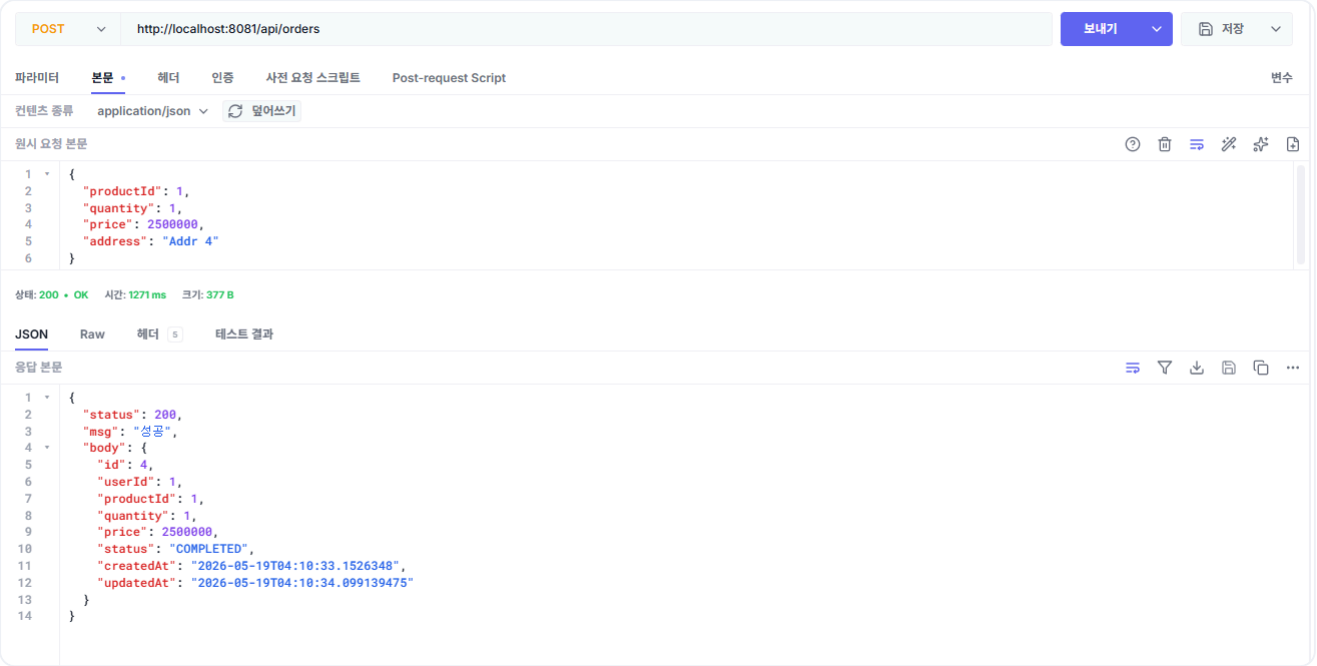


그림 2-8. 주문 생성 API 호출 결과

주문이 성공하면 상품 서비스에서 재고가 10 → 9로 줄어듭니다.

Hoppscotch **재고 조회**

```
GET http://localhost:8082/api/products/1
```

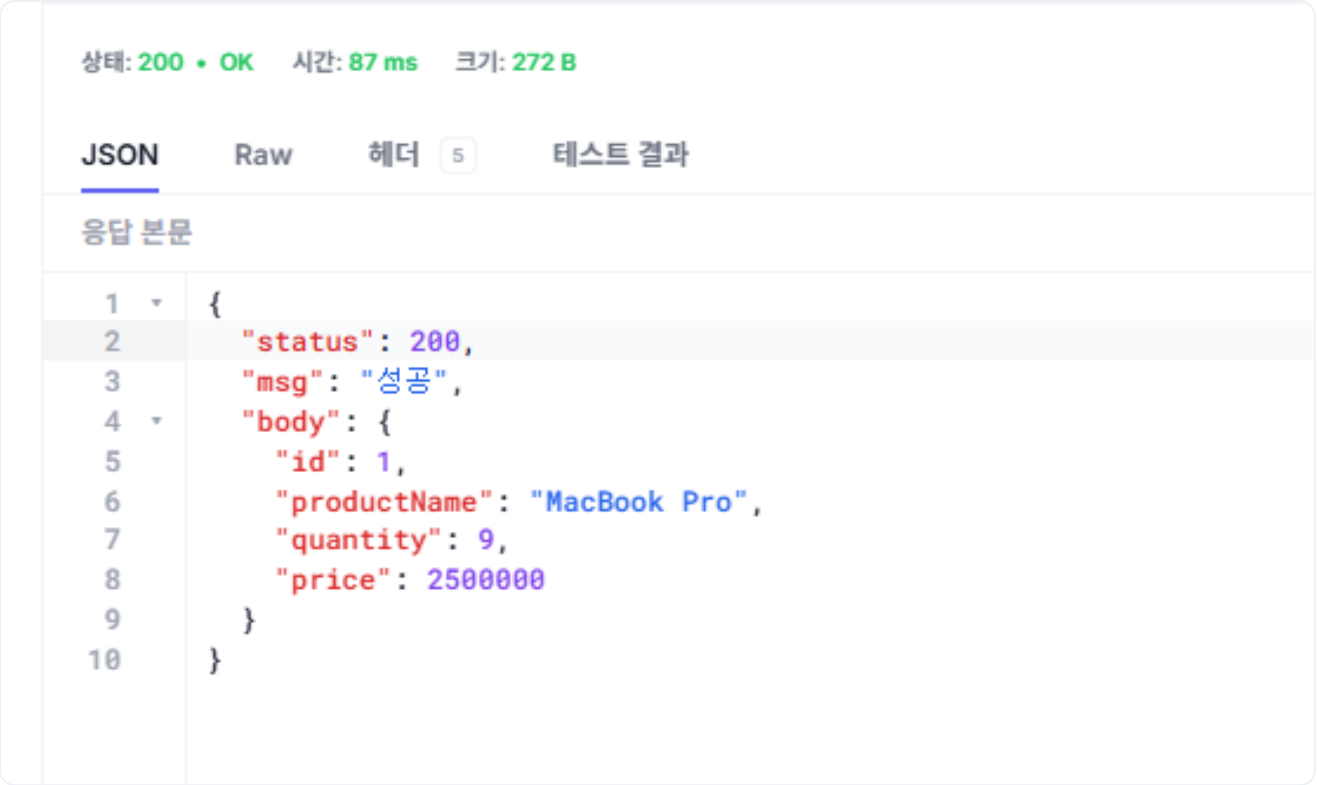


그림 2-9. 재고 감소 확인

배달 서비스에도 배달이 생성됐는지 확인합니다.

Hoppscotch **배달 조회**

```
GET http://localhost:8084/api/deliveries/4   # 더미 배달 3건 다음이라 새 배달은 4번
```

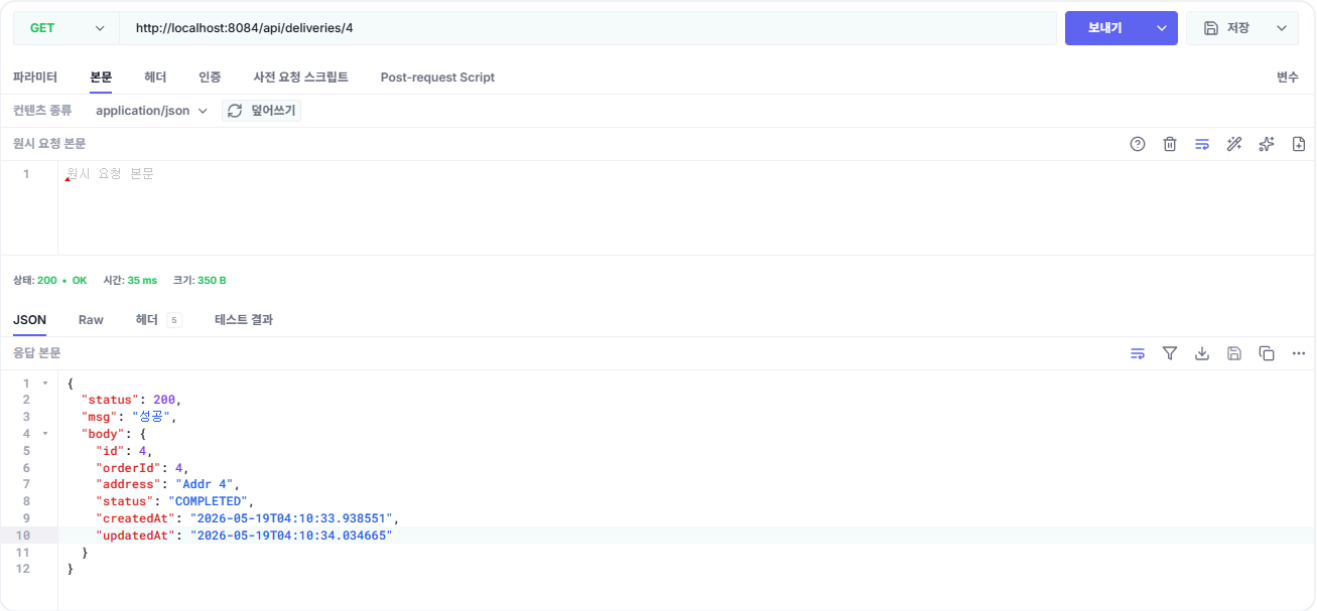


그림 2-10. 배달 생성 확인

2.6.4 시나리오 2: 재고 부족

이번에는 상품 ID가 2인 품절 상품 iPhone 15를 주문해 보겠습니다. 첫 번째 단계인 재고 차감에서 바로 실패하므로, 보상할 작업이 없어 즉시 에러가 반환됩니다.

Hoppscotch 주문 생성 (재고 부족)

```
POST http://localhost:8081/api/orders

{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 4"
}
```



그림 2-11. 재고 부족 시 에러 응답

2.6.5 시나리오 3: 주소 누락

이번에는 주소를 빈 문자열로 보내 보겠습니다. 주문 자체는 재고 차감까지 진행되지만, 배달 서비스에서 주소가 없으므로 실패합니다. 이때 보상 트랜잭션이 작동하여 차감된 재고가 복구되고, 주문 데이터는 트랜잭션 롤백으로 처음부터 없던 일처럼 DB에서 사라집니다.

Hoppscotch 주문 생성 (주소 누락)

```
POST http://localhost:8081/api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": ""
```

```
}
```

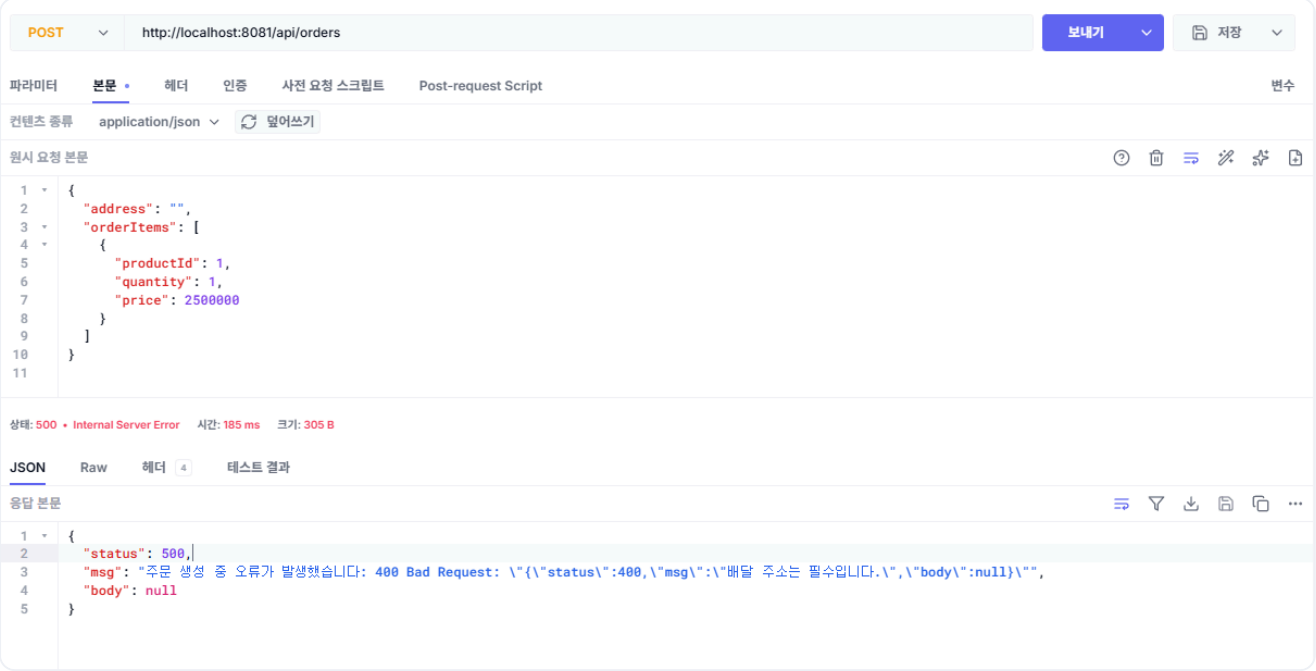


그림 2-12. 주소 누락 시 에러 응답

그리고 재고가 원복되었는지 확인합니다. 시나리오 1에서 재고가 10개에서 9개로 줄었으니, 이번에 차감된 재고가 보상으로 복구되면 다시 9개로 돌아옵니다.

Hoppscotch **재고 조회**

```
GET http://localhost:8082/api/products/1
```

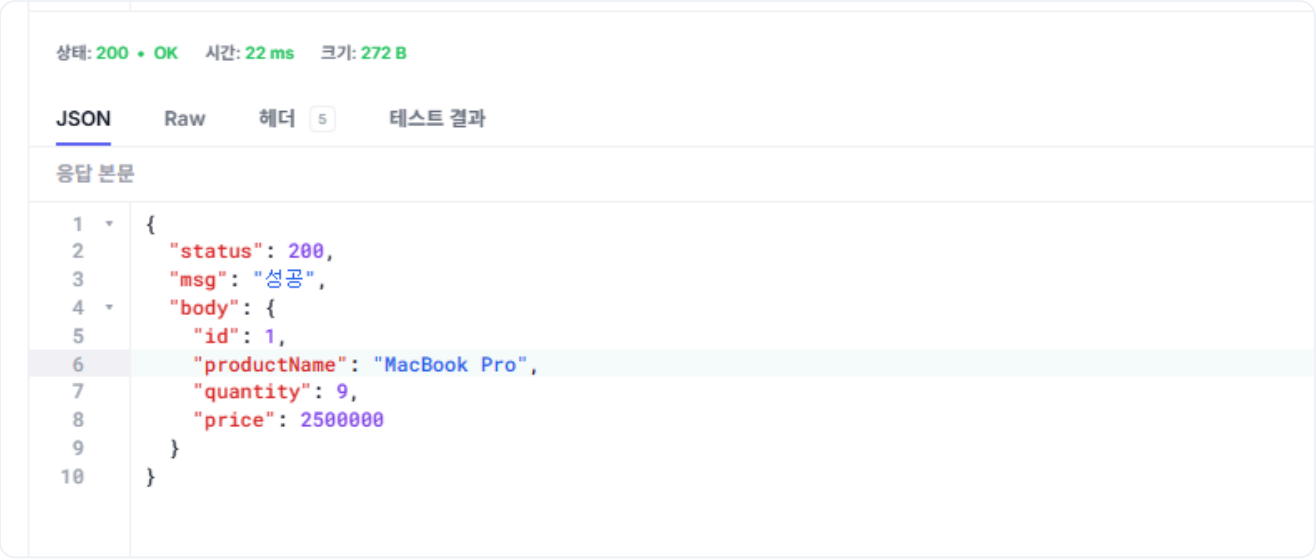


그림 2-13. 재고 원복 확인

테스트가 끝났으면 실행 중인 컨테이너를 정리합니다.

터미널 **컨테이너 정리**

```
docker compose down
```

오픈이 : "기능별로 서비스를 분리했는데도 서로 통신이 되네요. 중간에 실패해도 보상 트랜잭션이 동작하구요."

선배 : "맞아요. 그런데 이 방식은 주문 서비스가 상품이랑 배달을 직접 호출하다 보니, 서비스만 분리됐을 뿐 결국 다른 서비스의 장애가 그대로 전파돼요. 상품이나 배달 서비스가 조금만 느려지거나 죽어도, 주문 서비스까지 같이 느려지거나 멈추죠. 이제 여기서부터 하나씩 고쳐 나가봅시다."

지금 구조는 각 서비스 안에서 컨트롤러 계층과 서비스 계층이 직접 의존합니다. 이 때문에 처리 방식을 비동기로 전환하려면 컨트롤러를 비롯해 연관된 메서드까지 모두 수정해야 합니다.

다음 챕터에서는 이 결합도를 낮추기 위해 아키텍처를 개선합니다. 외부 요청과 비즈니스 로직을 분리해, 어느 한쪽을 변경하더라도 다른 쪽에 영향을 주지 않는 구조를 만듭니다.

이것만은 기억하자

- 서비스가 분리되면 세션을 공유할 수 없어, **JWT**로 인증하고 호출할 때 토큰을 전달합니다.
- 분산 트랜잭션은 자동으로 롤백되지 않아, 실패하면 **보상 트랜잭션**으로 이미 끝난 작업을 되돌립니다.
- 동기 직접 호출은 서비스 간 **결합도를 높여**, 한 서비스가 멈추면 호출한 쪽도 함께 멈춥니다.

CHAPTER

3

도메인을 중심으로

DDD + 클린 아키텍처

이번 챕터가 끝나면

- 비즈니스 규칙을 도메인에 모으는 **DDD(도메인 주도 개발)** 를 이해할 수 있습니다.
- 구현 대신 인터페이스에 의존하는 **클린 아키텍처**를 이해할 수 있습니다.
- 게이트웨이와 **쿠버네티스**로 서비스를 묶어 배포하는 구조를 이해할 수 있습니다.

챕터 3. 도메인을 중심으로 - DDD + 클린 아키텍처

며칠 뒤, 팀장이 오픈이의 자리로 찾아왔습니다.

팀장 : "주문 기능 하나 테스트하려고 하는데, 컨트롤러가 서비스에 직접 의존해서 가짜(Mock) 객체로 테스트할 수가 없어요. 이거 직접 의존하지 않게 구조 정리해 줘요."

오픈이는 곧바로 코드를 열어 봤습니다. 컨트롤러와 서비스가 직접 의존하다 보니, 컨트롤러를 테스트하려면 서비스까지 함께 동작해야 했습니다. 그리고 가짜(Mock) 객체를 넣으려 해도 컨트롤러 코드를 직접 고쳐야 했습니다.

고민에 빠진 오픈이는 선배 자리로 찾아가 코드를 보여 주며 물었습니다.

선배 : "지금 구조는 결합도가 너무 높아서 그래요. 해결하려면 두 가지를 알아야 해요.

첫째는 구현이 아니라 **인터페이스에 의존**하게 만드는 거예요. 서비스가 다른 객체를 직접 호출하지 않고, 둘 사이에 인터페이스를 두는 거죠. 그래야 코드를 고치지 않고도 진짜 객체 대신 가짜 객체로 테스트할 수 있어요. 이게 **클린 아키텍처**의 기본이에요.

둘째는 **도메인 주도 개발(DDD)** 이에요. 핵심 비즈니스 로직을 외부 환경에 두지 말고 도메인 내부에 모아 둬야 해요. 그래야 외부가 어떻게 바뀌든 영향을 받지 않고, 테스트도 도메인만 독립적으로 깔끔하게 끝낼 수 있어요."

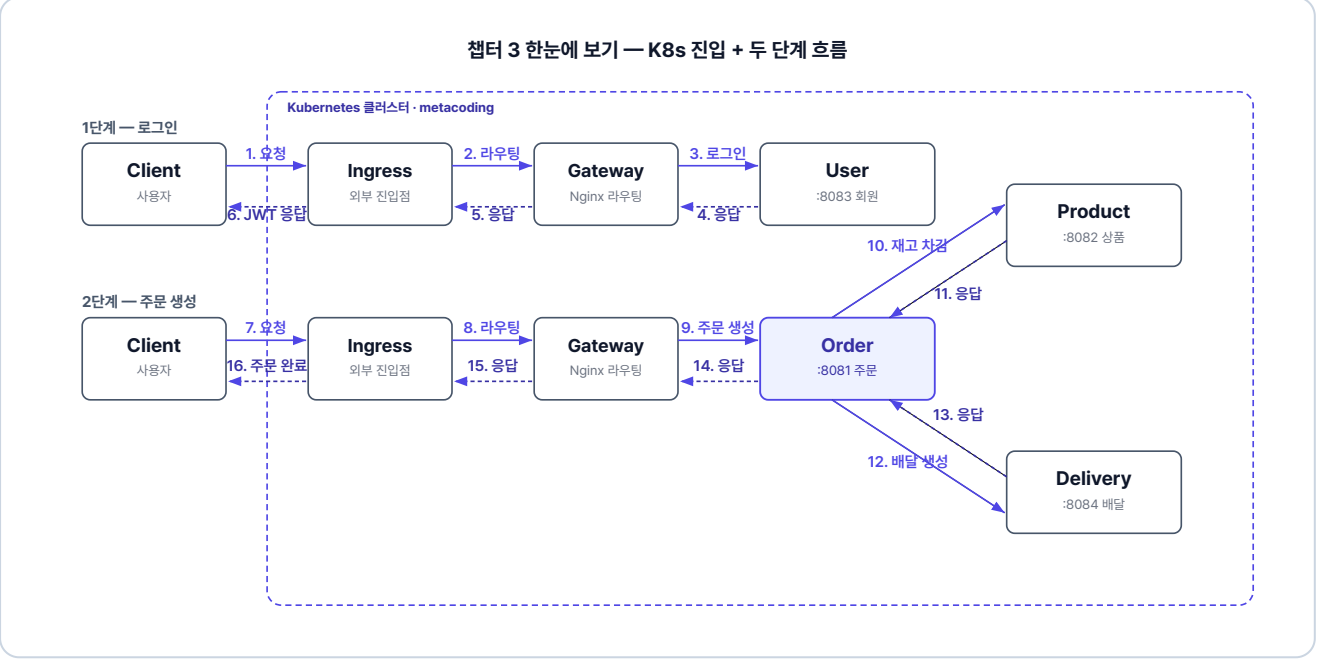


그림 3-1. 챕터 3 한눈에 보기 - K8s 진입과 두 단계 흐름

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex02
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex02` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex02 디렉토리

```
ex02/
├── db/                 # MySQL Dockerfile
├── delivery/           # 포트 8084
├── gateway/            # Nginx API Gateway
├── k8s/                # Kubernetes YAML 파일
├── order/              # 포트 8081
├── product/            # 포트 8082
└── user/               # 포트 8083
```

주문 서비스 패키지 구조 (챕터 3에서 재구성)

```
src/main/java/com/metacoding/order/
├── adapter/        # 외부 서비스 클라이언트 (order 전용)
├── core/           # JWT, 예외처리 (챕터 2와 동일)
├── domain/         # 엔티티 + 비즈니스 규칙
├── repository/     # Spring Data JPA
├── usecase/        # UseCase 인터페이스 + 서비스 코드
└── web/            # 컨트롤러 + DTO
src/main/resources/
└── application.properties        # DB·JWT 설정 (값은 환경변수로 주입)
```

user/product/delivery도 동일한 구조이며, adapter/ 패키지만 order 전용입니다.

3. 실습 환경

도구	용도	비고
Docker Desktop	컨테이너 런타임	챕터 2에서 설치한 그대로. 실행 중이어야 함
Minikube	로컬 Kubernetes 클러스터	https://minikube.sigs.k8s.io/

4. 실습 순서

1. 챕터 2 코드를 DDD + 클린 아키텍처로 재구성하는 과정 살펴보기
2. 주문 서비스에 UseCase 인터페이스 + 도메인 캡슐화 적용
3. Nginx API Gateway와 MySQL 인프라 살펴보기
4. Kubernetes YAML 파일(ConfigMap·Secret·Deployment·Service·Ingress) 5종 살펴보기
5. Minikube에서 빌드·배포·실행

3.1 도메인 주도 개발(Domain-Driven Design) - 비즈니스 로직을 도메인으로

가게 운영 규칙이 **사장 한 명의 머릿속**에만 있다고 해보겠습니다. 환불 가능 시간, 결제 방식, 재고 처리까지 전부 사장이 외우고 있습니다. 그래서 누가 손님을 응대하든 사장이 대답을 해야 합니다.

이 문제는 **가게 운영 매뉴얼**을 만들면 해결됩니다. 규칙은 매뉴얼에 정리해 두고, 사장은 손님 응대 흐름만 진행하면서 매뉴얼에 적힌 대로 따릅니다. **누가 응대하든 매뉴얼만 보면 같은 판단을 할 수 있습니다.** 새 규칙이 들어와도 **매뉴얼 한 곳만 업데이트**하면 끝입니다.

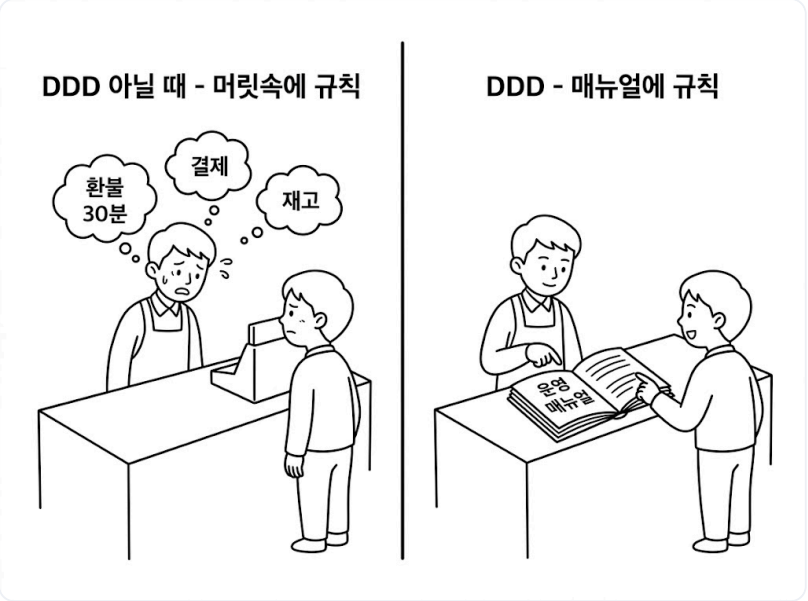


그림 3-2. 머릿속 규칙에서 운영 매뉴얼로

여기서 `OrderService`가 비즈니스를 수행하는 **사장님**이라면, `Order` 도메인 객체는 그 안에 담긴 핵심 규칙인 **운영 매뉴얼**입니다. **비즈니스 로직을 서비스에서 분리해 도메인에 모아 두면** 기술적 환경이 변해도 비즈니스의 본질은 영향을 받지 않으며, 복잡한 요구사항 속에서도 코드의 가독성과 유지보수성을 지킬 수 있습니다.

3.2 UseCase 인터페이스(Use Case Interface) - USB 허브

비즈니스를 수행하는 서비스가 컨트롤러에 직접 묶여 있으면 두 코드가 **강하게 결합됩니다.** 그래서 서비스를 고칠 때마다 컨트롤러도 함께 고쳐야 하고, 테스트할 때 진짜 서비스 대신 **가짜(Mock) 객체를 끼워 넣기도 불가능**해집니다.

이 문제를 해결하려면 컨트롤러와 서비스 사이에 **느슨한 연결 고리**가 필요합니다. 컴퓨터와 USB 허브를 떠올려 보세요. 여러 장치를 컴퓨터에 직접 연결하지 않고 USB 허브를 거쳐 연결하면, 뒤에서 장치를 아무리 바꾸더라도 **컴퓨터와 허브 사이의 연결에는 아무런 변화가 없습니다.**

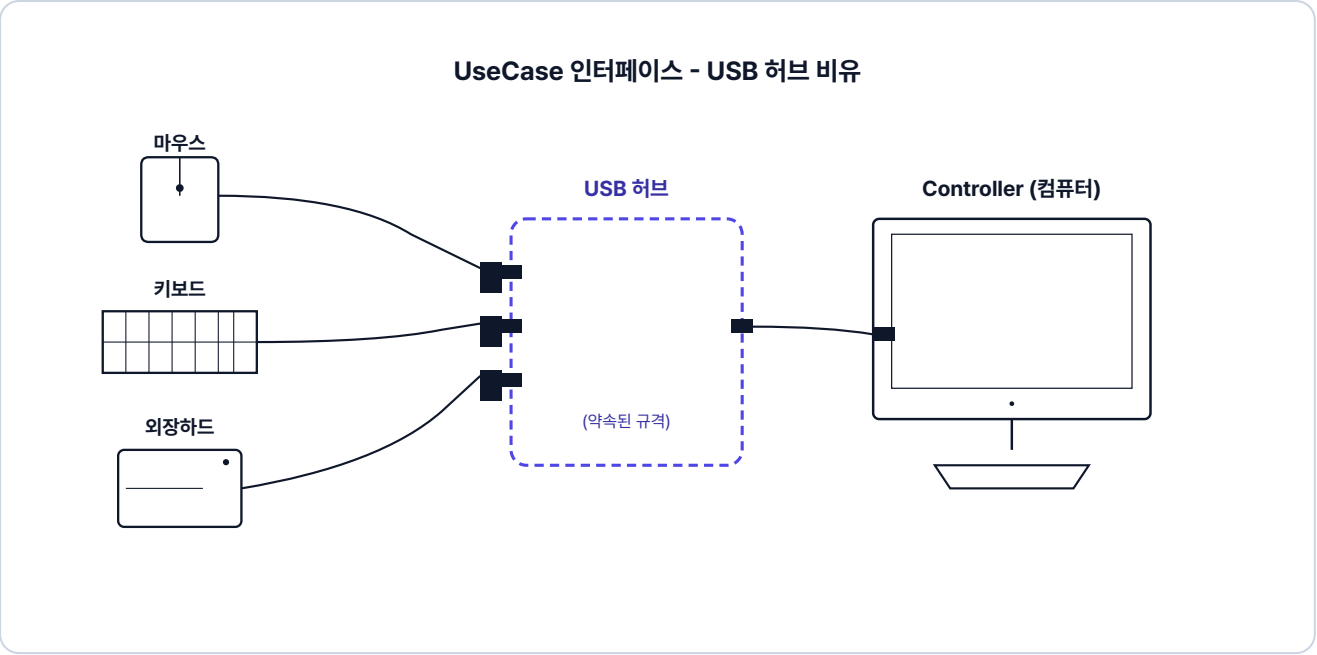


그림 3-3. UseCase 인터페이스 - USB 허브 비유

여기서 USB 허브의 역할을 하는 것이 **UseCase 인터페이스**입니다.

> **UseCase 인터페이스란?** 시스템이 수행할 비즈니스 **행위(주문 생성·조회·취소 같은)** 를 메서드로 약속한 인터페이스입니다.

컨트롤러가 구현체 대신 UseCase 인터페이스를 참조하고, 서비스가 이 인터페이스를 구현하면 둘은 독립적으로 동작합니다. 이렇게 의존 관계를 느슨하게 만들어 내부 로직을 보호하는 것이 **클린 아키텍처**의 원칙입니다.

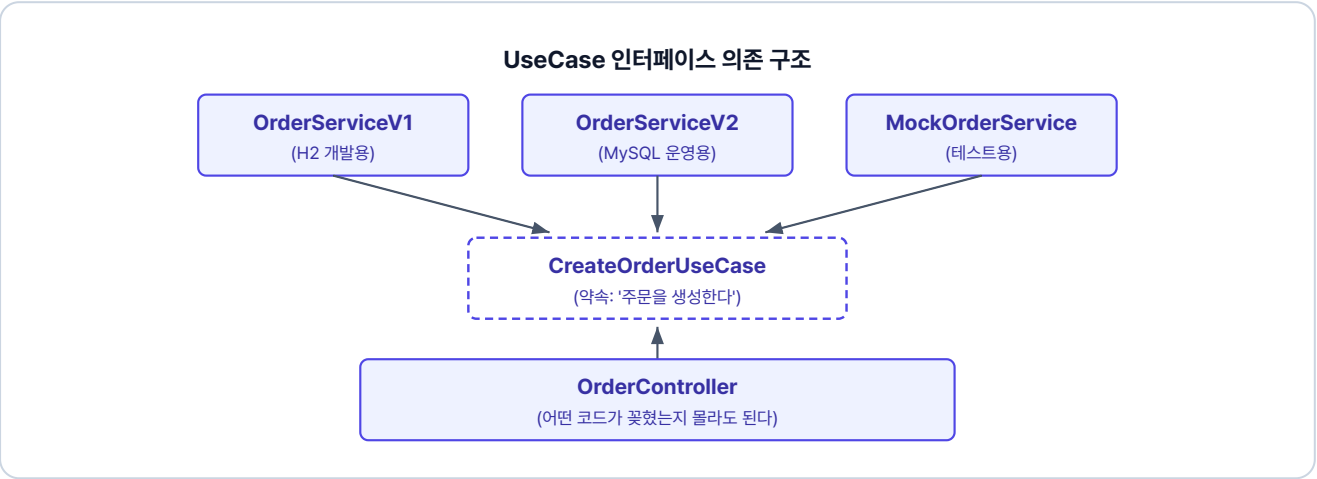


그림 3-4. UseCase 인터페이스 의존 구조

"무엇을 할 것인가"(UseCase 인터페이스) 와 **"어떻게 할 것인가"(Service 코드)** 를 분리하는 것이 핵심입니다.

3.3 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로

챕터 2는 **단일 패키지 구조**입니다. 주문과 관련된 모든 클래스가 `orders/` 한 폴더에 모여 있습니다.

반면 챕터 3은 **책임별 패키지 구조**입니다. 같은 주문 코드가 역할에 따라 네 폴더로 나뉩니다. 앞에서 다룬 도메인 응집과 인터페이스 분리가 폴더 구조에 그대로 반영됩니다.

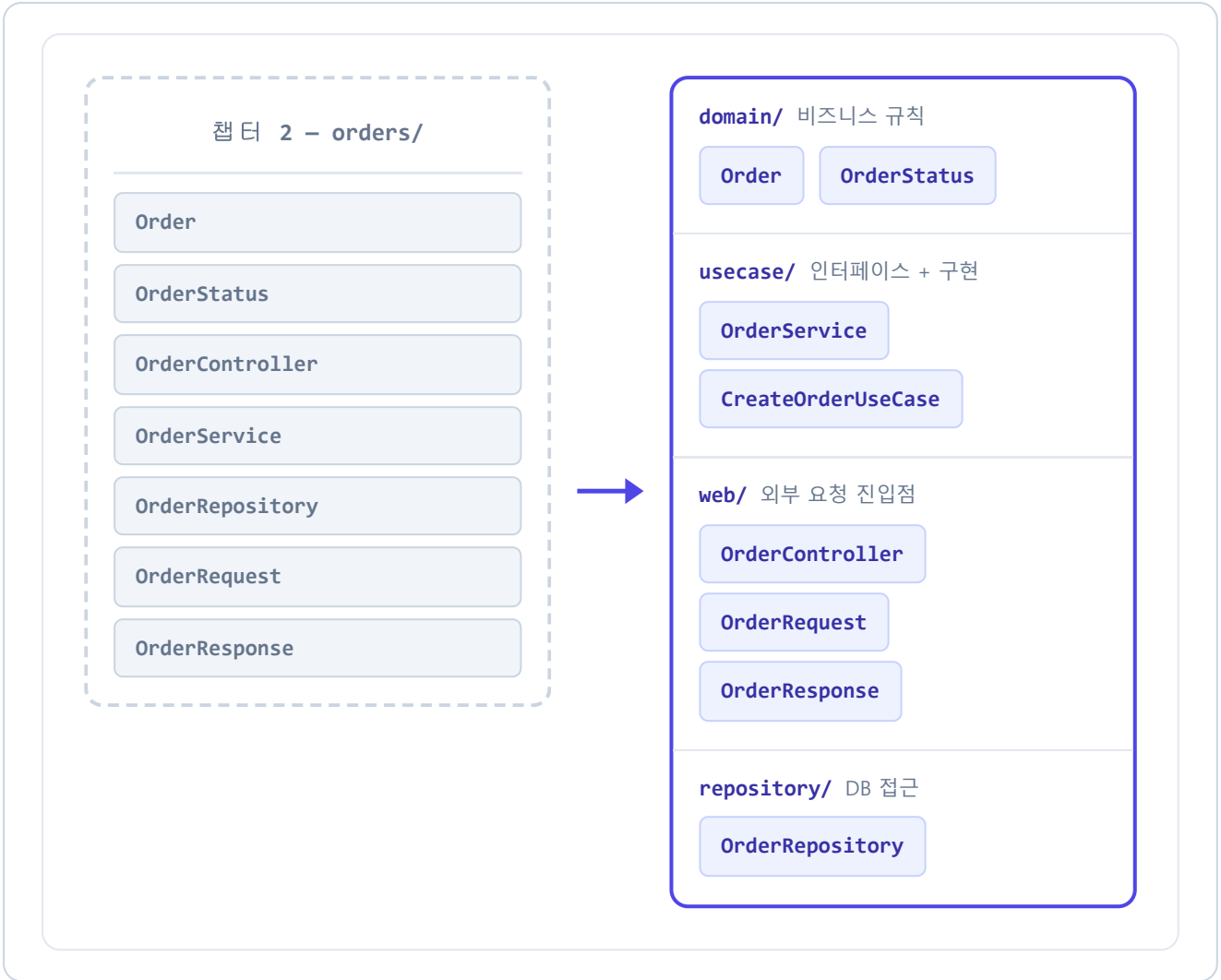


그림 3-5. 패키지 구조 비교 - 단일 패키지에서 책임별 패키지로

이렇게 도메인을 중심에 두고 외부 의존을 인터페이스로 분리하는 패키지 구조를 **헥사고날 패턴(Hexagonal Architecture)** 이라고 합니다. 이 책에서는 완전한 아키텍처보다는 실습에 필요한 개념만 적용합니다.

3.4 UseCase 인터페이스 + 도메인 캡슐화 도입

이제 실제 코드에 적용해보겠습니다.

3.4.1 UseCase 인터페이스 정의

주문·상품·회원·배달 서비스의 각 기능을 인터페이스 형태로 UseCase로 정의합니다. **인터페이스 하나가 하나의 행위(Use Case)를 의미합니다.**

주문 서비스의 `usecase/CreateOrderUseCase.java`를 열고 주문 생성 인터페이스를 작성합니다.

실습 1 **주문 서비스 - usecase/CreateOrderUseCase.java. 주문 생성 인터페이스**

```
// 주문을 생성한다 - 행위 하나를 인터페이스로 약속
public interface CreateOrderUseCase {
    OrderResponse createOrder(int userId, int productId,
            int quantity, Long price, String address);
}
```

조회는 `GetOrderUseCase`, 취소는 `CancelOrderUseCase`로 같은 방식으로 정의합니다.

3.4.2 엔티티의 비즈니스 로직 — DDD의 핵심

"주문 금액이 최소 기준을 넘는가?" 같은 비즈니스 규칙은 서비스가 아닌 엔티티에 둡니다. 엔티티 메서드로 캡슐화하면 어디서 호출하든 동일한 규칙이 적용되고, 새 규칙이 들어와도 도메인 메서드만 추가하면 됩니다.

주문 서비스의 `domain/Order.java`를 열고 최소 주문 금액 검증 메서드를 작성합니다.

실습 2 **주문 서비스 - domain/Order.java. 비즈니스 규칙을 도메인에 캡슐화**

```
public class Order {
    // 챕터 2 Order.java 참조 — 필드·create()·complete() 동일

    // 최소 주문 금액 검증 (챕터 2에서 서비스에 있던 검증을 도메인으로 옮김)
    public void validateMinAmount() {
        if (this.quantity * this.price < 1000) {
            throw new Exception400("최소 주문 금액은 1,000원입니다.");
        }
    }
```

```
}
```

3.4.3 OrderService - 인터페이스 구현

OrderService는 주문 생성, 주문 조회, 주문 취소 인터페이스를 구현하고, 도메인 객체의 비즈니스 메서드를 호출합니다.

주문 서비스의 `usecase/OrderService.java`를 열고 UseCase 인터페이스 구현을 작성합니다.

실습 3 **주문 서비스 - usecase/OrderService.java. UseCase 인터페이스 구현**

```
@RequiredArgsConstructor
@Service
@Transactional(readOnly = true)
public class OrderService
        implements CreateOrderUseCase, GetOrderUseCase, CancelOrderUseCase {
    // 1. UseCase 인터페이스를 구현

    @Override
    @Transactional
    public OrderResponse createOrder(int userId, int productId,
            int quantity, Long price, String address) {
        Order createdOrder = orderRepository.save(
                Order.create(userId, productId, quantity, price));
        // 2. 검증 메서드 호출
        createdOrder.validateMinAmount();
        // ... 재고 차감 → 배달 생성 → 완료 (보상 트랜잭션)
    }
}
```

3.4.4 OrderController 수정

컨트롤러는 서비스가 아닌 인터페이스에 의존합니다. `OrderService`를 다른 코드로 바꿔도 이 컨트롤러는 전혀 수정할 필요가 없습니다.

주문 서비스의 `web/OrderController.java`는 아래처럼 인터페이스에 의존하도록 작성되어 있습니다.

참고 **주문 서비스 - web/OrderController.java. UseCase 인터페이스 주입**

```
@RestController
@RequestMapping("/api/orders")
```

```java
@RequiredArgsConstructor
public class OrderController {
    private final CreateOrderUseCase createOrderUseCase;   // 인터페이스 주입
    private final GetOrderUseCase getOrderUseCase;
    private final CancelOrderUseCase cancelOrderUseCase;

    @PostMapping
    public ResponseEntity<?> createOrder(...) {
        return Resp.ok(createOrderUseCase.createOrder(...));  // 인터페이스 메서드 호출
    }

    // GET /{orderId} — 주문 조회
    // PUT /{orderId} — 주문 취소
}
```

주문 서비스와 동일한 패턴으로 상품, 배달, 회원 서비스도 구성되어 있습니다.

내부 구조를 정리했으니, 이제 네 서비스를 운영 환경에 올릴 차례입니다.

오픈이 : "그런데 지금까지는 요청할 때마다 서비스 포트를 바꿔야 했어요. 이건 어떻게 해결하나요?"

선배 : "내부를 수정했으니, 외부에서 들어오는 단일 진입점도 만들면 좋겠네요."

3.5 Gateway와 MySQL 인프라

3.5.1 Nginx - API Gateway 라우팅

서비스가 늘어날수록 클라이언트는 모든 포트를 알아야 하고, 서비스 주소가 바뀌면 그때마다 코드를 고쳐야 합니다. 이때 **API Gateway**를 앞에 두면, 클라이언트는 하나의 진입점으로만 요청하고 게이트웨이가 URL 경로에 따라 알맞은 서비스로 전달합니다.

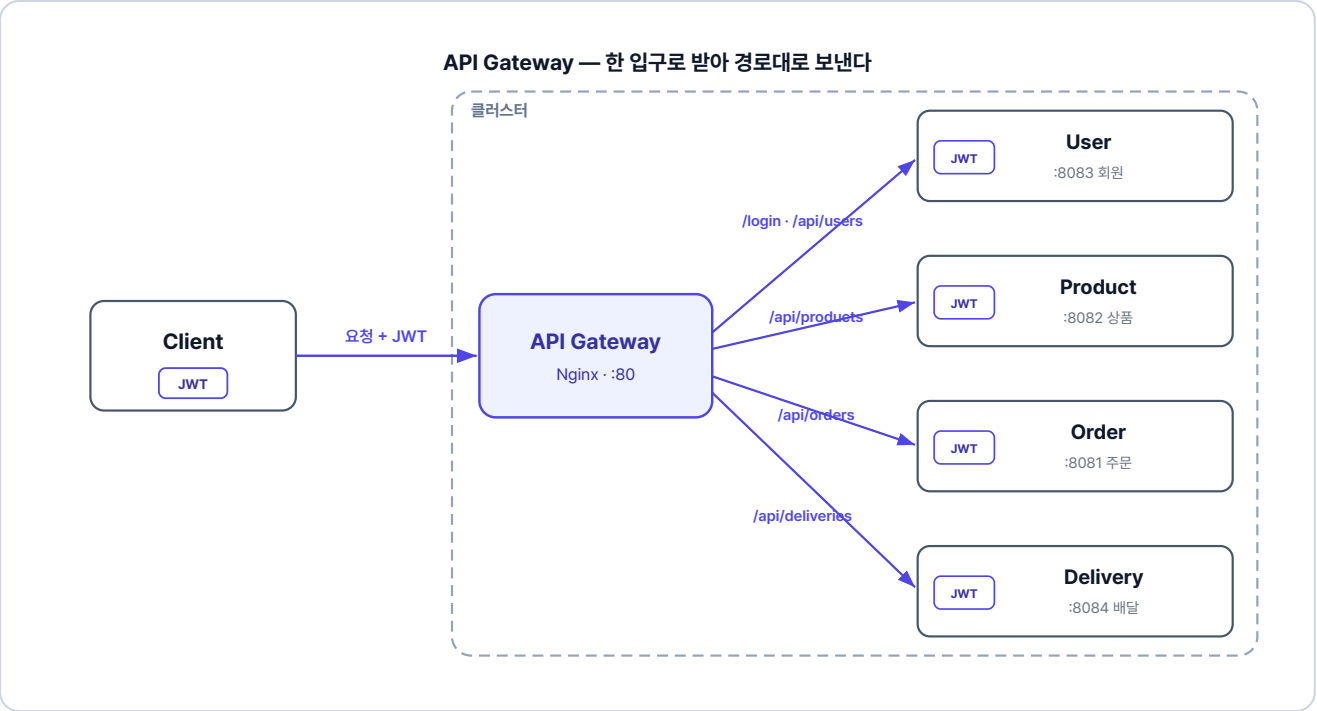


그림 3-6. 한 입구로 받아 경로대로 전달하고, 토큰 검증은 각 서비스가 합니다

`gateway/` 디렉토리에는 두 파일이 있습니다. 전체 설정은 GitHub을 참고하세요.

파일	역할
Dockerfile	Nginx 베이스 이미지에 설정 파일을 넣어 게이트웨이 컨테이너를 만듭니다.
nginx.conf	URL 경로별로 어느 서비스에 요청을 보낼지 정의합니다.

게이트웨이에서 토큰을 검증하는 방식

지금 이 책에서는 게이트웨이(Nginx)가 요청을 경로대로 넘기기만 하고, JWT는 각 서비스가 직접 검증합니다. 다른 방법으로, 게이트웨이가 입구에서 토큰을 먼저 검증하고 통과한 요청만 서비스로 보내는 구조도 널리 쓰입니다. 이때 게이트웨이는 토큰에서 꺼낸 사용자 정보를 헤더에 실어 전달하고, 내부 서비스는 게이트웨이를 지나온 요청을 이미 인증된 것으로 신뢰합니다.

게이트웨이에서 검증하면 인증이 한 곳에 모여 각 서비스는 비즈니스 로직에만 집중할 수 있습니다. 다만 Nginx만으로는 토큰 검증이 어려워 Spring Cloud Gateway 같은 도구가 필요합니다. 이 책은 단순함을 위해 서비스별 검증을 택했습니다.

3.5.2 MySQL - 데이터베이스 인프라

모든 서비스가 동일한 MySQL 인스턴스를 공유합니다. 서비스별로 테이블이 분리되어 있으나, 물리적으로는 단일 DB 인스턴스입니다.

> **이 책에서는 학습 편의를 위해 하나의 DB를 공유합니다.** 실제 MSA에서는 서비스마다 독립된 DB를 두지만, 학습 흐름을 익히는 데는 차이가 없으니 DB 구성보다 흐름에 집중해 주세요.

DB 컨테이너는 `db/` 디렉토리의 두 파일로 구성됩니다.

파일	역할
Dockerfile	MySQL 공식 이미지를 베이스로 초기화 SQL을 컨테이너에 넣어 띄웁니다.
init.sql	네 서비스에 필요한 테이블을 만들고 더미 데이터를 채웁니다.

3.6 Kubernetes - YAML로 선언하는 배포

오픈이 : "준비가 끝났으니, 이제 배포를 시작할까요?"

선배 : "그 전에 실무에 올리려면 한 가지 더 생각해야 해요. 지금은 Docker Compose로 컨테이너를 직접 띄우고 있잖아요? 만약 서버가 멈추거나 컨테이너가 갑자기 내려가면 어떻게 될까요?"

오픈이 : "음... 개발자가 서버에 접속해서 다시 띄워야 하지 않을까요?"

선배 : "사람이 24시간 내내 서버만 지켜볼 수는 없죠. 쿠버네티스는 우리가 원하는 상태를 정해 두면, 컨테이너가 죽더라도 알아서 다시 살려내서 그 상태를 유지해 줘요."

3.6.1 리소스 구조 설계

Kubernetes는 YAML 파일로 원하는 상태를 선언합니다. **"이 서비스는 이렇게 실행되어야 한다"** 고 파일에 적어두면, Kubernetes가 그 상태를 유지합니다.

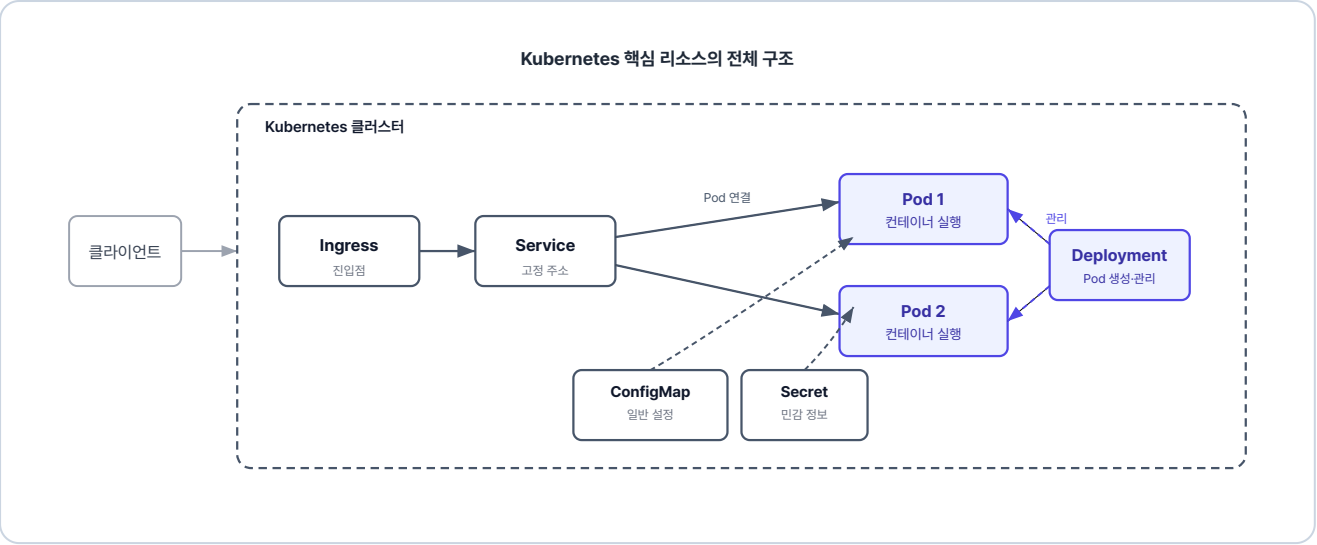


그림 3-7. Kubernetes 리소스 관계

각 Kubernetes 리소스의 역할을 정리하면 다음과 같습니다.

리소스	역할
ConfigMap	일반 환경변수(DB 주소 등)를 외부에서 주입합니다.
Secret	DB 계정·비밀번호 같은 민감 정보를 분리해 관리합니다.
Deployment	Pod를 어떻게 실행할지 정의하고, ConfigMap과 Secret을 한꺼번에 주입합니다.
Service	Pod에 고정 주소를 부여해 클러스터 안에서 안정적으로 통신할 수 있게 합니다.
Ingress	클러스터 외부 요청을 클러스터 안으로 들여보냅니다.

나머지 서비스(product, user, delivery)도 동일한 패턴입니다. 전체 YAML 파일은 레포의 `k8s/` 디렉토리를 참고하세요.

Gateway API로 두면 로컬 설정을 클라우드로 그대로 옮기기 쉽다

이 책은 외부 진입을 Ingress로 구성하지만, 후속 표준인 **Gateway API(우리가 만든 Gateway와 다름)** 를 쓰는 방법도 있습니다. Gateway API는 외부 진입점과 경로 규칙을 따로 정의합니다.

이렇게 나뉘어 있으면 로컬에서 클라우드로 옮길 때 편합니다. 경로 규칙은 그대로 두고, 진입점만 환경에 맞게 바꾸면 됩니다. 게다가 AWS EKS 같은 클라우드에서는 진입점조차 직접 만들 필요가 없습니다. 클라우드가 Gateway API 설정을 읽어 로드 밸런서를 자동으로 붙여 주기 때문입니다.

3.7 Minikube - 실행 및 결과 확인

3.7.1 Minikube 시작

Minikube는 로컬 PC에 가벼운 Kubernetes 클러스터를 만들어주는 도구입니다. 설치되어 있지 않다면 OS에 맞게 먼저 설치합니다.

터미널 **Minikube 설치**

```
# macOS
brew install minikube

# Windows
winget install Kubernetes.minikube
```

설치한 뒤 새 터미널을 열고, Docker Desktop이 실행 중인 상태에서 아래 명령을 입력하면 클러스터가 생성됩니다.

터미널 **Minikube 시작**

```
minikube start
```

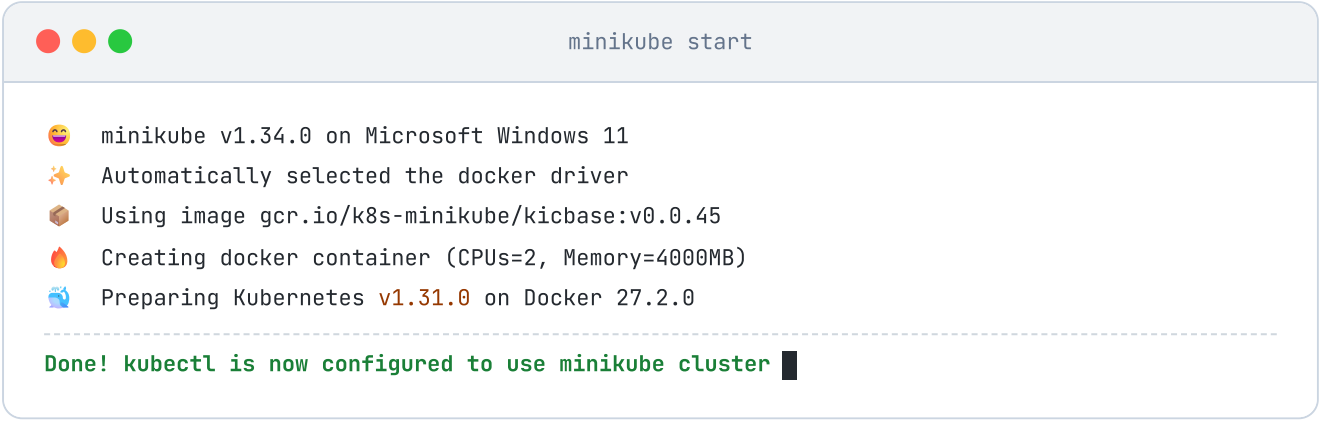


그림 3-8. Minikube 시작

3.7.2 이미지 빌드

`minikube image build`는 Minikube 내부에 직접 이미지를 빌드합니다.

터미널 **이미지 빌드**

```
minikube image build -t metacoding/db:1 ./db
minikube image build -t metacoding/order:1 ./order
minikube image build -t metacoding/product:1 ./product
minikube image build -t metacoding/user:1 ./user
minikube image build -t metacoding/delivery:1 ./delivery
minikube image build -t metacoding/gateway:1 ./gateway
```



그림 3-9. 이미지 빌드 결과

3.7.3 배포 순서

네임스페이스를 먼저 생성하고, DB가 준비된 뒤에 나머지 서비스를 배포합니다.

터미널 **배포 순서**

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. DB 관련 리소스 먼저 배포
kubectl apply -f k8s/db

# 3. 각 서비스 배포
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway

# 4. Ingress 활성화 (Minikube에서는 애드온 활성화 필요)
minikube addons enable ingress
```

kubectl apply · 네임스페이스 + 6개 서비스

```
namespace/metacoding created
secret/db-secret created
deployment.apps/db-deploy created
service/db-service created
configmap/order-configmap created
secret/order-secret created
deployment.apps/order-deploy created
service/order-service created
  … product · user · delivery · gateway 동일 패턴 …
ingress.networking.k8s.io/gateway-ingress created

전체 리소스 배포 완료
```

그림 3-10. 네임스페이스 생성 및 배포

3.7.4 배포 상태 확인

배포가 끝나면 모든 Pod가 제대로 실행되고 있는지 확인합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   42s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   38s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   36s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   35s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   33s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   30s

모든 Pod Running
```

그림 3-11. Pod 상태 확인

모든 Pod가 `Running` 상태가 되면 배포 완료입니다.

3.7.5 서비스 접근

Ingress를 통해 외부에서 접속하려면 `minikube tunnel`을 실행합니다.

터미널 **외부 접근 터널**

```
minikube tunnel
```

`minikube tunnel`은 터미널을 점유합니다.

터널이 실행되면 `http://127.0.0.1:80`로 gateway-service에 접속할 수 있습니다. 회원 서비스가 발급한 토큰은 하루 동안 유효하므로, 챕터 2에서 받은 토큰을 그대로 써서 아래 주문을 생성합니다. 만료됐다면 같은 방법으로 다시 발급받습니다.

MacBook Pro(상품 ID 1) 1개를 배달 주소와 함께 주문합니다.

Hoppscotch **주문 생성**

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

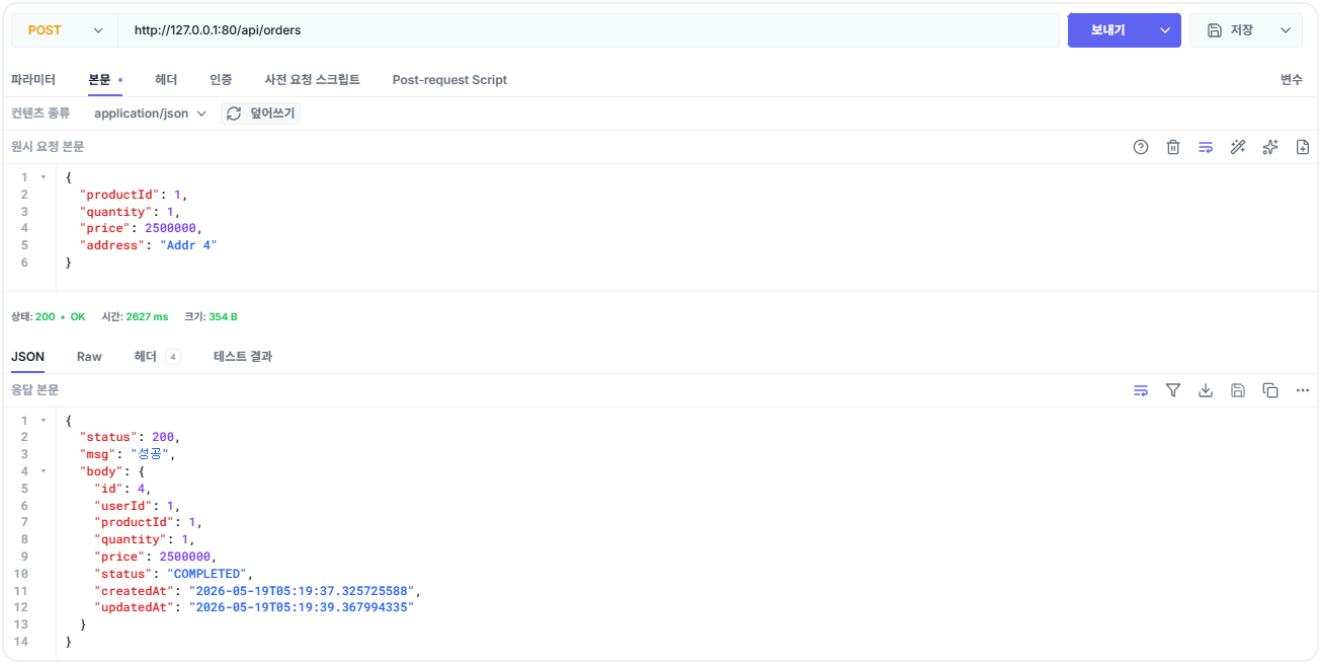


그림 3-12. 주문 결과 확인

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 **리소스 정리**

```
kubectl delete namespace metacoding
```

오픈이 : "쿠버네티스 위에서도 주문이 문제없이 만들어졌어요. 구조도 한결 깔끔해졌어요."

선배 : "맞아요. 이제 결합도가 낮아졌으니, 서비스를 수정해도 컨트롤러는 영향을 받지 않아요."

다음 챕터에서는 동기 호출 방식에서 메시지를 통한 비동기 호출 방식으로 전환합니다.

이것만은 기억하자

- → **DDD**로 비즈니스 규칙을 서비스가 아니라 도메인 객체에 둡니다.
- → **클린 아키텍처**(UseCase 인터페이스)로 컨트롤러가 구현이 아닌 인터페이스에 의존합니다. 덕분에 환경이나 테스트에 따라 구현을 바꿀 수 있습니다.
- → **Nginx API Gateway**로 여러 서비스의 진입점을 하나로 모으고, URL 경로에 따라 알맞은 서비스로 전달합니다.
- → **Kubernetes**로 원하는 상태를 선언하면, 컨테이너가 내려가도 그 상태를 유지합니다.

CHAPTER

4

비동기 MSA

Kafka로 서비스를 분리하다

이번 챕터가 끝나면

- 동기 호출의 한계와 **비동기 메시지** 방식을 이해할 수 있습니다.
- **Kafka**로 메시지를 발행하고 구독하는 방식을 이해할 수 있습니다.
- **orchestrator**가 여러 서비스의 주문 흐름을 조율하는 방식을 이해할 수 있습니다.
- 분산 트랜잭션을 **Saga 패턴**으로 단계별 처리하고, 실패 시 **보상 트랜잭션**으로 되돌리는 방식을 이해할 수 있습니다.

챕터 4. 비동기 MSA - Kafka로 서비스를 분리하다

동기 호출로 서비스를 운영한 지 며칠 뒤, 오픈이는 에러 로그가 갑자기 늘어나는 것을 확인했습니다. 상품 서비스가 죽은 것이 원인이었습니다. 이후 쿠버네티스가 상품 서비스를 복구하자 에러 로그도 멈췄습니다.

문제는 그 사이에 들어온 주문이 전부 실패한 것입니다. 주문 서비스가 상품 서비스를 호출했지만, 상품 서비스가 죽어 있어 주문 실패가 발생했습니다.

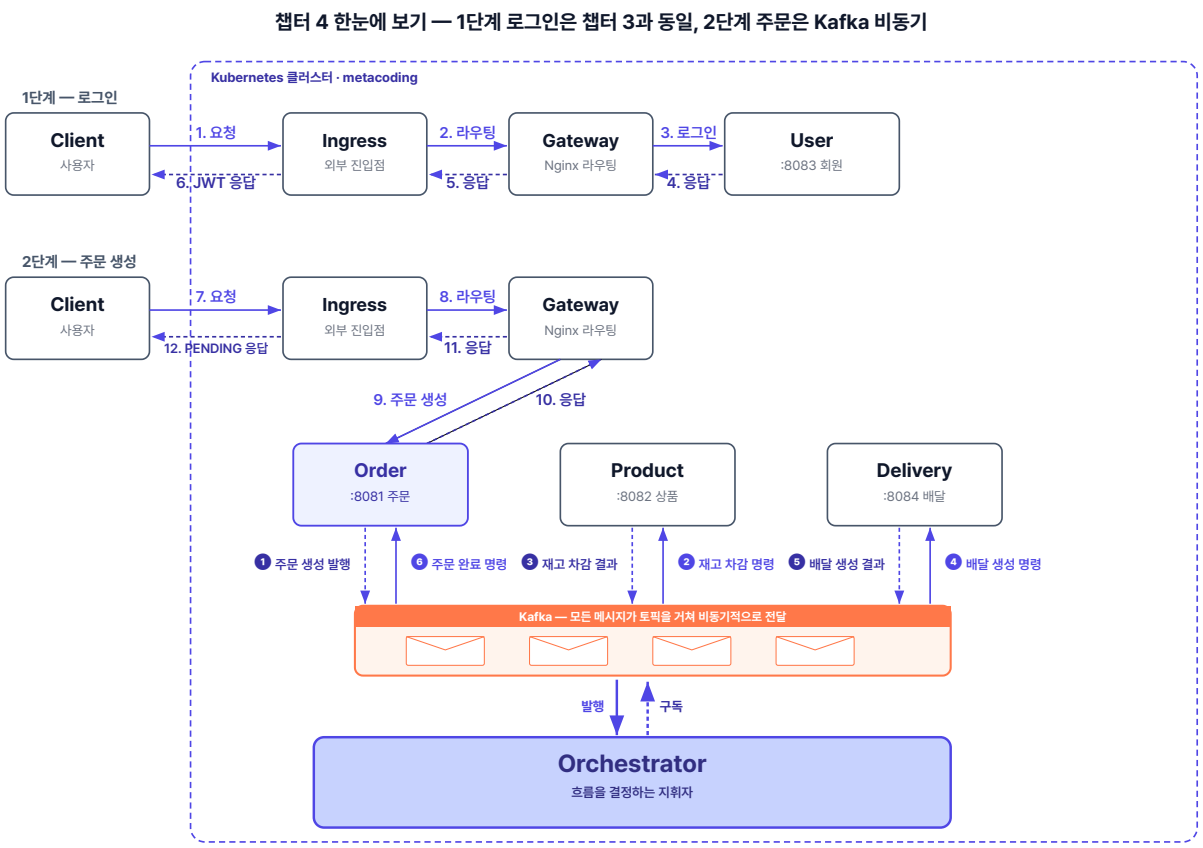
서로 직접 호출하니까, 하나가 죽으면 호출한 쪽도 같이 실패하는구나.

상황을 확인한 선배가 다가왔습니다.

선배 : "이거 지난번에 얘기했던 것처럼, 서비스끼리 동기적으로 직접 호출해서 그래요. 호출한 쪽은 상대가 응답할 때까지 기다리는데, 그 상대가 죽으면 결국 요청이 실패하는 거죠. 그래서 MSA는 동기 방식보다 메시지를 주고받는 비동기 방식을 써야 해요."

오픈이 : "메시지요? 메시지 방식은 어떻게 다른가요?"

선배 : "보내는 쪽은 메시지를 발행하고, 응답을 기다리지 않고 다음 작업을 진행해요. 받는 쪽은 자기 차례에 그 메시지를 읽고 다시 메시지를 발행하죠. 메시지는 한 번 발행되면 바로 사라지지 않아서, 받는 쪽이 잠깐 죽어도 메시지는 그대로 남아 있다가 복구되면 그때 처리돼요."



서비스끼리 직접 호출하지 않고, Kafka로 메시지를 비동기로 주고받습니다.

그림 4-1. 챕터 4 한눈에 보기 - 서비스·Orchestrator·Kafka 3층 구조

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex03
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex03` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex03 디렉토리

```
ex03/
├── db/                  # MySQL
├── delivery/            # 포트 8084
├── gateway/             # Nginx API Gateway
├── k8s/                 # Kubernetes YAML 파일 (kafka 포함)
├── orchestrator/        # Kafka 워크플로우 조율 (이번 챕터 신규)
├── order/               # 포트 8081
├── product/             # 포트 8082
└── user/                # 포트 8083
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. Kafka

이 챕터는 Kafka를 별도로 설치하지 않습니다. Kubernetes YAML(`k8s/kafka/`)로 Minikube 안에 띄우므로, 챕터 3에서 준비한 Docker Desktop과 Minikube만 있으면 됩니다.

4. 실습 순서

1. order-service의 REST 호출을 Kafka 이벤트 발행으로 교체
2. product-service · delivery-service에 Kafka Consumer/Producer 추가
3. 새 서비스 **orchestrator**에서 워크플로우 조율 로직 작성
4. Kubernetes에 Kafka + orchestrator 배포
5. 정상 주문 / 품절 보상 시나리오 검증

4.1 동기 호출의 한계 - 한 서비스가 멈추면 전체가 멈춘다

동기적 방식과 비동기적 방식을 비유를 통해 알아보겠습니다.

4.1.1 카운터 대기 vs 진동벨

먼저 커피를 주문하면 카운터에서 대기하는 방식입니다. 내가 주문한 커피가 나와야 다음 손님이 주문할 수 있습니다. 만약 커피 머신이 고장 나면 뒤에 줄 선 손님 전부가 기다려야 합니다. 이렇게 요청을 보낸 쪽이 응답이 올 때까지 기다리는 방식이 **동기(synchronous)** 호출입니다.

반대로 커피 주문 후 진동벨을 받으면 자리에 앉아 다른 일을 할 수 있습니다. 커피가 완성되면 벨이 울립니다. 뒤에 줄 선 손님도 곧바로 주문할 수 있습니다. 커피 머신이 멈춰도 주문을 먼저 받아 두고, 고친 뒤에 처리할 수 있습니다. 이렇게 요청을 보낸 쪽이 기다리지 않고 결과가 준비되면 따로 받는 방식이 **비동기(asynchronous)** 호출입니다.

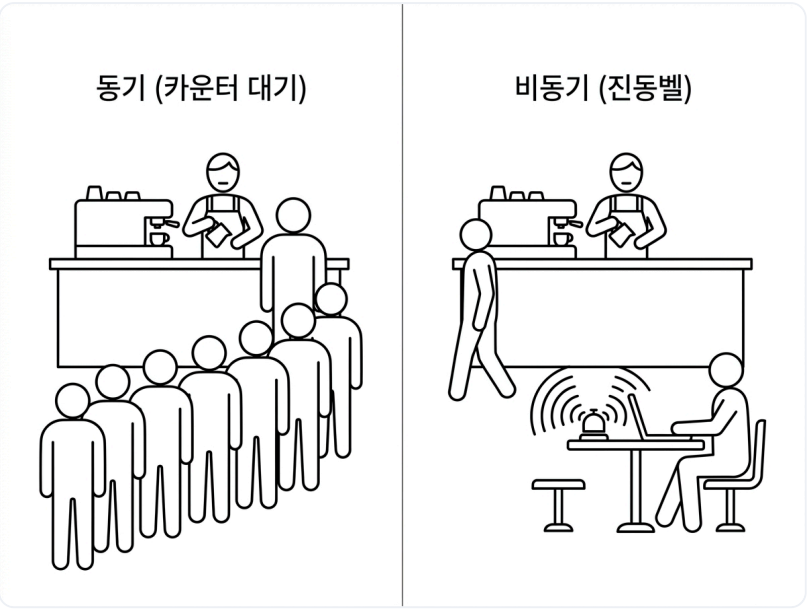


그림 4-2. 동기 vs 비동기 통신

카페의 진동벨처럼, MSA의 서비스도 비동기 통신을 위해 메시지를 사용합니다. 그렇다면 이 메시지는 어떤 방식으로 주고받을까요?

4.2 Kafka - 메시지를 전달하는 우체국

4.2.1 프로듀서·토픽·컨슈머

MSA 구조에서 비동기 메시지는 **Kafka**를 통해 주고받습니다. Kafka는 시스템 사이에서 비동기 통신을 담당하며, 메시지를 안전하게 전달해 주는 역할을 합니다.

> **Apache Kafka**는 대량의 메시지를 빠르고 안정적으로 주고받기 위해 만들어진 **분산 메시지 시스템**입니다. 받은 메시지를 바로 지우지 않고 일정 기간 보관하기 때문에, 필요하면 **지난 메시지를 다시 꺼내 볼 수도** 있습니다. **높은 처리량과 확장성** 덕분에 대규모 서비스에서 널리 사용합니다.

Kafka는 우체국과 같은 역할을 합니다. 발신자가 우편을 부치면, 우체국은 **일반 편지, 등기, 특송, 택배처럼 종류별로 나눠** 따로 보관합니다. 그리고 **집배원은 자기 담당의 칸에서만 우편을 꺼내 갑니다**. 우체국을 사이에 두고 우편을 주고받기 때문에, 비동기적으로 각자의 시간에 일을 처리할 수 있습니다.

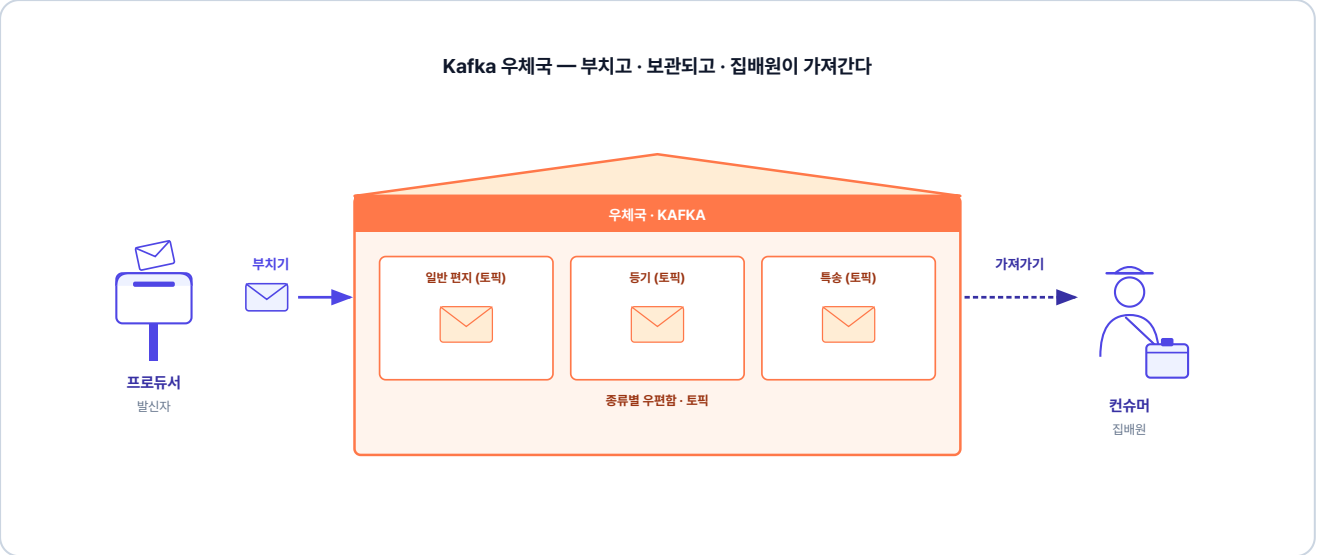


그림 4-3. Kafka 우체국 - 메시지를 종류별로 보관하고 전달하는 구조

그림 4-3의 Kafka 요소의 역할을 정리하면 다음과 같습니다.

구성요소	우체국 비유	하는 일
Kafka	우체국	메시지를 받아 보관하고 전달
토픽(Topic)	종류별 우편함 (일반/등기/특송 등)	메시지를 목적별로 나눠 담음
프로듀서(Producer)	발신자	토픽에 메시지를 발행
컨슈머(Consumer)	집배원	토픽을 구독해 메시지를 처리

프로듀서가 토픽에 메시지를 보내는 것을 **발행**(Publish), 컨슈머가 특정 토픽을 지정해 두고 메시지를 가져와 처리하는 것을 **구독**(Subscribe)이라고 합니다.

우체국이 우편을 일반·등기·특송으로 나눠 담듯, Kafka도 메시지를 **토픽이라는 우편함에 종류별로 나눠 담습니다. 토픽마다 이름이 있어서**, 프로듀서는 보낼 메시지에 맞는 **토픽 이름으로 발행**하고 컨슈머는 자기가 맡은 **토픽 이름으로 구독**합니다.



그림 4-4. Kafka의 프로듀서·토픽·컨슈머 구조

그렇다면 같은 일을 하는 컨슈머가 여러 대 떠 있을 때는 메시지를 어떻게 나눠 받아야 할까요?

4.2.2 컨슈머 그룹

상품 서비스를 2대로 늘려 운영한다고 가정해 보겠습니다. 두 서버가 같은 토픽을 구독하면 "재고 1개 차감" 메시지를 둘 다 받아 처리해, 재고가 2개 줄어듭니다. 이 중복 처리를 막아 주는 것이 **컨슈머 그룹**입니다. **같은 일을 하는 서버를 한 그룹으로 묶으면, 그 메시지는 그룹 안의 한 서버에만 전달됩니다.**

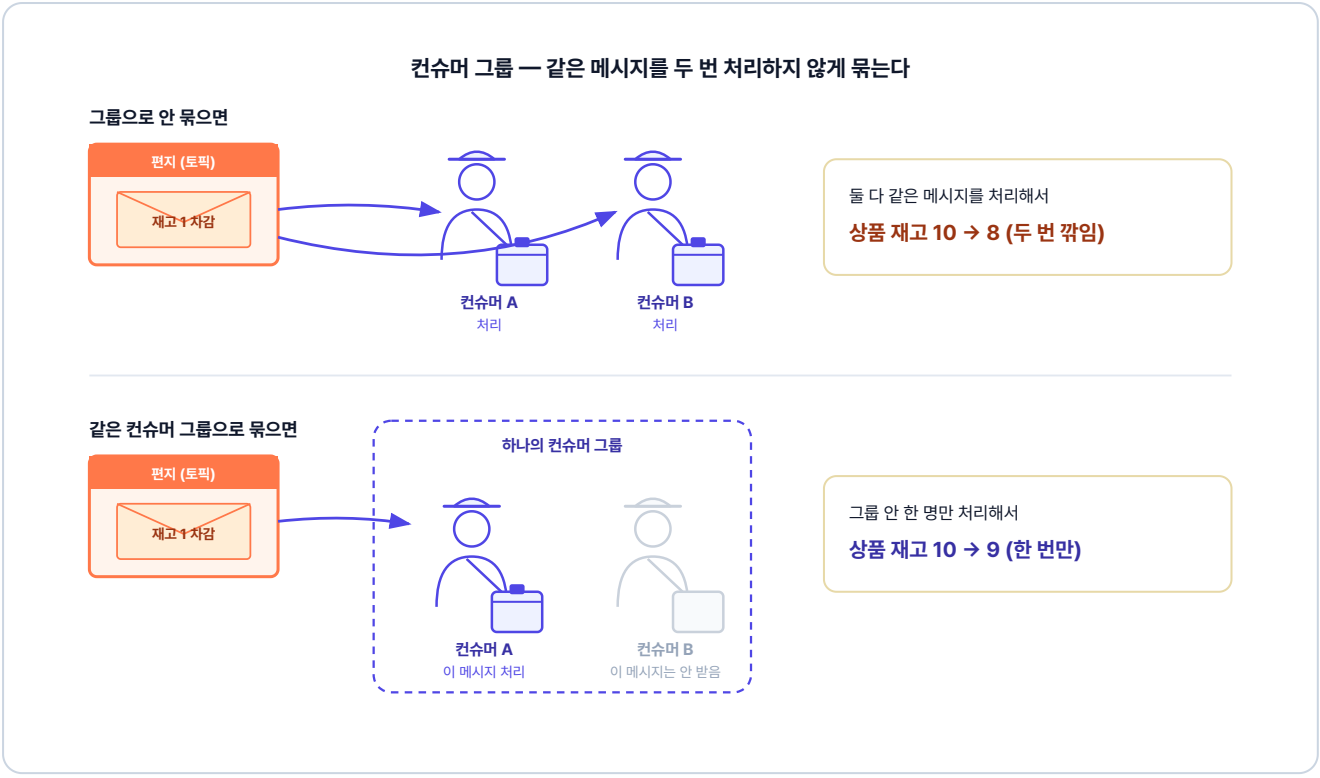


그림 4-5. 컨슈머 그룹 - 묶으면 한 번만 처리

Kafka 더 알아두기

- **컨슈머가 읽어도 메시지는 사라지지 않는다**: Kafka는 전통적인 메시지 큐와 달리, 컨슈머가 메시지를 읽어가도 파일 시스템에 그대로 보관합니다(기본 설정은 7일). 덕분에 서로 다른 컨슈머 그룹이 동일한 메시지를 각자의 속도대로 처음부터 다시 읽을 수 있고, 장애가 발생했을 때도 원하는 시점부터 재처리가 가능합니다.
- **서버 한 대가 죽어도 메시지는 안전하다**: Kafka는 보통 여러 대의 서버를 클러스터로 묶어서 운영하며, 메시지를 여러 서버에 복제해 둡니다. 따라서 특정 서버 한 대에 장애가 발생하더라도, 복제본을 가진 다른 서버가 즉시 역할을 넘겨받기 때문에 메시지 유실 없이 서비스를 안정적으로 유지할 수 있습니다.

오픈이 : "각 서비스가 메시지 방식으로 처리하다가 중간에 실패하면, 보상 트랜잭션은 어떻게 관리하나요?"

선배 : "그건 흐름을 누가 관리하느냐에 따라 달라요. 지금처럼 각 서비스가 서로 메시지를 발행하고 구독하며, 실패 시에도 직접 보상 메시지를 발행할 수 있어요. 다만 이 방식은 중간에 실패하면 어디까지 됐는지 알기 어려워 보상도 까다로워요. 반대로 전체 흐름을 관리하는 **지휘자**를 두고 지휘자가 메시지 발행과 구독을 관리하면, 어디까지 진행됐는지 알고 있으니 **이미 끝난 단계만 골라 되돌릴** 수 있어요."

4.3 Orchestration Saga - 지휘자가 흐름을 조율하다

하나의 주문을 여러 서비스가 단계별로 처리하다 보면, 중간 한 단계가 실패해도 앞 단계는 이미 데이터에 반영돼 있습니다. 이때 끝난 단계를 취소하는 **보상**으로 데이터를 맞추는 방식을 **Saga 패턴**이라고 합니다.

Saga 패턴에서 전체 흐름을 지휘자가 중앙에서 관리하는 방식을 **Orchestration Saga**라고 합니다. 흐름을 한 곳에 모으면 상태가 한 곳에 있어 추적과 보상이 단순해지고, 각 서비스는 자기 일에만 집중할 수 있습니다.

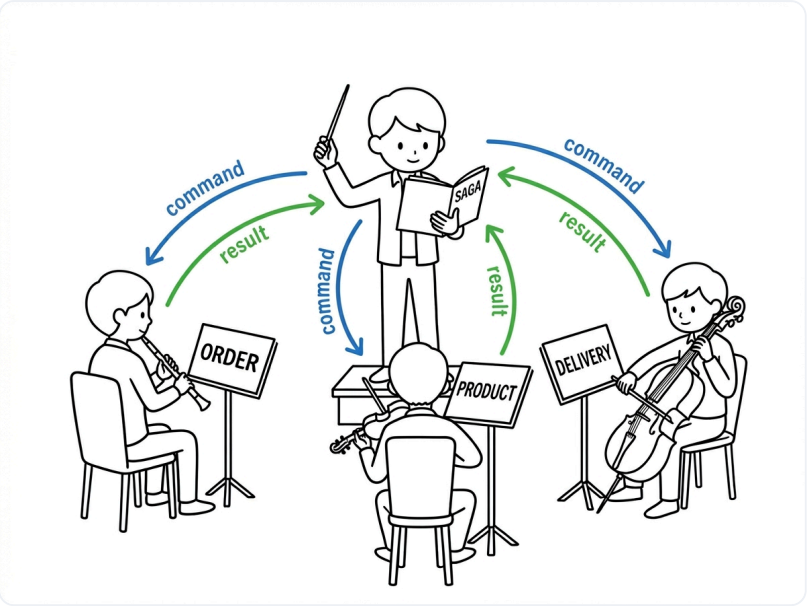


그림 4-6. Orchestration Saga 구조

챕터 2~3에서 구현한 주문 서비스 중심의 보상 트랜잭션 관리도 일종의 Orchestration Saga입니다.

이번 챕터에서는 조율 역할만 전담하는 **별도 orchestrator 서비스**를 두는 구조를 사용합니다. 주문 요청이 들어오면 orchestrator가 재고 차감, 배달 생성, 주문 완료 단계를 순차적으로 지휘합니다.

> **Saga를 구현하는 방식에는 Orchestration 외에 Choreography도 있습니다.** 중앙 지휘자 없이 **각 서비스가 서로 발행하고 구독하며 다음 단계를 이어 가는 방식**으로, 단계가 적을 때는 가볍지만 단계가 늘거나 보상이 복잡해질수록 전체 흐름이 여러 서비스에 흩어져 추적이 어렵습니다. 이 책은 상태를 한 곳에서 추적할 수 있는 Orchestration을 선택합니다.

4.3.1 이번 챕터에서 사용하는 토픽 맵

주문 흐름에서는 총 8개의 토픽을 사용합니다. 토픽은 orchestrator가 서비스에 작업을 지시하는 **명령(Command)** 과, 서비스가 결과를 돌려주는 **이벤트(Event)** 두 종류로 나뉩니다. 구분을 위해 토픽 이름에 `command`가 포함되면 명령, 없으면 이벤트로 정의합니다.

정상 흐름 (6단계)

단계	토픽	발행 → 구독	목적
1	`order-created`	order → orchestrator	새 주문 발생
2	`decrease-product-command`	orchestrator → product	재고 감소 명령
3	`product-decreased`	product → orchestrator	재고 감소 결과
4	`create-delivery-command`	orchestrator → delivery	배달 생성 명령
5	`delivery-created`	delivery → orchestrator	배달 생성 결과
6	`complete-order-command`	orchestrator → order	주문 완료 명령

보상 흐름 (2개)

토픽	발행 → 구독	목적
`cancel-order-command`	orchestrator → order	주문 취소
`increase-product-command`	orchestrator → product	재고 복구

핵심 흐름만 직접 다루므로, 각 토픽의 발행·구독 코드 전체는 깃헙 레포에서 확인합니다.

4.3.2 주문 요청 성공 흐름

이제 비동기 통신으로 주문 요청이 처리되는 전체 흐름을 따라가 보겠습니다.

먼저 주문 요청이 들어오면 주문 서비스는 클라이언트에게 **PENDING** 상태를 응답합니다. 이후 orchestrator가 아래 단계를 차례로 진행해 모두 성공하면 주문 상태가 **COMPLETED**로 바뀝니다.

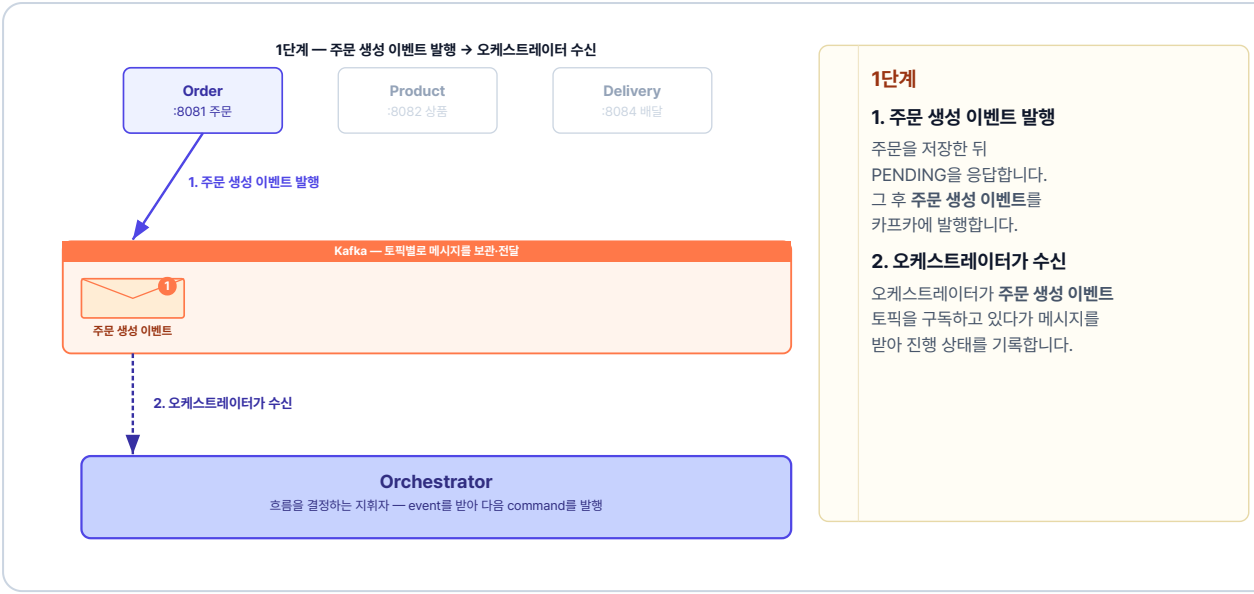


그림 4-7. 1단계 — 주문 생성 이벤트 발행 → 오케스트레이터 수신

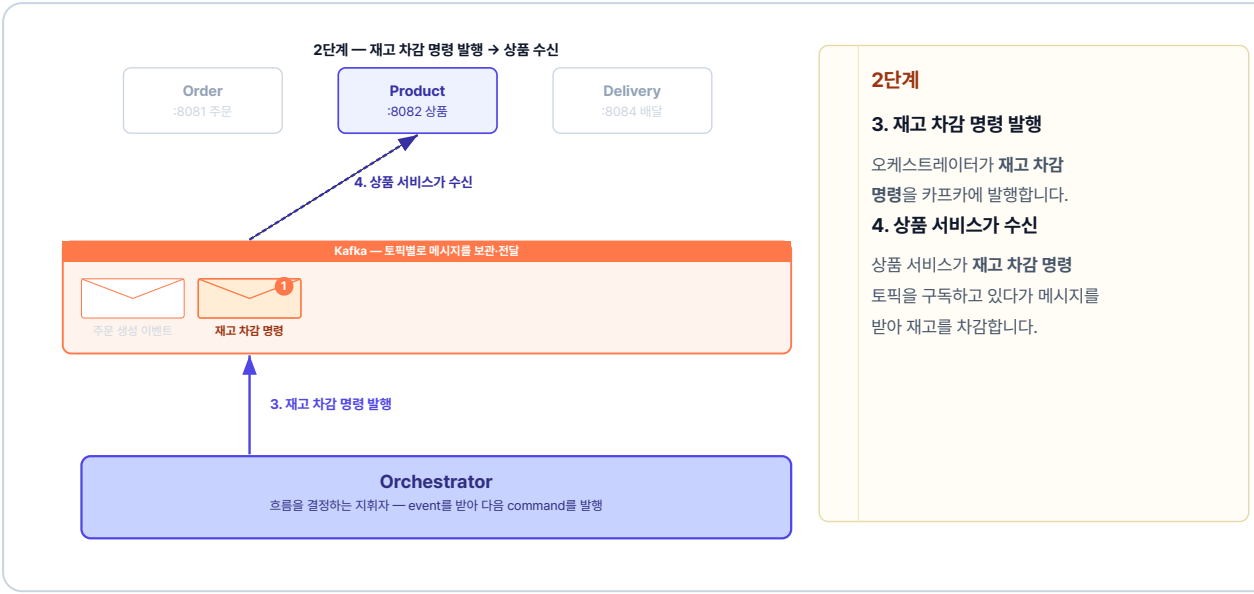


그림 4-8. 2단계 — 재고 차감 명령 발행 → 상품 수신



3단계

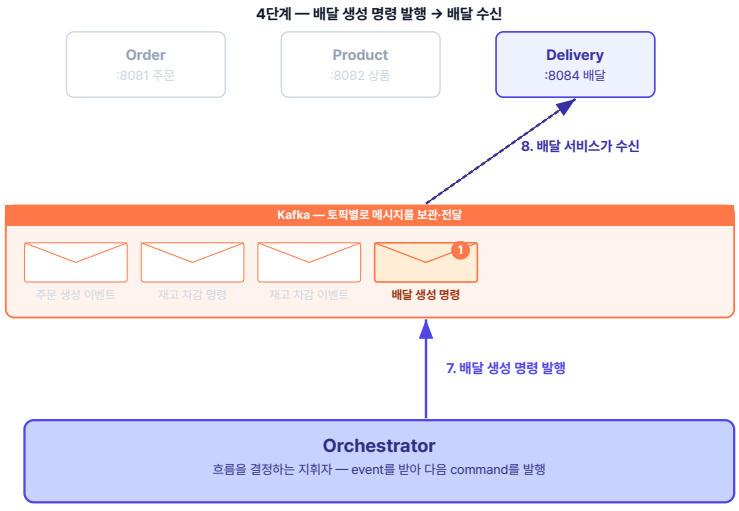
5. 재고 차감 이벤트 발행

상품 서비스가 재고를 줄인 뒤 성공 여부를 **재고 차감 이벤트**로 카프카에 발행합니다.

6. 오케스트레이터가 수신

오케스트레이터가 **재고 차감 이벤트** 토픽을 구독하고 있다가 메시지를 받아 성공을 확인합니다.

그림 4-9. 3단계 — 재고 차감 이벤트 발행(상품) → 오케스트레이터 수신



4단계

7. 배달 생성 명령 발행

오케스트레이터가 **배달 생성 명령**을 카프카에 발행합니다.

8. 배달 서비스가 수신

배달 서비스가 **배달 생성 명령** 토픽을 구독하고 있다가 메시지를 받아 배달을 생성합니다.

그림 4-10. 4단계 — 배달 생성 명령 발행 → 배달 수신

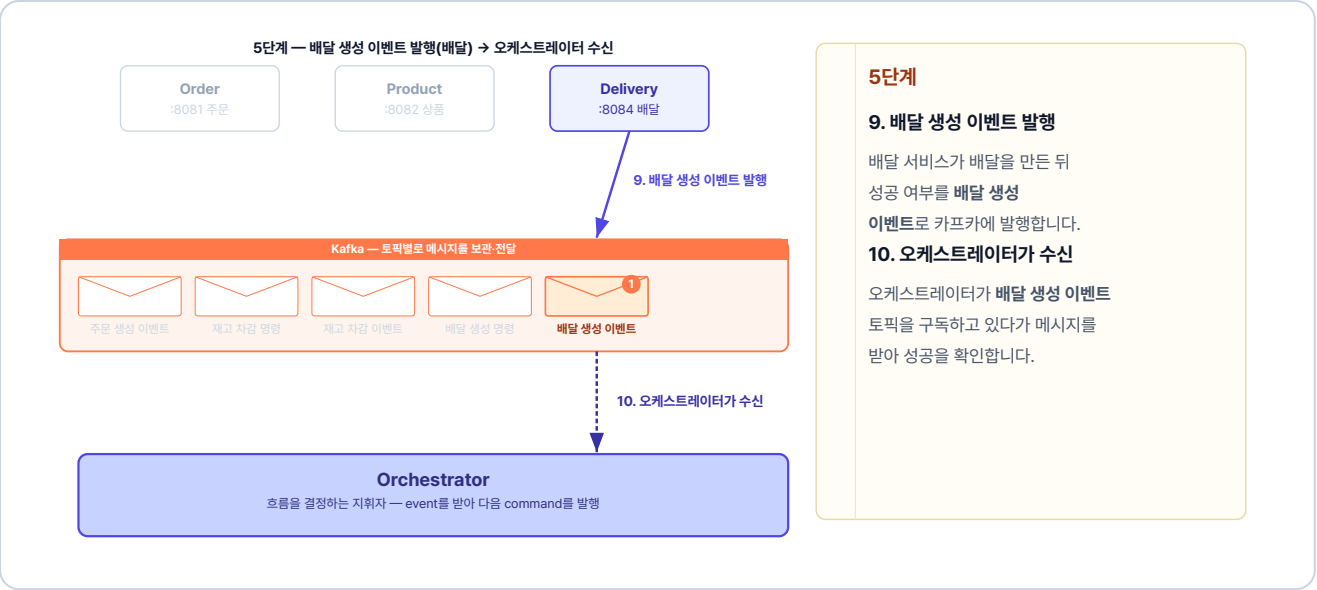


그림 4-11. 5단계 — 배달 생성 이벤트 발행(배달) → 오케스트레이터 수신

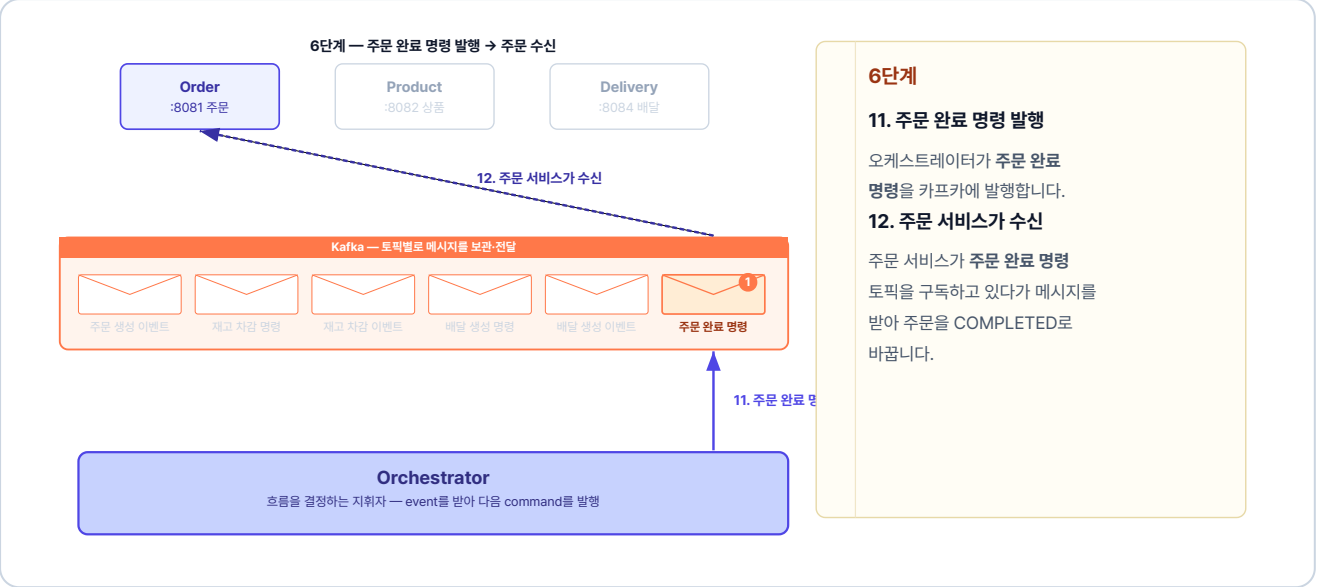


그림 4-12. 6단계 — 주문 완료 명령 발행 → 주문 수신

4.3.3 주문 요청 실패 흐름 (보상 트랜잭션)

만약 상품 서비스가 재고 차감에 실패하면 재고 차감 실패 이벤트를 발행합니다. orchestrator는 **이미 처리된 단계만** 역순으로 되돌립니다.

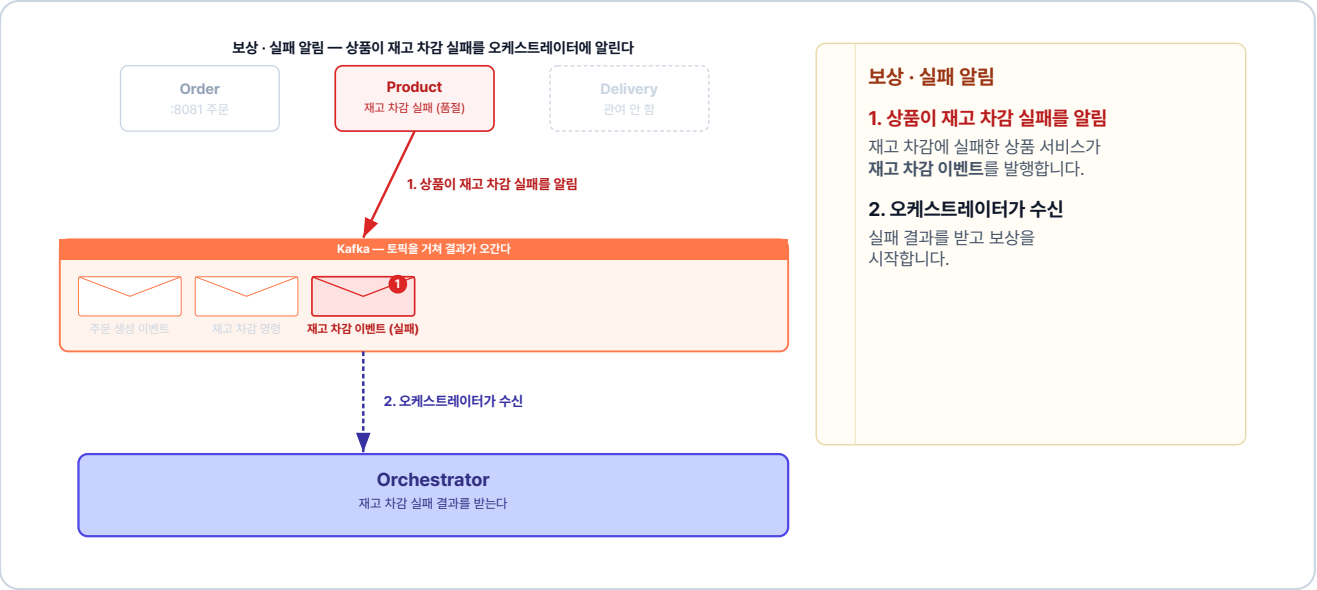


그림 4-13. 보상 · 실패 알림 — 상품이 실패를 알림

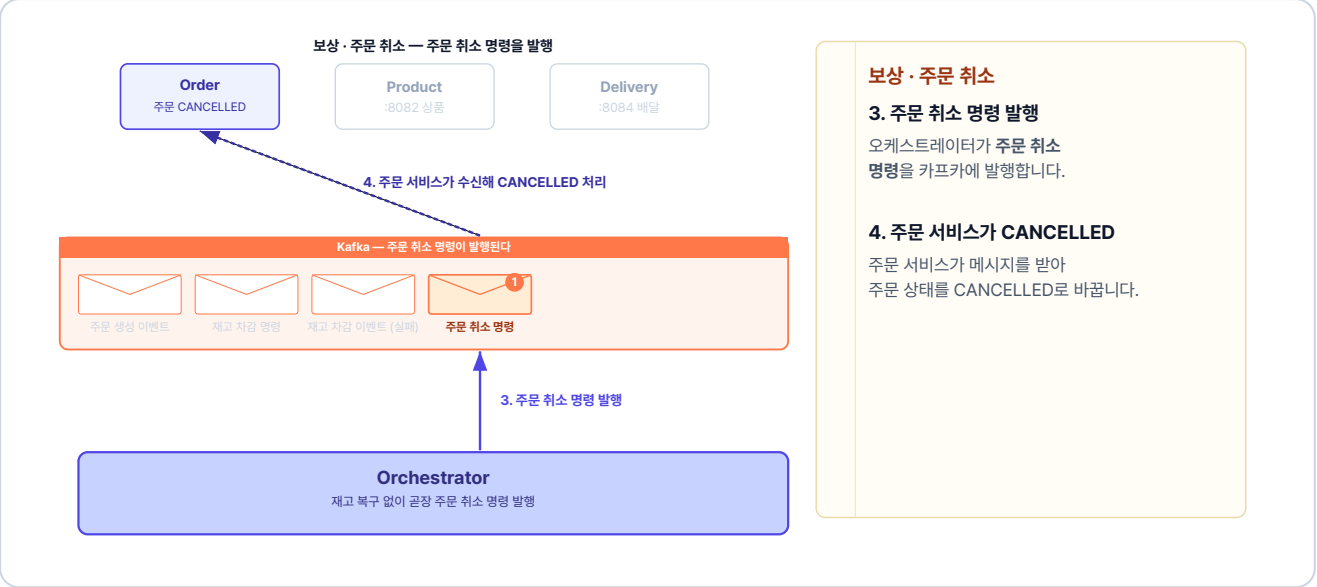


그림 4-14. 보상 · 주문 취소 — 주문 취소 명령 발행

4.4 Kafka로 주고받기 - 발행과 구독

4.4.1 발행 - 토픽에 메시지를 넣는다

프로듀서가 메시지를 발행할 때는 Spring이 제공하는 `KafkaTemplate` 을 사용합니다. **주문 생성 이벤트**를 발행하는 코드는 다음과 같습니다.

주문 서비스의 `adapter/producer/OrderEventProducer.java`를 열고 아래 메서드를 작성합니다.

실습 1 주문 서비스 - adapter/producer/OrderEventProducer.java. 주문 생성 이벤트 발행

```
@Component
@RequiredArgsConstructor
public class OrderEventProducer {
    private final KafkaTemplate<String, Object> kafkaTemplate;

    public void publishOrderCreated(OrderCreatedEvent event) {
        // "order-created" 토픽에 이벤트를 넣는다
        kafkaTemplate.send("order-created", event);
    }
}
```

`kafkaTemplate.send`에 `order-created` 같은 토픽 이름과 보낼 이벤트를 넘기면 해당 토픽으로 메시지가 들어갑니다.

주문 서비스는 주문을 **PENDING** 상태로 저장하고, **주문 생성 이벤트 프로듀서**를 호출합니다.

주문 서비스의 `usecase/OrderService.java`를 열고 `createOrder` 메서드를 작성합니다.

실습 2 주문 서비스 - usecase/OrderService.java. 주문 저장 후 주문 생성 이벤트 발행

```
@Override
@Transactional
public OrderResponse createOrder(int userId, int productId,
        int quantity, Long price, String address) {
    // 1. 주문 생성
    Order createdOrder = orderRepository.save(
            Order.create(userId, productId, quantity, price));
    createdOrder.validateMinAmount();

    // 2. Kafka로 주문 생성 이벤트 발행
    orderEventProducer.publishOrderCreated(
            new OrderCreatedEvent(
                    createdOrder.getId(), userId, productId,
                    quantity, price, address)
    );

    return OrderResponse.from(createdOrder);
}
```

4.4.2 orchestrator - 흐름을 조율하는 코드

이벤트를 받아 **다음 명령을 정하는 일**은 orchestrator가 수행합니다. **주문 생성 이벤트**를 받아 **재고 차감 명령**을 발행하는 코드는 다음과 같습니다.

오케스트레이터 서비스의 `handler/OrderOrchestrator.java`를 열고 아래 메서드를 작성합니다.

실습 3 오케스트레이터 서비스 - handler/OrderOrchestrator.java. 재고 차감 명령 발행

```
@KafkaListener(topics = "order-created", groupId = "orchestrator")
public void orderCreated(OrderCreatedEvent event) {
    // 1. 주문 진행 상태를 메모리에 기록
    states.put(event.orderId(), new WorkflowState(
            event.orderId(), event.address(),
            event.productId(), event.quantity(), event.price()));

    // 2. 다음 단계: 재고 차감 명령 발행
    kafkaTemplate.send(
            "decrease-product-command",
            // 같은 주문은 같은 파티션으로 (순서 보장)
            String.valueOf(event.orderId()),
            new DecreaseProductCommand(event.orderId(),
                    event.productId(), event.quantity(), event.price()));
}
```

`@KafkaListener`의 `topics`는 구독할 토픽 이름, `groupId`는 컨슈머 그룹 이름입니다. 그리고 `WorkflowState`는 **주문의 진행 정보를 메모리에 들고 있는 객체**로, 결과 이벤트가 돌아오면 orchestrator는 이 기록을 보고 다음 명령을 정합니다.

4.4.3 구독 - 토픽의 메시지를 받는다

앞에서 orchestrator가 발행한 **재고 차감 명령**을 이번에는 상품 서비스가 받습니다. 받은 명령으로 재고를 줄이고, 그 결과를 **재고 차감 이벤트**로 발행합니다.

상품 서비스의 `adapter/consumer/ProductCommandConsumer.java`를 열고 아래 메서드를 작성합니다.

실습 4 상품 서비스 - adapter/consumer/ProductCommandConsumer.java. 재고 차감 명령 구독

```
@KafkaListener(topics = "decrease-product-command", groupId = "product-service")
public void decreaseProductCommand(DecreaseProductCommand command) {
    boolean isSuccess = false;
```

```
    // 1. 재고 차감 (성공하면 isSuccess = true)
    try {
        productService.decreaseQuantity(
                command.productId(), command.quantity(), command.price());
        isSuccess = true;
    } catch (Exception e) {
        // 재고 부족 등 실패는 isSuccess = false로 그대로 보고
    }

    // 2. 처리 결과를 '재고 차감 이벤트'로 발행
    productEventProducer.publishProductDecreased(
            new ProductDecreasedEvent(
                    command.orderId(), command.productId(),
                    command.quantity(), isSuccess));
}
```

상품 서비스는 명령을 받아 재고를 줄인 뒤, 성공이든 실패든 그 결과를 이벤트에 담아 돌려줍니다.

각 서비스 코드는 모두 같은 발행·구독 패턴이고, 토픽 이름만 다릅니다. 그래서 코드를 일일이 보지 않아도, 각 서비스가 무슨 토픽을 구독해 어떻게 처리하고 무엇을 발행하는지만 알면 전체 흐름이 보입니다.

각 서비스의 전체 코드는 GitHub에서 확인하세요.

이제 Kubernetes에 Kafka와 orchestrator를 추가하고 전체 시스템을 배포합니다.

4.5 Kubernetes - Kafka와 orchestrator 배포

Kafka를 도입하면서 추가되는 건 **Kafka 서버**와 **orchestrator 서버**입니다.

Kubernetes 리소스	역할
`kafka-deploy.yml`	Kafka를 실행하는 Pod를 정의하고 원하는 상태로 유지합니다.
`kafka-service.yml`	Kafka Pod에 고정 주소 `kafka-service:9092`를 부여해, 주문·상품 등 각 서비스가 이 주소로 Kafka에 연결합니다.
`orchestrator-deploy.yml`	orchestrator를 실행하는 Pod를 정의합니다.
`orchestrator-configmap.yml`	orchestrator에 Kafka 주소 같은 설정값을 환경변수로 주입합니다.

모든 서비스는 Kafka 서버의 주소 `kafka-service:9092`로 접근합니다. 각 서비스는 이 주소를 ConfigMap에 넣어 둡니다.

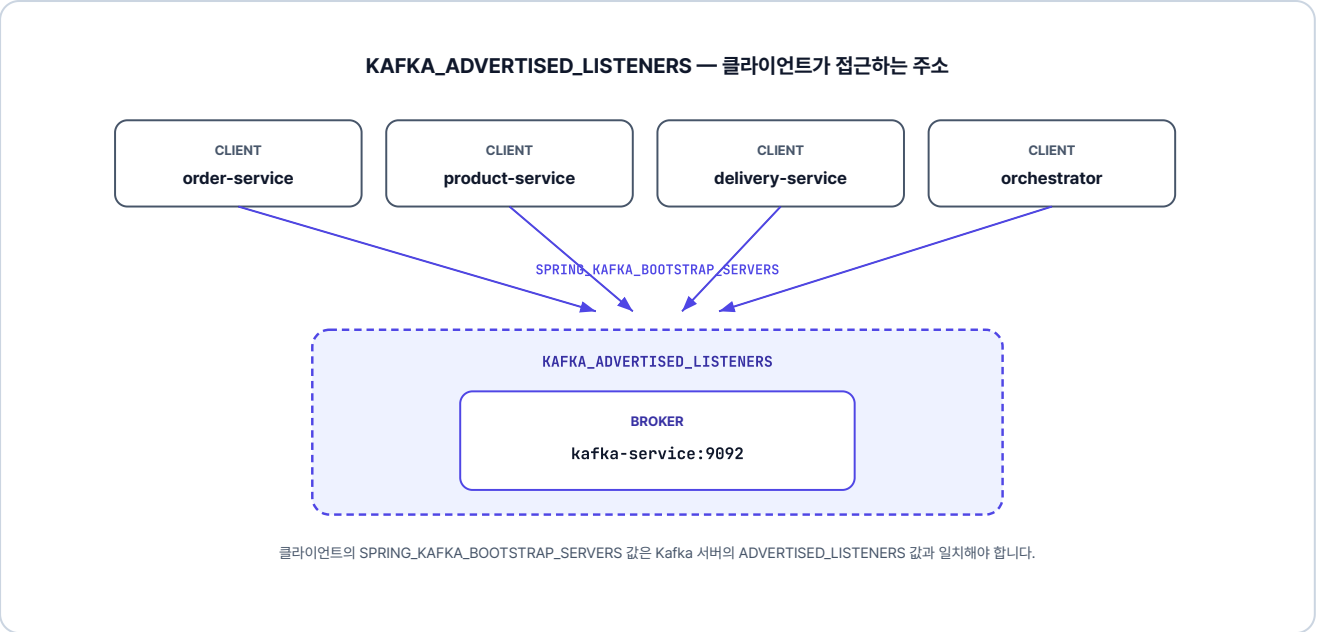


그림 4-15. 클라이언트가 kafka-service:9092로 접근

`kafka-deploy.yml`의 전체 환경변수는 GitHub에서 확인하세요. 각 변수의 역할은 주석으로 달려 있습니다.

KRaft 모드 알아두기

Kafka 서버는 크게 두 가지 역할을 합니다. **메시지를 받아 전달하는 브로커**, 그리고 **클러스터와 토픽 설정을 관리하는 컨트롤러**입니다.

과거에는 이 컨트롤러 역할을 **ZooKeeper**라는 외부 서비스에 따로 맡겨야 했습니다. 하지만 **KRaft(Kafka Raft)** 모드에서는 Kafka가 관리 역할까지 직접 도맡습니다. 덕분에 복잡한 ZooKeeper 연동 없이 Kafka 컨테이너 하나만으로 두 가지 역할을 모두 처리합니다.

실제 운영 환경에서는 데이터 유실을 막기 위해 여러 대의 브로커 노드를 구성해 안정성을 높입니다. 다만 이 책에서는 실습의 편의를 위해 하나의 컨테이너에 브로커와 컨트롤러를 함께 구성해 진행합니다.

4.6 실행 및 결과 확인

4.6.1 이미지 빌드

Minikube 내부에 이미지를 빌드합니다. 챕터 3 대비 orchestrator 서비스가 새로 추가됩니다.

터미널 이미지 빌드

```
minikube image build -t metacoding/db:2 ./db
minikube image build -t metacoding/gateway:2 ./gateway
minikube image build -t metacoding/order:2 ./order
minikube image build -t metacoding/product:2 ./product
minikube image build -t metacoding/user:2 ./user
minikube image build -t metacoding/delivery:2 ./delivery
minikube image build -t metacoding/orchestrator:2 ./orchestrator
```

4.6.2 배포 순서

Kafka가 준비되기 전에 서비스가 시작되면 연결 오류가 발생합니다. Kafka를 먼저 배포하고 준비된 것을 확인한 다음 나머지를 배포합니다.

터미널 배포 순서 (Kafka 우선)

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/gateway
kubectl apply -f k8s/orchestrator

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

kubectl apply · Kafka + 서비스 배포

```
namespace/metacoding created

[1] Kafka 우선 배포
deployment.apps/kafka-deploy created
service/kafka-service created
pod/kafka-xxx condition met (28s)

[2] 나머지 서비스 배포
  db · order · product · user · delivery · gateway · orchestrator …

8개 Deployment + 7개 Service 배포 완료
```

그림 4-16. Kafka 및 서비스 배포 실행

모든 Pod가 Running 상태인지 확인합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
kafka-deploy-7d4c8b9f5-2xk9p        1/1     Running   2m
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   90s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   88s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   85s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   83s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   80s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   78s
orchestrator-deploy-8c6f9b7d4-k9m4q 1/1     Running   75s

8개 Pod Running (Kafka + orchestrator 추가)
```

그림 4-17. Pod 상태 확인 (kubectl get pods)

4.6.3 서비스 접근

Ingress를 통해 외부에서 접속하기 위해 `minikube tunnel`을 실행합니다.

터미널 외부 접근 터널

```
minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80`로 gateway-service에 접속할 수 있습니다.

4.6.4 비동기 흐름 테스트

이제 AirPods (productId=3) 2개를 주문하는 API 요청을 보내 보겠습니다.

Hoppscotch 주문 생성

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 3,
  "quantity": 2,
  "price": 300000,
  "address": "Addr 4"
}
```



그림 4-18. 주문 생성 응답 (PENDING 상태)

챕터 3과 다르게 즉시 `PENDING` 상태로 반환됩니다. 잠시 후 주문 상태를 다시 조회하면, Kafka 이벤트가 처리되어 상태가 `COMPLETED`로 바뀐 것을 확인할 수 있습니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/4
```



그림 4-19. 주문 완료 확인 (COMPLETED 상태)

4.6.5 보상 트랜잭션 확인 - 품절 상품 주문

동기 방식에서는 주문 서비스가 트랜잭션을 관리했기 때문에 주문이 실패하면 주문 데이터가 **자동 롤백**되었습니다. 반면 비동기 방식에서는 주문이 **PENDING으로 먼저 저장**됩니다. 그래서 재고 감소가 실패하면 **보상 트랜잭션**에 의해 주문 상태가 `CANCELLED`로 변경됩니다.

iPhone 15(productId=2, 재고 0)로 확인해 보겠습니다.

Hoppscotch 주문 생성 (품절 상품)

```
POST http://127.0.0.1:80/api/orders

{
  "productId": 2,
  "quantity": 1,
  "price": 1300000,
  "address": "Addr 5"
}
```



그림 4-20. 품절 상품 주문 요청

잠시 후 상태를 확인하면 `CANCELLED`가 됩니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/5
```



그림 4-21. 주문 취소 확인 (CANCELLED 상태)

테스트가 끝났으면 이번 챕터에서 실행한 리소스를 정리합니다.

터미널 **리소스 정리**

```
kubectl delete namespace metacoding
```

오픈이 : "이제 직접 호출하지 않고 메시지로 주고받으니까, 다른 서비스가 일시적으로 멈춰도 주문은 계속 받을 수 있겠네요."

선배 : "맞아요. 서비스 간의 결합이 느슨해진 덕분이죠. 게다가 주문이 한꺼번에 몰려도 Kafka가 중간에서 메시지를 받아 두니, 받는 쪽은 감당할 수 있는 속도로 처리하면 돼요. 한 서비스에 장애가 나거나 부하가 걸려도 시스템 전체가 무너지지 않아요."

4.7 더 알아보기 - Outbox 패턴

오픈이 : "그런데 한 가지 걸리는 게 있어요. 주문을 저장하고 Kafka로 발행하는데, 발행이 실패하면 어떻게 되죠? 주문은 DB에 남아 있는데 이벤트만 사라지는 거 아닌가요?"

선배 : "정확히 봤어요. 그냥 넘기면 안 되는 부분이에요. 그걸 해결하는 방법으로 **Outbox 패턴**을 사용해요."

MSA에서는 보통 DB 변경과 메시지 발행을 함께 처리합니다. **문제는 두 작업이 서로 다른 시스템에서 일어나 하나의 트랜잭션으로 묶을 수 없다는 점입니다.** 그래서 주문 생성은 성공했는데 네트워크 오류나 서버 다운으로 발행만 실패하면, 오케스트레이터가 주문 생성을 알지 못해 상품·배달 처리가 시작되지 않고, 데이터가 어긋납니다.

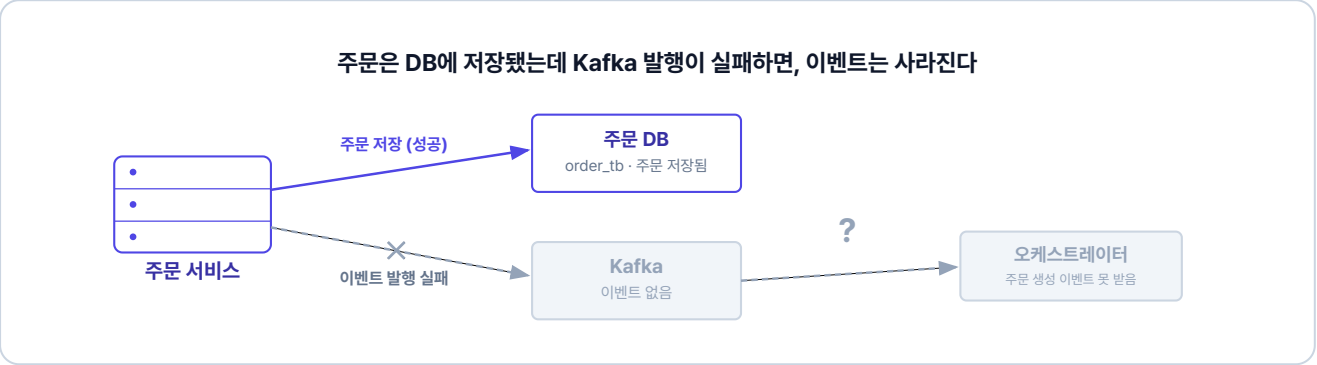


그림 4-22. 직접 발행의 한계 - DB 저장과 Kafka 발행이 별개라, 발행이 실패하면 이벤트가 유실됩니다

이 문제를 해결하는 것이 **Outbox 패턴**입니다.

먼저 주문 테이블과 같은 DB에 outbox 테이블을 하나 더 만듭니다. 그리고 주문 데이터를 저장할 때, 발행할 메시지도 outbox 테이블에 함께 저장합니다. 둘 다 같은 DB에 있으니 하나의 트랜잭션으로 묶을 수 있습니다. 주문과 메시지가 같이 커밋되거나 같이 롤백되므로, 주문만 저장되고 이벤트가 사라지는 일이 없어집니다.

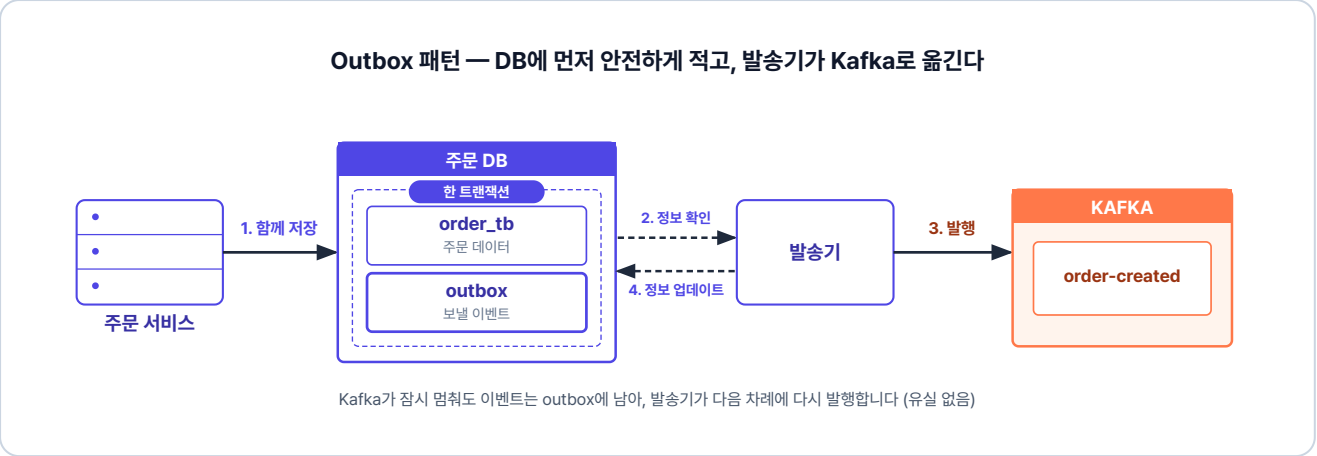


그림 4-23. Outbox 패턴 - 주문과 이벤트를 한 트랜잭션으로 저장한 뒤, 발송기가 outbox를 읽어 Kafka로 옮깁니다

그다음 **발송기(Relay)** 가 outbox 테이블을 주기적으로 확인합니다. 아직 발행하지 않은 메시지가 있으면 Kafka로 발행한 뒤, outbox 테이블에서 그 메시지의 상태를 보냄으로 업데이트합니다. 덕분에 발행이 실패해도 메시지가 outbox에 남아 있어, 다시 발행할 수 있습니다.

Outbox만으로 충분할까?

Outbox 패턴은 이벤트 유실을 막아주지만, 중복 발송까지 막지는 못합니다. 발송기가 이벤트를 보낸 직후 완료 처리를 하기 전에 멈추면, 다음 차례에 같은 이벤트를 또 보내기 때문입니다. 그래서 이벤트를 받는 쪽은 똑같은 요청을 여러 번 받아도 한 번만 처리한 것과 같은 결과를 내도록 방어해야 합니다. 이러한 성질을 **멱등성**이라고 합니다. 실무에서는 보통 이미 처리한 이벤트 ID를 기록해 두고, 같은 ID가 다시 들어오면 건너뛰는 방식으로 구현합니다.

이렇게 실시간 유실과 중복을 막더라도, 예상치 못한 시스템 오류로 데이터가 어긋날 수 있습니다. 따라서 규모가 큰 서비스에서는 주기적으로 누락되거나 어긋난 부분들을 찾아 배치 작업으로 보정합니다.

이 책에서는 패턴의 개념까지만 소개합니다. 구현은 다루지 않지만, 이벤트로 주고받는 시스템에서는 거의 빠지지 않는 장치입니다.

한편 현재 구조에서는 클라이언트가 처음에 **주문 대기** 상태만 응답받을 뿐, 이후 실제 처리가 완료되어도 그 결과를 따로 알 수 없습니다. 다음 챕터에서는 **웹소켓(WebSocket)** 을 도입해 이 문제를 해결하고, 주문 처리가 끝나는 즉시 클라이언트에게 실시간 알림을 보내는 방법을 알아보겠습니다.

이것만은 기억하자

- → 서비스끼리 REST로 직접 호출하는 대신 **Kafka**로 메시지를 주고받습니다. 이 **비동기 방식**으로 서비스 간 결합이 느슨해집니다.
- → **orchestrator**가 **명령**으로 각 서비스를 조율하고, 각 서비스는 **이벤트**로 결과를 알립니다.
- → **Saga 패턴**으로 분산 트랜잭션을 단계별로 처리하고, 실패한 단계는 **보상 트랜잭션**으로 역순으로 되돌립니다.
- → **Outbox 패턴**으로 주문과 이벤트를 한 트랜잭션으로 함께 저장해, 발행 실패로 인한 **이벤트 유실**을 막습니다.

CHAPTER

5

실시간 알림

주문 완료를 즉시 전달하다

이번 챕터가 끝나면

- 폴링과 푸시의 차이, 실시간 통신(**웹소켓**)이 필요한 이유를 이해할 수 있습니다.
- 비동기 완료 시점을 포착해 사용자에게 실시간으로 전달하는 흐름을 이해할 수 있습니다.

챕터 5. 실시간 알림 - 주문 완료를 즉시 전달하다

며칠 뒤, 베타 테스터로 새 시스템을 써 본 동료가 떨떠름한 표정으로 오픈이를 찾아왔습니다.

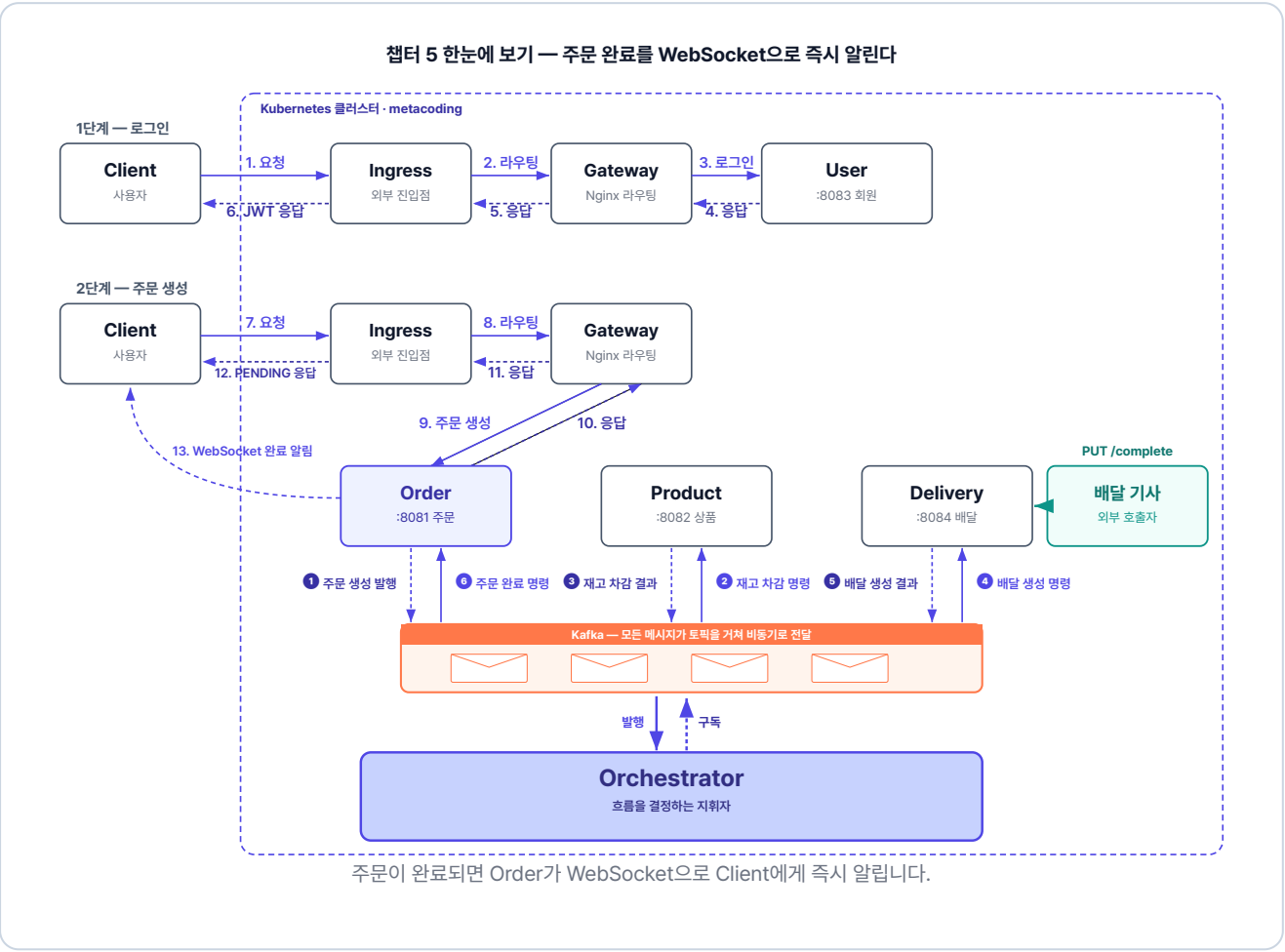
동료 : "어제 물건을 주문했는데, 화면이 계속 **처리 중**이더라고요. 끝났는지 알 수가 없어서 한참 뒤에 주문 내역을 다시 열어 보고서야 **주문 완료**된 걸 알았어요."

오픈이는 코드 흐름을 따라가 봤습니다. 주문이 생성되면 그대로 **PENDING** 상태로 응답 후, 사용자에게 **COMPLETED** 응답은 하지 않았습니다. 게다가 주문 완료는 실제 배달이 끝났는지와 무관하게, 배달이 생성되는 순간 곧바로 처리되었습니다.

이 문제를 들고 선배에게 갔습니다.

오픈이 : "서버에서 주문이 완료되었는데, 사용자는 처음 **PENDING** 상태만 응답을 받아서 주문이 완료된 사실을 알 수가 없어요. 이거는 어떻게 해결해야 하죠?"

선배 : "처리가 끝난 순간 사용자에게 알림을 줘야 해요. 서버에서 **주문 처리가 완료된 시점**을 감지해서 사용자에게 실시간으로 알려줄 방법이 필요하겠죠."



주문이 완료되면 Order가 WebSocket으로 Client에게 즉시 알립니다.

그림 5-1. 챕터 5 한눈에 보기 - 주문 완료를 웹소켓으로 즉시 알린다

준비하기

1. 소스 코드 클론

터미널 레포 클론

```
git clone https://github.com/metacoding-12-msa/start.git
cd start/ex04
```

완성 코드는 final 레포(`https://github.com/metacoding-12-msa/final`)의 `ex04` 폴더에서 확인할 수 있습니다.

2. 파일 구조

ex04 디렉토리

```
ex04/
├── db/                 # MySQL
├── delivery/           # 포트 8084 (배달 완료 API 추가)
├── frontend/           # Nginx + SockJS 클라이언트 (이번 챕터 신규)
├── gateway/            # Nginx API Gateway
├── k8s/                # Kubernetes YAML 파일 (kafka·frontend 포함)
├── orchestrator/       # Kafka 워크플로우 조율
├── order/              # 포트 8081 (웹소켓 Push 추가)
├── product/            # 포트 8082
└── user/               # 포트 8083
```

서비스마다 패키지 구조가 조금씩 다르므로, 코드를 작성할 파일 경로는 각 실습 코드블록 바로 위에서 안내합니다.

3. 실습 순서

1. 배달 서비스에 배달 생성·완료 분리 + 배달 완료 API + `delivery-completed` 이벤트 추가
2. orchestrator에 `delivery-completed` 처리 + `delivery-created` 성공 시 대기로 변경
3. 주문 서비스에 STOMP 웹소켓 설정 + 주문 완료 시 Push
4. SockJS 기반 index.html 프론트엔드와 Nginx 프록시 구성

5. Kubernetes에 frontend 추가 배포 → 통합 시나리오 검증

5.1 웹소켓 - 폴링의 한계를 넘다

5.1.1 폴링 vs 푸시

서버에 생긴 변화를 알아내는 방법은 크게 두 가지입니다.

택배가 왔는지 확인하려고 5분마다 현관문을 열어보는 방식이 있습니다. 도착 여부는 직접 문을 열어봐야만 알 수 있습니다. 이처럼 **클라이언트가 서버에 "처리가 완료되었나요?"라고 일정 간격으로 반복해서 묻는** 방식을 **폴링(Polling)** 이라고 합니다.

반면, 택배가 도착했을 때 초인종이 울리는 방식도 있습니다. 안에 있는 사람은 문을 계속 열어볼 필요 없이, 벨이 울리는 순간 도착 사실을 알게 됩니다. 이처럼 **클라이언트가 요청하지 않아도 서버에 변화가 생겼을 때 먼저 신호를 보내는** 방식을 **푸시(Push)** 라고 합니다.

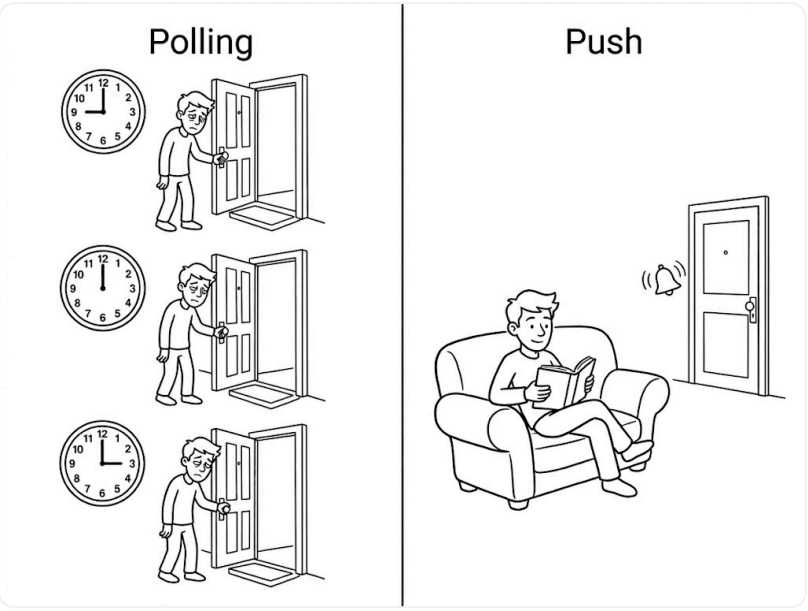


그림 5-2. 폴링 vs 푸시

폴링은 정해진 간격마다 서버에 요청을 보냅니다. 하지만 서버의 상태가 바뀌지 않았다면 의미 없는 요청과 응답을 반복하게 됩니다. 또한, 서버에 변화가 생기더라도 다음 요청 주기가 돌아올 때까지는 이를 감지할 수 없어, 설정한 간격만큼 데이터 전달이 지연되는 한계가 있습니다.

반면 푸시는 서버에 이벤트가 발생한 순간에만 신호를 보내기 때문에, 클라이언트는 지속적으로 상태를 확인하지 않고도 변경 사항을 즉시 수신할 수 있습니다.

5.1.2 웹소켓 - 실시간 양방향 통신

푸시를 구현하는 대표적인 기술이 바로 **웹소켓(WebSocket)** 입니다.

전통적인 HTTP 요청-응답 방식은 '편지'와 같습니다. 편지를 한 통 보내고 답장이 오면 한 번의 통신이 끝나며, 다음 상태가 궁금하면 다시 편지를 보내야 합니다. **클라이언트가 먼저 요청을 하지 않으면 서버는 아무것도 응답할 수 없는 구조**입니다.

반면 웹소켓은 '전화 통화'에 가깝습니다. 한 번 연결되면 끊지 않고 채널을 유지하므로, **상대가 묻지 않아도 어느 쪽이든 먼저 말을 할 수 있습니다**. 그래서 서버에 변화가 생기는 순간 알림을 클라이언트에게 보내는 것이 가능해집니다.

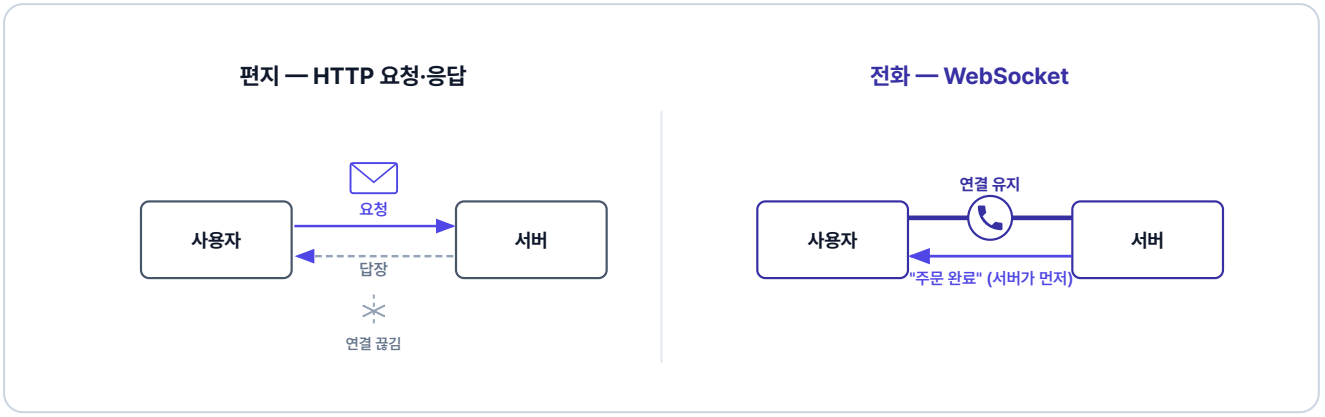


그림 5-3. 편지와 전화로 본 HTTP 요청·응답과 웹소켓

> **웹소켓(WebSocket)이란?** 클라이언트와 서버가 한 번 연결을 맺으면 이를 끊지 않고 유지하는 통신 방식입니다. 연결이 유효한 동안에는 서버가 클라이언트의 요청을 기다리지 않고도 데이터를 보낼 수 있어, 상태 변화를 실시간으로 전달할 수 있습니다.

지금은 주문이 생성됨과 동시에 완료 처리가 됩니다. 실시간 알림이 올바르게 작동하려면 실제 배달이 끝난 뒤에 주문이 완료되어야 하므로, 배달 완료 기능을 추가해야 합니다.

5.2 배달 완료 - 생성과 완료를 분리한다

이제부터 배달 생성이 발생하면 배달이 **PENDING**으로 남고, 배달 기사가 배달 완료 처리를 해야 **COMPLETED**가 됩니다. 배달이 만들어진 뒤 완료되기까지를 순서대로 보겠습니다.

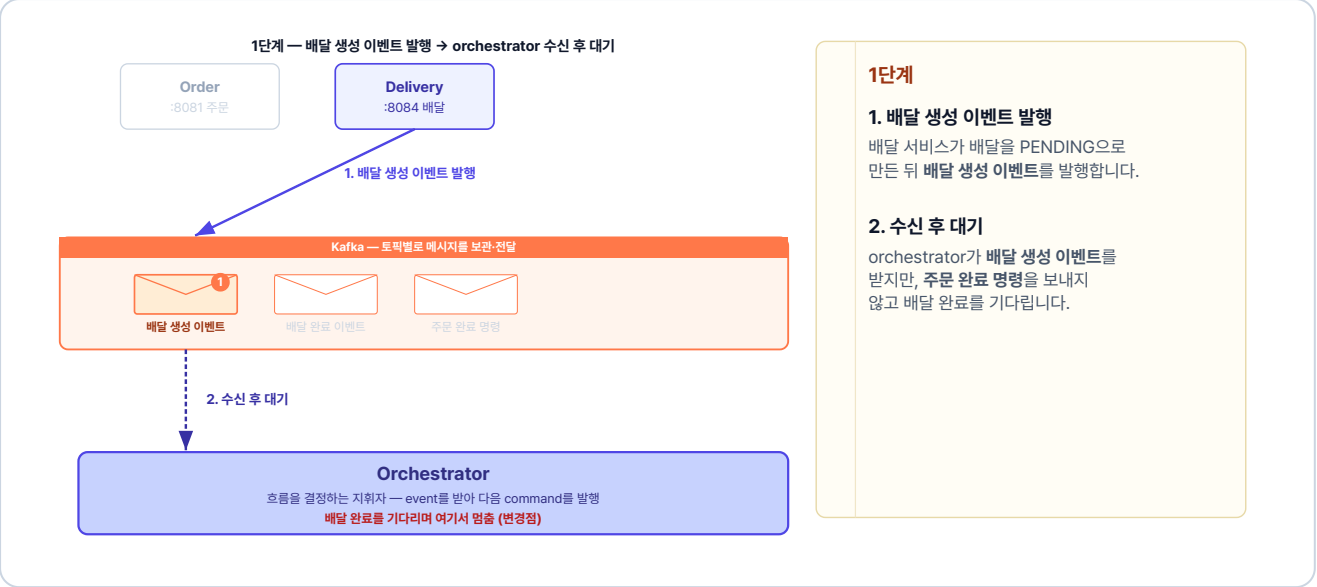


그림 5-4. 1단계 - 배달 생성 이벤트 발행 → orchestrator 수신 후 대기

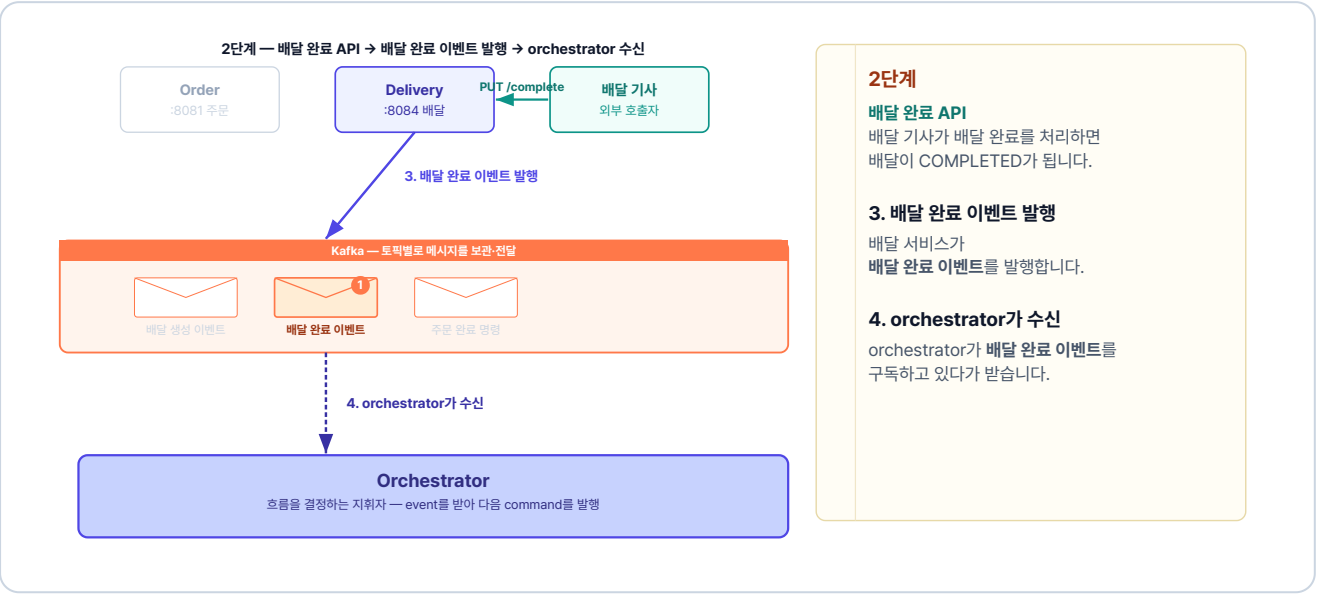


그림 5-5. 2단계 - 배달 완료 API → 배달 완료 이벤트 발행 → orchestrator 수신

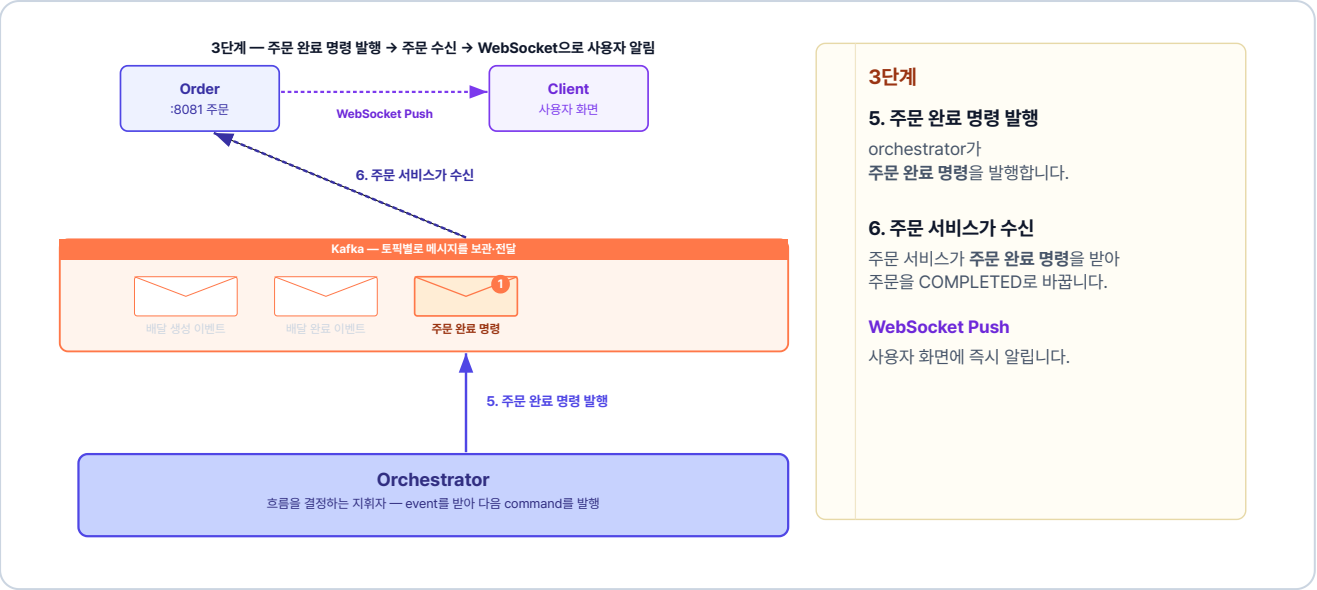


그림 5-6. 3단계 - 주문 완료 명령 발행 → 주문 수신 → 웹소켓 알림

이 세 단계가 차례로 이어지면, 배달이 실제로 완료되는 순간 사용자에게 완료 알림이 전달됩니다.

5.3 웹소켓 연결 흐름

이번에는 웹소켓 연결 흐름을 살펴보겠습니다. 주문 서비스가 보낸 알림이 사용자 화면에 뜨기까지, 크게 세 단계를 거칩니다.

5.3.1 브라우저와 주문 서비스의 연결 - HTTP에서 웹소켓으로

전화는 한쪽이 걸고 상대가 받아야 통화가 이어집니다. 마찬가지로 브라우저가 연결을 요청하고 주문 서비스가 받아들이면 **양방향 연결**이 열립니다. 이 연결은 **업그레이드 헤더**를 통해 일반 HTTP에서 **웹소켓**으로 바뀝니다.

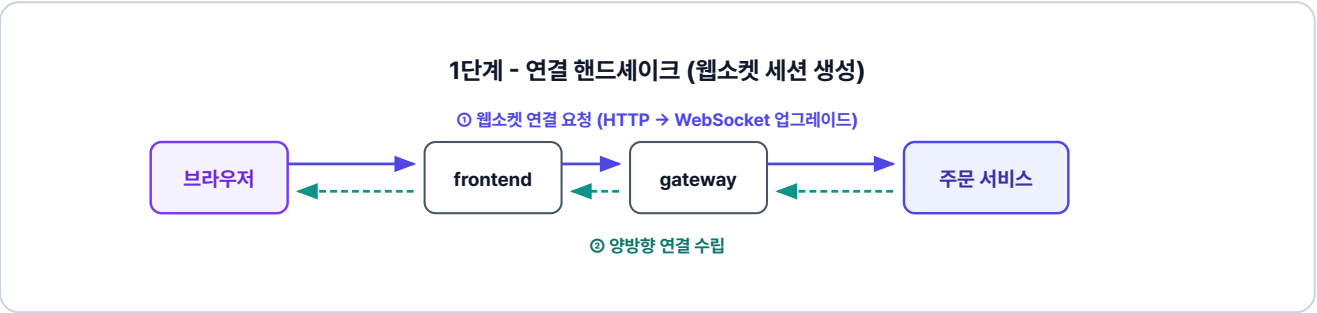


그림 5-7. 1단계 - 브라우저가 청하고 서버가 받아들여 양방향 연결이 열립니다

> **업그레이드 헤더란?** HTTP Upgrade 헤더는 클라이언트와 서버가 현재 사용 중인 HTTP 연결을 다른 프로토콜로 전환하기 위해 사용하는 헤더입니다. 주로 웹소켓 연결을 설정할 때 사용되며, 이를 통해 동일한 연결에서 새로운 통신 방식을 사용할 수 있습니다.

5.3.2 브라우저의 채널 구독 - 명부에 등록

웹소켓이 연결되더라도 서버는 누구에게 어떤 알림을 보낼지 스스로 알 수 없습니다. 따라서 브라우저가 먼저 서버에 자신이 받을 채널을 알려주며 **구독(Subscribe)** 해야 합니다.

브라우저가 특정 채널을 구독하면, 주문 서비스 안의 웹소켓 브로커는 **구독 명부**에 그 사실을 기록해 둡니다. 즉, 브라우저가 구독하고 서버가 명부를 작성해 관리하는 구조입니다. 이제 서버는 이 명부를 보고 정확한 대상에게 알림을 발송합니다.



그림 5-8. 2단계 - 브라우저가 구독하면 웹소켓 브로커가 구독 명부에 등록합니다

> **웹소켓 브로커란?** 메시지를 보내는 쪽과 받는 쪽 사이에서 전달을 중개하는 역할입니다. 어떤 클라이언트가 어떤 채널을 구독했는지 명부로 관리하다가, 메시지가 들어오면 같은 채널을 구독한 클라이언트에게 전달합니다.

5.3.3 주문 서비스의 알림 발송 - 같은 채널 찾아 전달

채널 주소에는 **알림을 받을 사용자**에 대한 정보가 들어 있습니다. 그래서 주문이 완료되면, 주문 서비스는 완료된 주문이 누구의 것인지부터 확인합니다. 그리고 주문한 사용자의 채널 주소를 구독 명부에서 찾아 알림을 보냅니다.

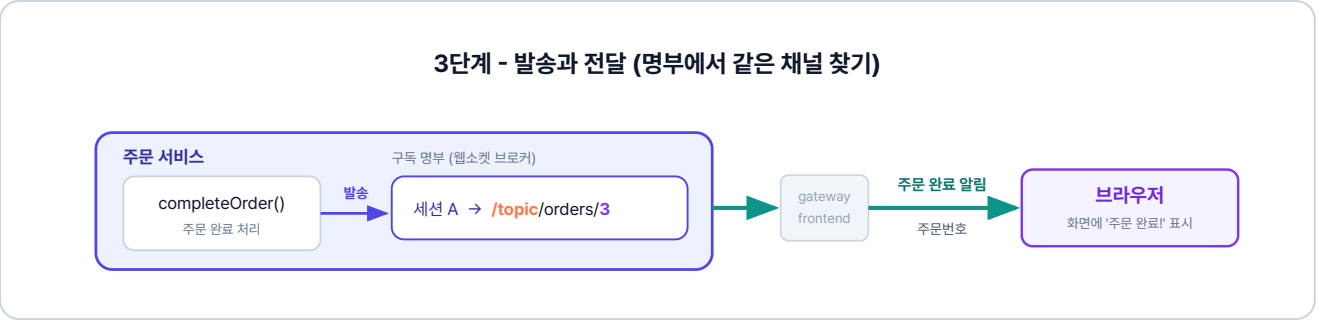


그림 5-9. 3단계 - 발송 주소와 같은 채널을 명부에서 찾아 구독한 브라우저에 보냅니다

세 단계가 모두 갖춰지면, 배달이 끝나는 순간 사용자 화면에 주문 완료가 표시됩니다.

이제 코드로 구현해 보겠습니다. 먼저 배달 서비스에서 배달의 생성과 완료 과정을 분리하는 작업부터 시작합니다.

5.4 배달 서비스 - 배달 완료 API

배달 서비스는 배달의 생성과 완료를 분리하고, 배달 기사가 호출할 배달 완료 API를 추가합니다.

5.4.1 createDelivery 수정 - 배달 생성·완료 분리

배달 생성 시 배달 완료 호출을 지우면 배달은 **PENDING**으로 남습니다. 배달 완료는 배달 기사가 직접 호출할 때까지 미뤄집니다.

`usecase/DeliveryService.java`의 `createDelivery`를 아래처럼 고칩니다.

실습 1 **배달 서비스 - usecase/DeliveryService.java. 생성 시 완료 호출 제거**

```
@Transactional
public DeliveryResponse createDelivery(int orderId, String address) {
    Delivery createdDelivery = deliveryRepository.save(
            Delivery.create(orderId, address));
    Delivery.validateAddress(address);
    // 삭제: createdDelivery.complete();  ← 생성 시 완료 호출 제거
    return DeliveryResponse.from(createdDelivery);
}
```

5.4.2 completeDelivery 추가 - 배달 완료 API

이번에는 배달 기사가 호출할 배달 완료 메서드를 추가합니다.

실습 2 **배달 서비스 - usecase/DeliveryService.java. completeDelivery 추가**

```
@Override
@Transactional
public DeliveryResponse completeDelivery(int deliveryId) {
    Delivery findDelivery = deliveryRepository.findById(deliveryId)
            .orElseThrow(() -> new Exception404("배달 정보를 조회할 수 없습니다."));
    findDelivery.complete();
    deliveryEventProducer.publishDeliveryCompleted(
            new DeliveryCompletedEvent(findDelivery.getOrderId()));
    return DeliveryResponse.from(findDelivery);
}
```

배달 기사가 배달 완료를 호출하면 배달이 완료되고, 배달 완료 이벤트가 발행됩니다. 이제 이 이벤트를 orchestrator가 받도록 수정합니다.

5.5 orchestrator - 배달 완료 이벤트 처리 추가

챕터 4에서는 배달 생성이 성공하면 orchestrator가 곧바로 주문 완료 명령을 발행했습니다. 이번에는 배달이 완료될 때 주문 완료 명령을 발행하도록 바꿉니다.

`handler/OrderOrchestrator.java`에 `deliveryCompleted` 리스너를 추가합니다.

실습 3 **오케스트레이터 서비스 - handler/OrderOrchestrator.java. 주문 완료 명령 발행**

```
@KafkaListener(topics = "delivery-completed", groupId = "orchestrator")
public void deliveryCompleted(DeliveryCompletedEvent event) {
    // 배달기사가 완료 API를 호출한 시점 → 주문 완료 명령 발행
    kafkaTemplate.send(
            "complete-order-command",
            String.valueOf(event.orderId()),
            new CompleteOrderCommand(event.orderId())
    );
}
```

5.6 주문 서비스 - STOMP로 실시간 Push 구현

마지막으로 주문 서비스가 주문 완료 명령을 받으면, 클라이언트에게 알리기 위해 웹소켓을 추가합니다.

5.6.1 웹소켓 설정

WebSocketConfig는 두 가지 주소를 등록합니다. 하나는 **클라이언트가 웹소켓으로 연결할 주소(`/api/ws/orders`)** 이고, 다른 하나는 **서버와 클라이언트가 알림을 주고받을 주소의 접두사(`/topic`)** 입니다.

주문 서비스의 `core/config/WebSocketConfig.java`를 열고 아래 클래스를 작성합니다.

실습 4 **주문 서비스 - core/config/WebSocketConfig.java. STOMP 웹소켓 설정**

```
@Configuration
@EnableWebSocketMessageBroker // 이 애너테이션을 붙이면 STOMP 메시징 기능이 켜집니다
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

    @Override
    public void configureMessageBroker(MessageBrokerRegistry config) {
        // /topic으로 시작하는 주소로 메시지가 오면,
        // 서버가 같은 주소를 구독한 클라이언트에게 전달합니다
        config.enableSimpleBroker("/topic");
    }

    @Override
    public void registerStompEndpoints(StompEndpointRegistry registry) {
        // 클라이언트가 웹소켓 연결을 시작할 주소입니다. 어떤 출처에서든 연결을 허용합니다
        registry.addEndpoint("/api/ws/orders")
                .setAllowedOriginPatterns("*")
                .withSockJS();
    }
}
```

웹소켓 위에 STOMP 프로토콜을 사용합니다. 웹소켓은 서버와 클라이언트를 계속 연결해 주지만, 연결만으로는 메시지를 누구에게 보낼지 가려내지 못합니다. 그래서 이 연결 위에 **STOMP(Simple Text Oriented Messaging Protocol)** 라는 메시징 규칙을 얹습니다. STOMP는 메시지마다 채널(주소)을 붙이고, **그 채널을 구독한 클라이언트에게만 전달하는 발행-구독 구조**를 제공합니다. 이 예제에서 서버는 `/topic/orders/{userId}` 채널로 보내고, 같은 채널을 구독한 사용자만 자기 주문 완료 알림을 받습니다.

클라이언트는 연결 주소로 웹소켓을 연 뒤, `/topic` 주소를 구독해 알림을 받습니다. 이때 웹소켓 브로커는 구독한 주소를 명부에 등록해 둡니다.

5.6.2 주문 완료 시 알림 발송

완료 알림은 `/topic/orders/{userId}` 채널로 보냅니다. 주문 완료 후 채널을 구독한 클라이언트에게 메시지를 전달합니다.

`usecase/OrderService.java`의 `completeOrder` 메서드를 아래처럼 수정합니다.

실습 5 **주문 서비스 - usecase/OrderService.java. completeOrder + WebSocket Push**

```
@Transactional
public void completeOrder(int orderId) {
    Order findOrder = orderRepository.findById(orderId)
            .orElseThrow(() -> new Exception404("주문을 찾을 수 없습니다."));
    findOrder.complete();
    // 추가: 이 채널을 구독한 클라이언트에게 메시지를 보냄
    messagingTemplate.convertAndSend("/topic/orders/" + findOrder.getUserId(), orderI
d);
}
```

핵심 코드 외에 주문 서비스에 필요한 설정은 깃헙 레포에서 확인합니다.

5.7 프론트엔드 연결

서버는 주문이 완료되면 채널로 알림을 보냅니다. 이제 **같은 채널을 구독해 알림을 받는 클라이언트**를 만들 차례입니다. 먼저 프록시가 웹소켓 연결을 끊지 않게 하고, 브라우저가 STOMP로 자기 채널을 구독하게 합니다.

5.7.1 업그레이드 헤더 전달

앞에서 본 업그레이드 헤더는 브라우저와 주문 서비스 사이의 frontend와 gateway를 거쳐야 합니다. 그런데 이 둘이 업그레이드 헤더를 넘기지 않으면 일반 요청처럼 처리돼 **연결이 끊깁니다**. 그래서 frontend와 gateway 두 곳의 nginx에 **업그레이드 헤더를 전달**하도록 설정합니다.

`frontend/nginx.conf`의 `/api/ws/` 위치에 아래처럼 설정합니다.

참고 **frontend/nginx.conf. 웹소켓 업그레이드 헤더**

```
location /api/ws/ {
    proxy_pass http://gateway;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection "upgrade";
}
```

`gateway/nginx.conf`의 `/api/ws/` 블록에도 같은 코드를 넣습니다. 그래야 브라우저에서 gateway까지 업그레이드 헤더가 끊기지 않고 전달됩니다.

5.7.2 클라이언트 - STOMP 구독

`frontend/index.html`은 서버 알림을 받아 화면에 주문 완료를 표시하는 STOMP 클라이언트입니다. 주문하기 버튼을 누르면, 먼저 웹소켓을 연결하고 자기 주문 완료 알림이 올 `/topic/orders/{userId}` 채널을 구독합니다. 알림 받을 준비를 마친 다음 주문 API를 호출합니다.

참고 **frontend/index.html. STOMP 연결과 구독**

```
// 1. /api/ws/orders로 웹소켓 연결
stomp = Stomp.over(new SockJS('/api/ws/orders?token=' + TOKEN));
stomp.connect({}, function () {
    // 2. 내 주문 알림 채널 구독
    stomp.subscribe('/topic/orders/' + userId, function (msg) {
        // 3. 서버가 보낸 메시지를 화면에 표시
        const orderId = JSON.parse(msg.body);
        status.textContent = '주문 완료! (주문번호: ' + orderId + ')';
    });
});
```

클라이언트와 서버 양쪽 코드에서 웹소켓 연결 주소(`/api/ws/orders`)와 구독 채널 주소(`/topic/orders/{userId}`)를 동일하게 맞춰주어야 정상적으로 알림이 전달됩니다. 전체 index.html은 깃헙 레포에서 확인합니다.



그림 5-10. 웹소켓 주소 일치 - 같은 색은 글자까지 똑같아야 동작합니다

5.8 전체 시스템 통합 테스트

이제 전체 시스템을 실행해, 주문 생성부터 완료 알림까지 전체 흐름을 확인합니다.

5.8.1 Kubernetes 리소스 정의

이전 챕터까지의 구성에서 frontend 서비스가 새로 추가됩니다. `k8s/frontend/` 폴더에 Deployment, Service, Ingress가 정의되어 있습니다.

파일	역할
frontend-deploy.yml	Nginx 기반 프론트엔드 Pod
frontend-service.yml	클러스터 내부 접근용 Service
frontend-ingress.yml	외부 요청을 frontend-service로 라우팅

이번 챕터부터 Ingress는 gateway-service 대신 **frontend-service**로 요청을 보냅니다. 프론트엔드 Nginx가 정적 파일을 직접 응답하고, `/api/` 요청만 gateway-service로 전달합니다.

5.8.2 이미지 빌드

Minikube 내부에 이미지를 빌드합니다.

터미널 **이미지 빌드**

```
minikube image build -t metacoding/db:3 ./db
minikube image build -t metacoding/gateway:3 ./gateway
minikube image build -t metacoding/order:3 ./order
minikube image build -t metacoding/product:3 ./product
minikube image build -t metacoding/user:3 ./user
minikube image build -t metacoding/delivery:3 ./delivery
minikube image build -t metacoding/orchestrator:3 ./orchestrator
minikube image build -t metacoding/frontend:3 ./frontend
```

5.8.3 배포

Kafka를 먼저 배포하고, 준비될 때까지 기다린 다음 나머지를 배포합니다.

터미널 **배포 순서 (Kafka 우선)**

```
# 1. 네임스페이스 생성
kubectl create namespace metacoding

# 2. Kafka 먼저 배포
kubectl apply -f k8s/kafka

# 3. Kafka가 준비될 때까지 대기
kubectl wait --for=condition=ready pod -l app=kafka -n metacoding --timeout=120s

# 4. 나머지 서비스 배포
kubectl apply -f k8s/db
kubectl apply -f k8s/gateway
kubectl apply -f k8s/order
kubectl apply -f k8s/product
kubectl apply -f k8s/user
kubectl apply -f k8s/delivery
kubectl apply -f k8s/orchestrator
kubectl apply -f k8s/frontend

# 5. Ingress 활성화 (최초 1회)
minikube addons enable ingress
```

모든 Pod가 Running 상태가 될 때까지 대기합니다.

터미널 **Pod 상태 확인**

```
kubectl get pods -n metacoding
```

kubectl get pods -n metacoding

```
NAME                                READY   STATUS    AGE
kafka-deploy-7d4c8b9f5-2xk9p        1/1     Running   2m
db-deploy-6f9b7c4d8-m4t2q           1/1     Running   90s
gateway-deploy-5c8d6f7b9-h7w3r      1/1     Running   88s
order-deploy-8b7f6c9d4-q2k8m        1/1     Running   85s
product-deploy-7c9d8b6f5-x4r2t      1/1     Running   83s
user-deploy-6d8c7b9f4-p3m9k         1/1     Running   80s
delivery-deploy-9f7c8b6d5-t6w2x     1/1     Running   78s
orchestrator-deploy-8c6f9b7d4-k9m4q 1/1     Running   75s
frontend-deploy-7b9c6d8f5-w3k2m     1/1     Running   72s
```

9개 Pod Running (frontend 추가)

그림 5-11. Pod 상태 확인

5.8.4 서비스 접근

외부에서 Ingress로 접속하기 위해 `minikube tunnel`을 실행합니다.

터미널 **외부 접근 터널**

```
minikube tunnel
```

터널이 실행되면 `http://127.0.0.1:80`로 프론트엔드에 접속할 수 있습니다.

5.8.5 통합 테스트 시나리오

Step 1: 웹소켓 연결 및 주문 생성

브라우저로 index.html에 접속합니다. 그다음 로그인 API(`POST /login`)에서 발급받은 JWT 토큰을 입력하여 웹소켓을 연결합니다.

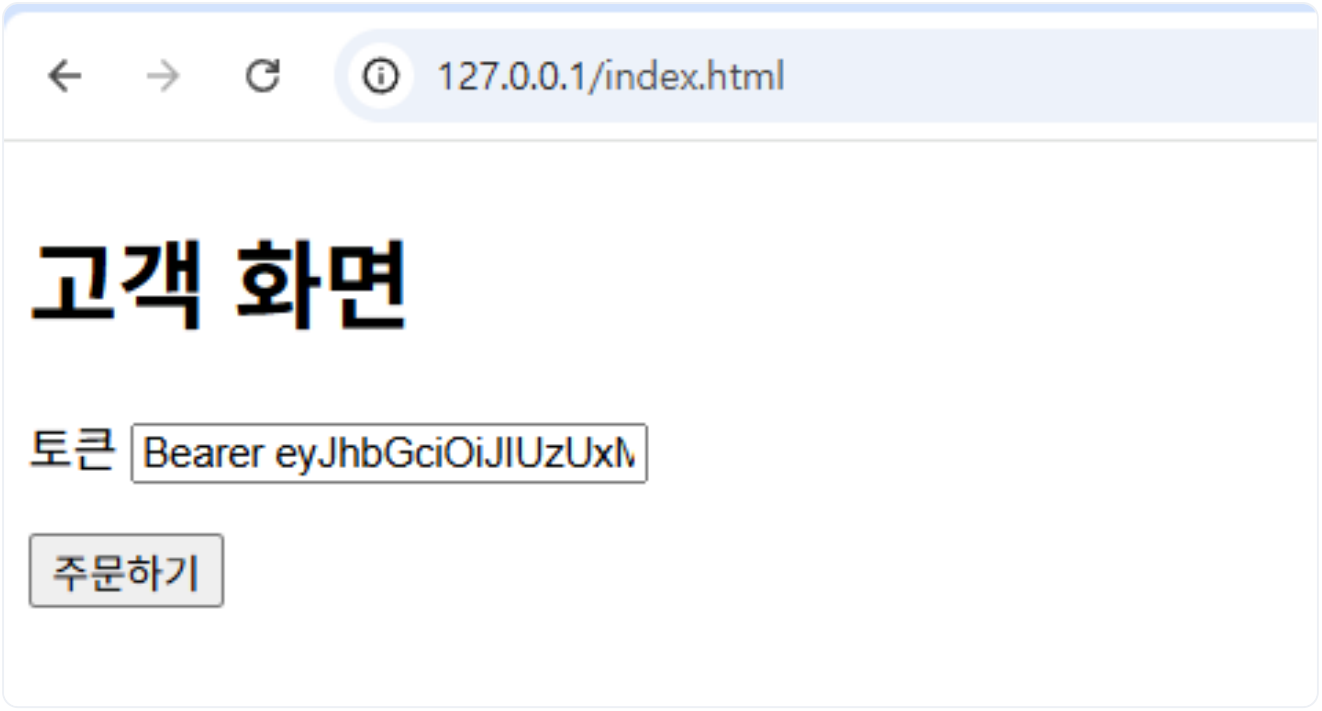


그림 5-12. 브라우저에서 index.html 접속 화면

주문하기 버튼을 클릭합니다. 그러면 index.html이 웹소켓에 연결하고 `/topic/orders/{userId}` 채널을 구독합니다.

자동 전송 **index.html이 보내는 주문 요청**

```
POST /api/orders

{
  "productId": 1,
  "quantity": 1,
  "price": 2500000,
  "address": "Addr 4"
}
```

이 요청은 주문하기 버튼을 누르면 index.html이 자동으로 보내므로, 직접 호출하지 않아도 됩니다.

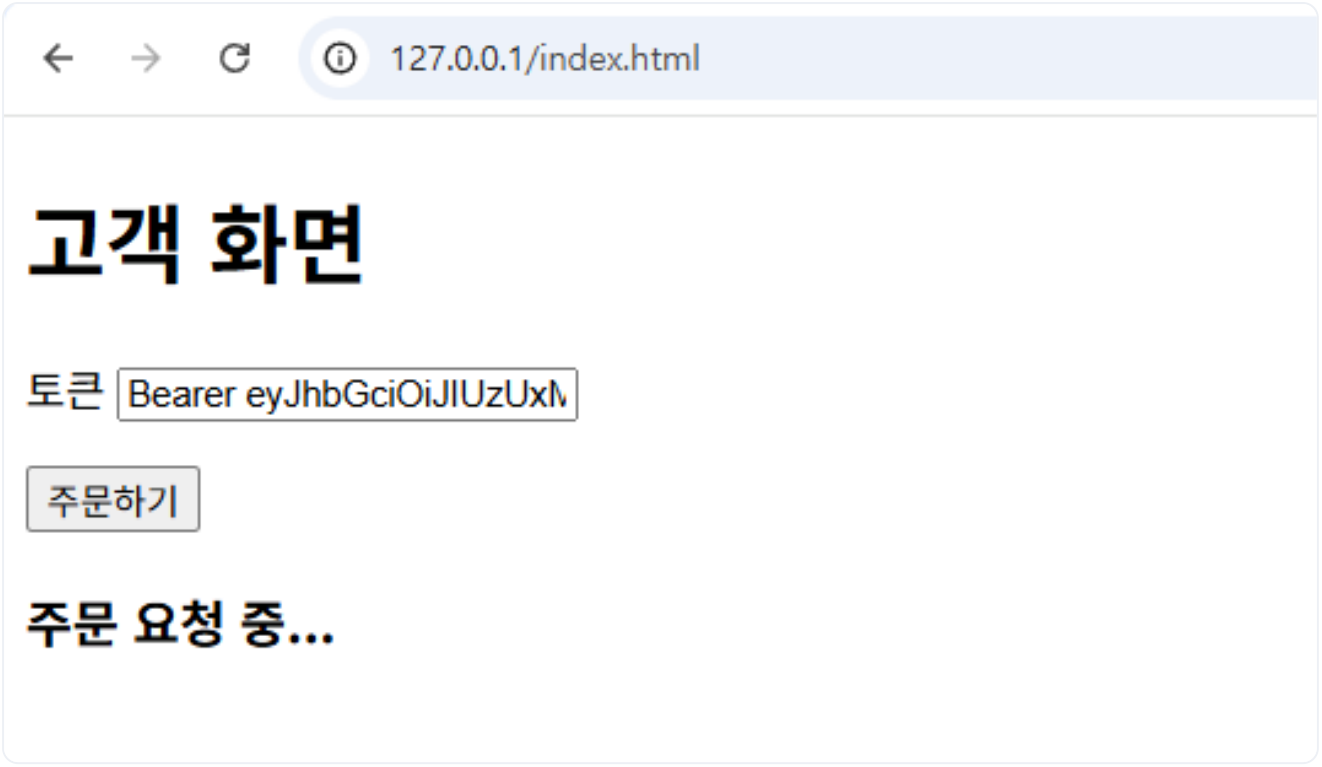


그림 5-13. 토큰 입력 후 주문하기 버튼 클릭

브라우저 `F12` - `Console`에서 웹소켓이 연결됨을 확인할 수 있습니다.

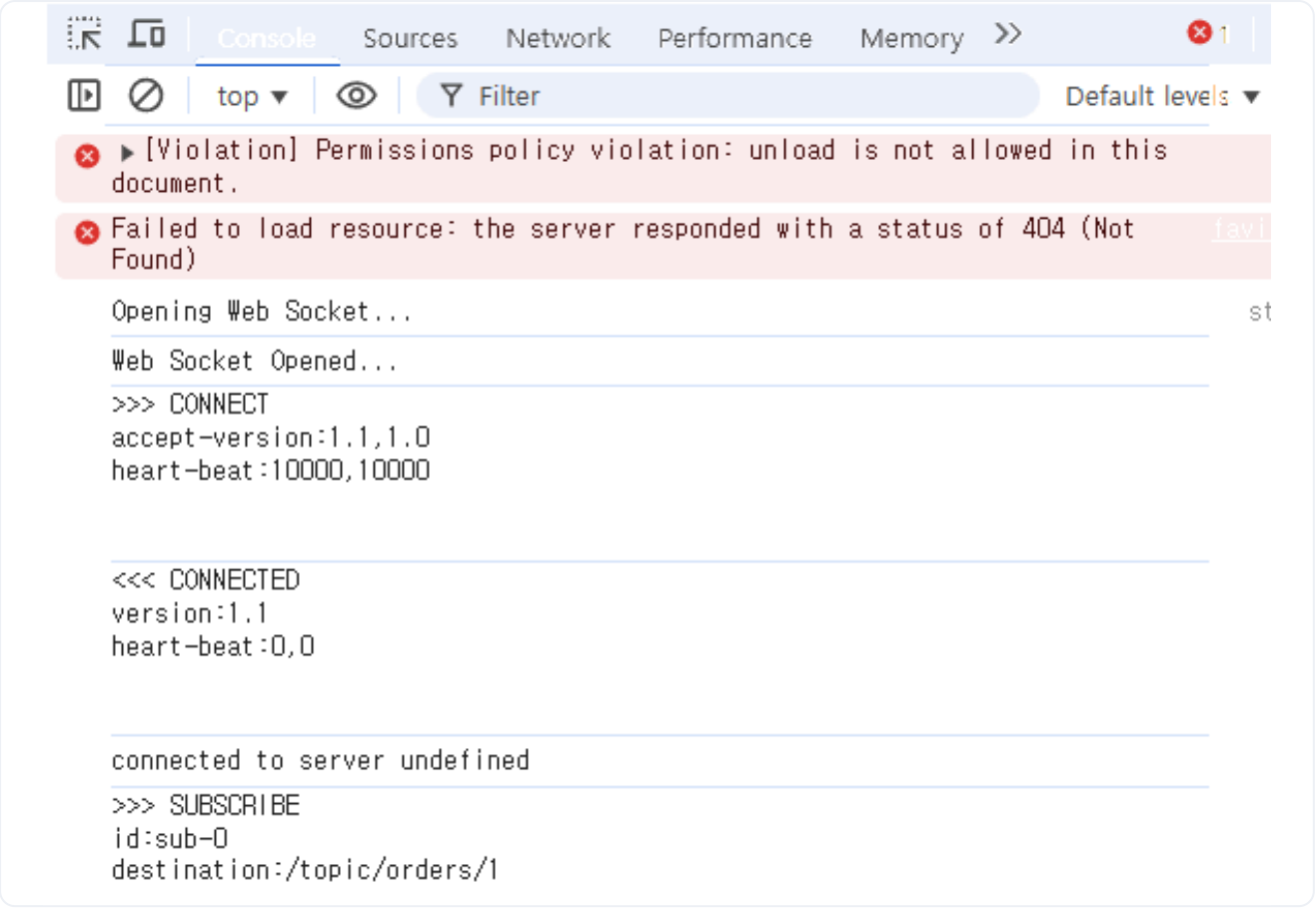


그림 5-14. 브라우저 Console에서 웹소켓 연결 확인

Hoppscotch로 생성된 주문을 확인하면 `PENDING` 상태로 머물러 있습니다.

Hoppscotch **주문 조회**

```
GET http://127.0.0.1:80/api/orders/4
```

상태: 200 • OK 시간: 77 ms 크기: 330 B

JSON Raw 헤더 4 테스트 결과

응답 본문


```
{
  "status": 200,
  "msg": "성공",
  "body": {
    "id": 4,
    "userId": 1,
    "productId": 1,
    "quantity": 1,
    "price": 2500000,
    "status": "PENDING",
    "createdAt": "2026-05-19T07:46:36",
    "updatedAt": "2026-05-19T07:46:36"
  }
}
```


그림 5-15. 주문 조회 결과 - PENDING 상태

Step 2: 배달 완료

먼저 생성된 배달을 확인해보겠습니다.

Hoppscotch **배달 조회**

```
GET http://127.0.0.1:80/api/deliveries/4
```



그림 5-16. 배달 조회 결과 - PENDING 상태

배달 ID가 4인 배달이 `PENDING` 상태로 생성되었습니다.

배달 완료 API를 호출해 완료 처리를 합니다.

Hoppscotch 배달 완료 호출

```
PUT http://127.0.0.1:80/api/deliveries/4/complete
```

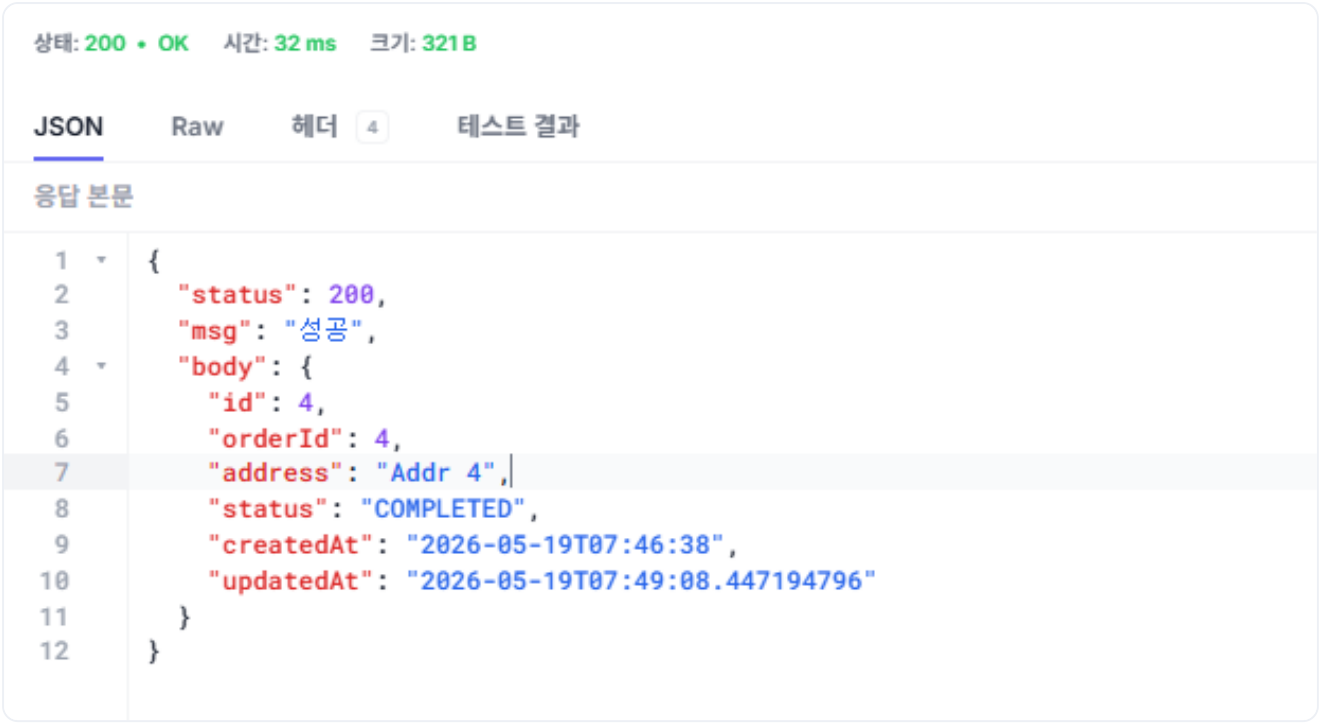


그림 5-17. 배달 완료 API 호출 결과 - COMPLETED 상태

배달 완료 버튼을 눌렀다. 이제 주문도 바뀌었을까?

Step 3: 주문 완료 및 웹소켓 응답 확인

배달 완료 처리 후, 주문 완료 명령으로 주문이 `COMPLETED` 상태가 됩니다.

Hoppscotch 주문 조회

```
GET http://127.0.0.1:80/api/orders/4
```



그림 5-18. 주문 조회 결과 - COMPLETED 상태

주문이 완료되면 웹소켓이 클라이언트에게 주문 완료 메시지를 전송합니다. 웹소켓 응답을 수신하면 클라이언트 화면이 주문 완료 상태로 변경됩니다.



그림 5-19. 웹소켓 알림 수신 - 클라이언트 화면에 주문 완료 표시

완성한 화면을 동료에게 보여 줬습니다. 주문하기를 누르자 잠시 뒤 '처리 중'이 '주문 완료'로 바뀌었습니다.

동료 : "이제 사용자도 새로고침 없이 주문이 끝난 걸 바로 알 수 있겠네요."

하나로 구성됐던 서비스를 기능별로 분리하고, 사용자에게 실시간으로 알리는 웹소켓까지 더했습니다. 이제 한쪽 서비스에 부하가 몰려도 전체가 멈추지 않고, 주문이 끝나는 순간 사용자 화면에 완료가 바로 표시됩니다.

이것만은 기억하자

- → **폴링**은 클라이언트가 변화를 계속 확인해야 하지만, **웹소켓**은 서버가 변화 순간 먼저 알립니다.
- → 배달 생성과 배달 완료를 분리해, 주문 완료를 **실제 배달이 끝난 시점**에 맞춥니다.
- → 주문이 완료되면 주문 서비스가 **웹소켓**으로 사용자에게 실시간으로 알립니다.

에필로그. 다시, 트래픽이 몰리던 날

그 후로 몇 달이 지났습니다.

대규모 할인 행사가 열린 금요일 저녁 일곱 시. 예전 같았으면 쏟아지는 트래픽에 시스템 어딘가가 멈춰서 복구하느라 정신없었을 시간입니다.

하지만 이번에는 주말 내내 휴대폰이 조용했습니다. 한쪽 서비스에 부하가 몰려도 장애가 시스템 전체로 번지지 않도록 구조를 바꾼 덕분입니다. 서버 장애 알림 없이 주말을 온전히 쉰 건 참 오랜만이었습니다.

월요일 아침, 팀장님이 지나가며 물었습니다.

팀장 : "이번에 트래픽 엄청나던데, 주말에 별일 없었나 봐요?"

오픈이 : "네, 별일 없었습니다."

주문이 몰린다고 서버 전체가 다운되던 일은 이제 과거가 되었습니다.

물론 사이트가 발전하면 새로운 서비스가 추가될 테고, 시스템이 커지는 만큼 또 어딘가에서 예상치 못한 문제가 생길 수 있습니다.

하지만 이제는 전처럼 막연하게 걱정되지 않습니다. 시스템을 직접 분리하고 연결하며 문제를 해결해 본 경험이 있으니, 어디가 막히든 원인을 찾고 다시 개선해 나가면 되기 때문입니다.

마치며

하나의 모놀리식 서비스가 여러 갈래로 나뉘고, 그 기반에 쿠버네티스와 카프카가 더해지기까지의 과정을 함께 짚어봤습니다. 우리가 지나온 길은 유행하는 기술을 하나씩 추가하는 숙제가 아니었습니다. 당면한 문제를 풀기 위해 시스템의 경계를 넓히고, 코드를 바꾸며 아키텍처를 진화시켜 나간 과정이었습니다.

동기에서 비동기로, 하나에서 여럿으로 시선을 옮기는 일이 처음에는 복잡하고 막막했을지도 모릅니다. 하지만 거대해 보이던 마이크로서비스 아키텍처(MSA)도 결국은 작은 문제를 하나씩 풀어간 고민들이 모여 만들어집니다. 처음 'MSA'라는 단어 앞에서 막막하던 때와 지금의 여러분은 분명 다릅니다.

직접 만든 시스템에는 아쉬운 구석이 남기도 합니다. 그건 부족함이 아니라, 다음 걸음이 보이기 시작했다는 신호입니다. 앞으로도 막히는 날은 옵니다. 그때 처음부터 다 알아야 한다고 자신을 몰아세우지 마세요. 부딪히고, 고치고, 다시 만들면 됩니다. 좋은 구조를 고민하고 코드를 바꾸는 일은 결국 우리 같은 개발자의 몫이니까요.

이제 책장을 닫고 코드로 돌아갈 시간입니다. 이 책에서 다룬 기술과 힌트가, 앞으로 마주할 수많은 트래픽과 서비스 앞에서 작은 이정표가 되기를 바랍니다.

이제 여러분 차례입니다.